



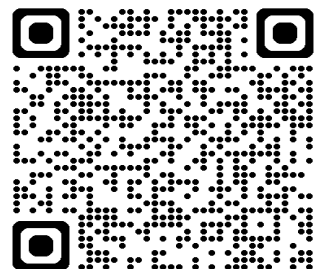
Agrosnap

Plant diseases detection

Team Members :

**ASEEL KHANFAR - TALA
SHARAIREH**

Supervised By : Dr. Sawsan Abu Shuqeir



1.Introduction

What is Agrosnap ?

This project is an AI-powered web application designed to detect five types of plant diseases from uploaded plant images. Once a disease is identified, the system generates a detailed report containing the disease name, its symptoms, and the recommended treatments. The application also supports offline functionality, allowing users to access and use it without an internet connection. This feature makes the system especially useful for farmers and agricultural workers in remote areas where Internet access is limited or unavailable.

The main goal of this project is to assist farmers in detecting plant diseases quickly and accurately while providing essential information that supports timely treatment and helps reduce crop losses.

2.Objectives

- ✓ Develop a user-friendly and intuitive system.
- ✓ Provide offline access to the system's features.
- ✓ Generate detailed reports containing comprehensive disease information.
- ✓ Store generated reports, allowing users to access them whenever needed.

3. Technologies Used

Categories	Technology	Used for
Programming Language	Python	Backend development and AI model development
Programming Language	Javascript	Client-side logic and interactivity
Web Technologies	HTML	Structure of the user interface
Web Technologies	CSS	Styling and layout
AI / Machine Learning	Tensorflow	Used to build, train, and evaluate the deep learning model.
Backend Framework	FastAPI	Building REST API endpoints and handling client-server communication
AI Deployment	Tensorflow.js	Running the trained AI model directly in the browser
AI Model	EfficientNetB2	Plant disease classification model
Database	SQLite3	Server-side persistent data storage
Client-Side Storage	IndexedDB	Local storage of reports and application data for offline use

3. Technologies Used - continue

Categories	Technology	Used for
Authentication	JWT	User authentication and session management.
Programming Language	JWT - decode	Client-side logic and interactivity
PWA Technologies	Service Worker	Caching resources and supporting offline functionality
PWA Technologies	Web App Manifest	Making AgroSnap installable as a PWA
Development / Deployment	Git & GitHub	Source-code version control and project hosting
JWT Library	jwt-decode	Decoding JWT payload information on the client side
IndexedDB Library	idb	Simplifying interaction with IndexedDB
Deployment / Tunneling	ngrok	Exposing the local development server through a public HTTPS URL
Deployment / Tunneling	Cloudflare Tunnel (trycloudflare)	Providing a public HTTPS URL for local development and testing
Deep Learning Framework	Keras (tensorflow.keras)	Provides a high-level interface for building and training neural network models using TensorFlow.

3. Technologies Used - continue

Categories	Technology	Used for
Numerical Computing	NumPy	Used for numerical operations and handling image data and prediction results as arrays.
Model Evaluation	Scikit-learn (sklearn.metrics)	Used to evaluate the model using metrics such as the confusion matrix and classification report.
Data Visualization	Matplotlib	Used to create plots for visualizing training and validation results.
Data Visualization	Seaborn	Used to create statistical visualizations, such as the confusion matrix heatmap.
Development / Training Environment	Google Colab	Used as the development environment for training and evaluating the AI model.
Web API	Web Worker	Runs AI model processing in the background to keep the user interface responsive.
Cloud Platform / Hosting	Render	Hosts and deploys the AgroSnap backend and web application online.

4.Requirements Analysis

❖ Functional

1. The system shall allow users to register and create an account.
2. The system shall allow users to log in using their registered credentials
3. The system shall allow users to log out of their accounts.
4. The system shall have a profile page.
5. The system shall maintain the user's authenticated session across subsequent visits after a successful login.
6. The system shall allow users to upload plant leaf images one image at a time.
7. The system shall analyze the uploaded plant leaf image and generate a corresponding disease report.
8. The system shall display the predicted disease and its confidence score in the generated report.
9. The system shall allow users to save generated reports and assign a custom name to each report.
- 10.The system shall allow user to delete any saved report.
11. The system shall allow users to view previously saved reports.
- 12.The system shall allow users to access saved reports while offline.
- 13.The system shall allow users to use AI detection while offline.
- 14.The system shall synchronize locally stored data with the main database when an Internet connection is restored.
- 15.The system shall be accessible as a web application and installable as a Progressive Web Application (PWA).

❖ Non-functional Requirements

1. The performance & Speed

1. **(inference time):** The deep learning model shall process an uploaded plant leaf image and return classification results within **2.0 seconds** on standard device hardware.
2. **(API Latency):** The FastAPI backend shall process authentication and database query requests in under **200 milliseconds** under a concurrent load of up to **50 active users**.
3. **(GUI Responsiveness):** The Single Page Application (SPA) user interface shall maintain a smooth rendering frame rate and respond to user clicks/touches within **100 milliseconds**.

2. Availability & Offline-First Operation

1. **(Offline Capability):** The client-side application shall allow users to view cached disease (that user fetch without internet), past report locally via **Indexed DB** without requiring an active internet connection.
2. **Safe Saving:** If a save operation fails halfway through, the system must automatically cancel it to keep data clean.
3. **Smart Retry:** If internet drops during sync, the app must keep the report safe in local storage and try again later.

3. Reliability & Data Protection

1. **Data Linking:** The database must correctly link users, diseases, and saved reports so no data gets lost or corrupted.
2. If a save operation fails halfway through, the system must automatically cancel it to keep data clean.
3. **Smart Retry:** If internet drops during sync, the app must keep the report safe in local storage and try again later.

4. Security

1. **Password Safety:** User passwords must be encrypted (hashed using `bcrypt`) before saving to the database.
2. **Secure Access:** Users must be logged in with a secure token (JWT) to save or view personal reports.

5. Compatibility

1. **Screen Fitting:** The application design must work smoothly on mobile phones, tablets, and computers.

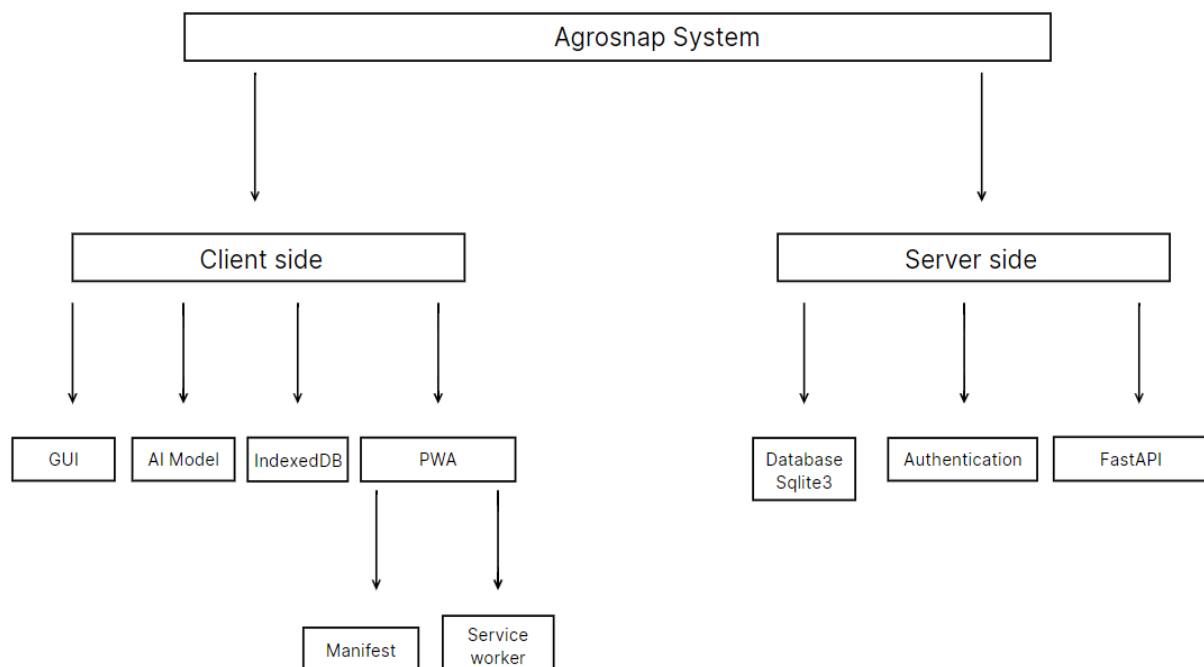
6. System architecture

1- Client side [front-end]

- Graphical user interface - GUI
- Tensorflow.js [AI Model]
- IndexedDB
- Progressive web application - PWA
 - ✓ Manifest
 - ✓ Service Worker

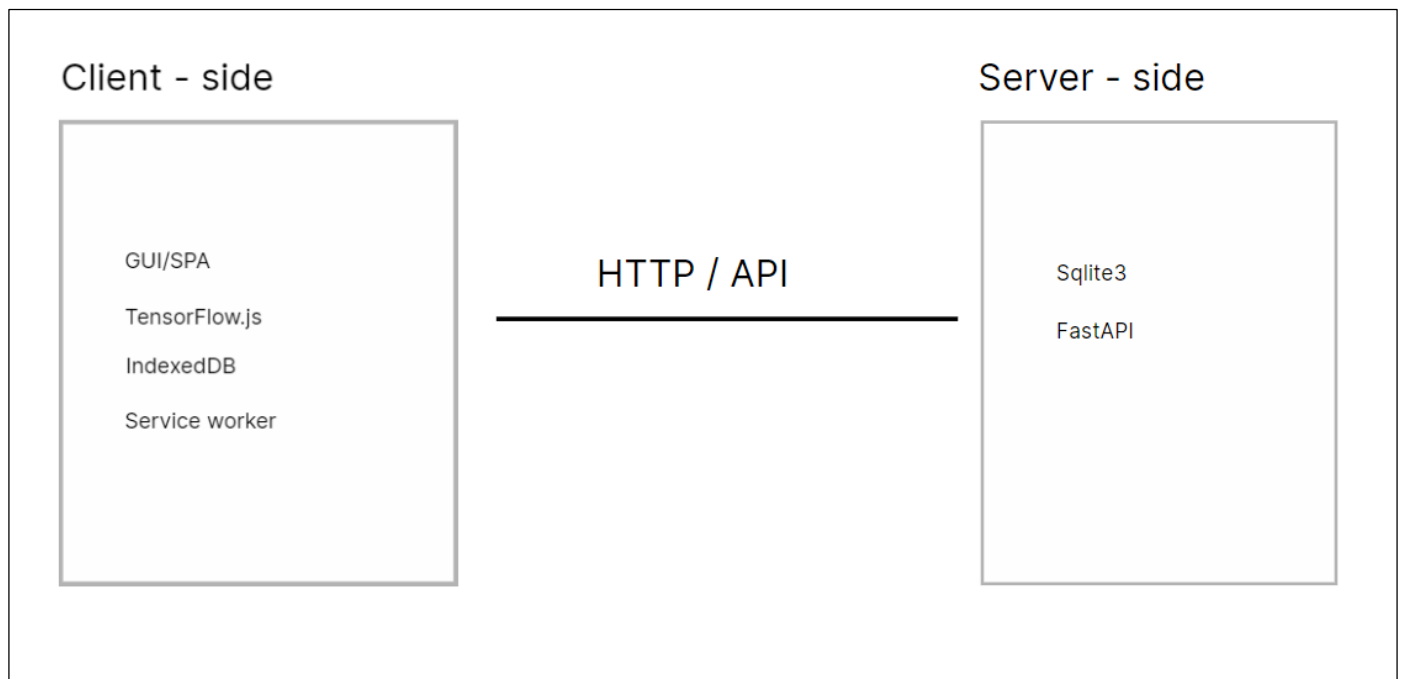
2- Server side [back-end]

- Sqlite3
- FastAPI
- Authentication



6.1 Architecture overview

The AgroSnap system follows a client-side architecture supported by a backend server. The client side contains the graphical user interface, authentication pages, the main single-page application, the AI model, local data storage, and the service worker. The backend consists of a FastAPI server connected to a SQLite3 database. The system is designed to perform disease prediction locally in the browser while using the backend for data retrieval, authentication, and synchronization when an Internet connection is available.

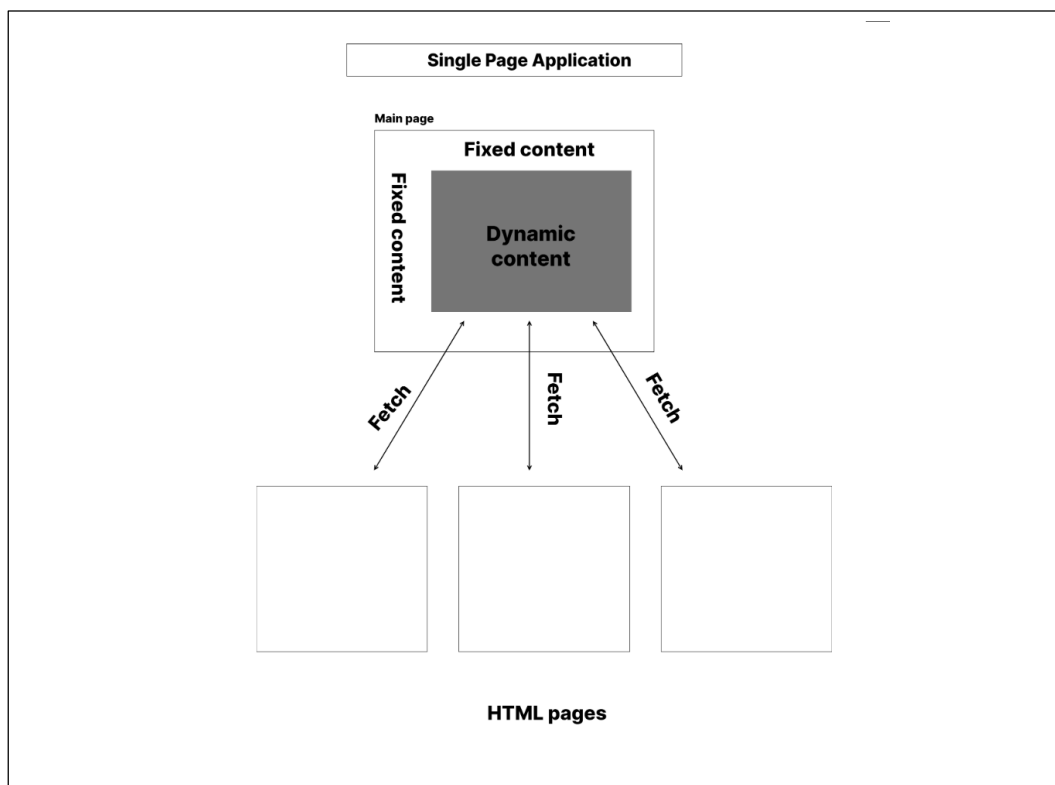


6.2 Client - side architecture

6.2.1 Graphical user interface - GUI

The Graphical User Interface (GUI) was designed to provide a clear, intuitive, and comfortable interface that facilitates user interaction with the AgroSnap system. The interface organizes the system's functionality in a simple and accessible manner to make navigation and interaction easier for users. The application uses a Single Page Application (SPA) architecture to provide smoother navigation between the main application pages and reduce unnecessary full-page reloads, contributing to a faster and more responsive user experience.

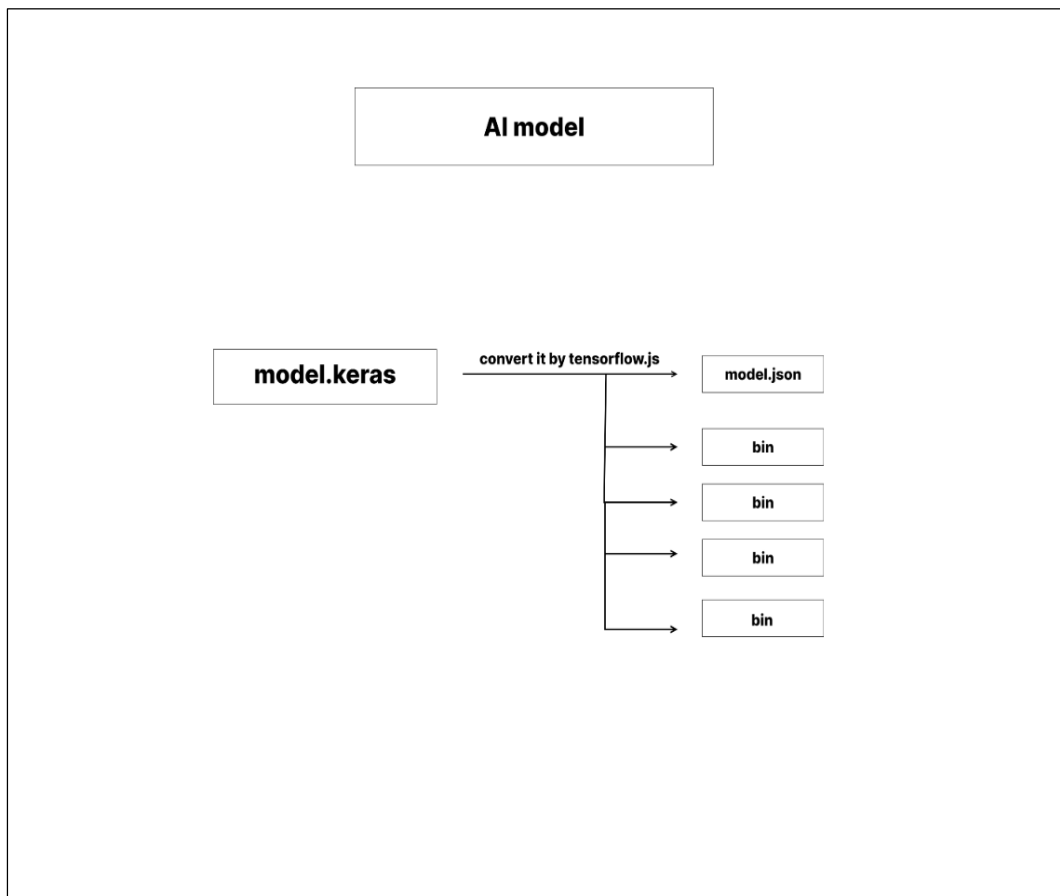
The GUI also acts as the primary interaction layer between the user and the client-side components of the system, including the AI model, local storage, and report management functionality.



6.2.2 AI model – Tensorflow.js

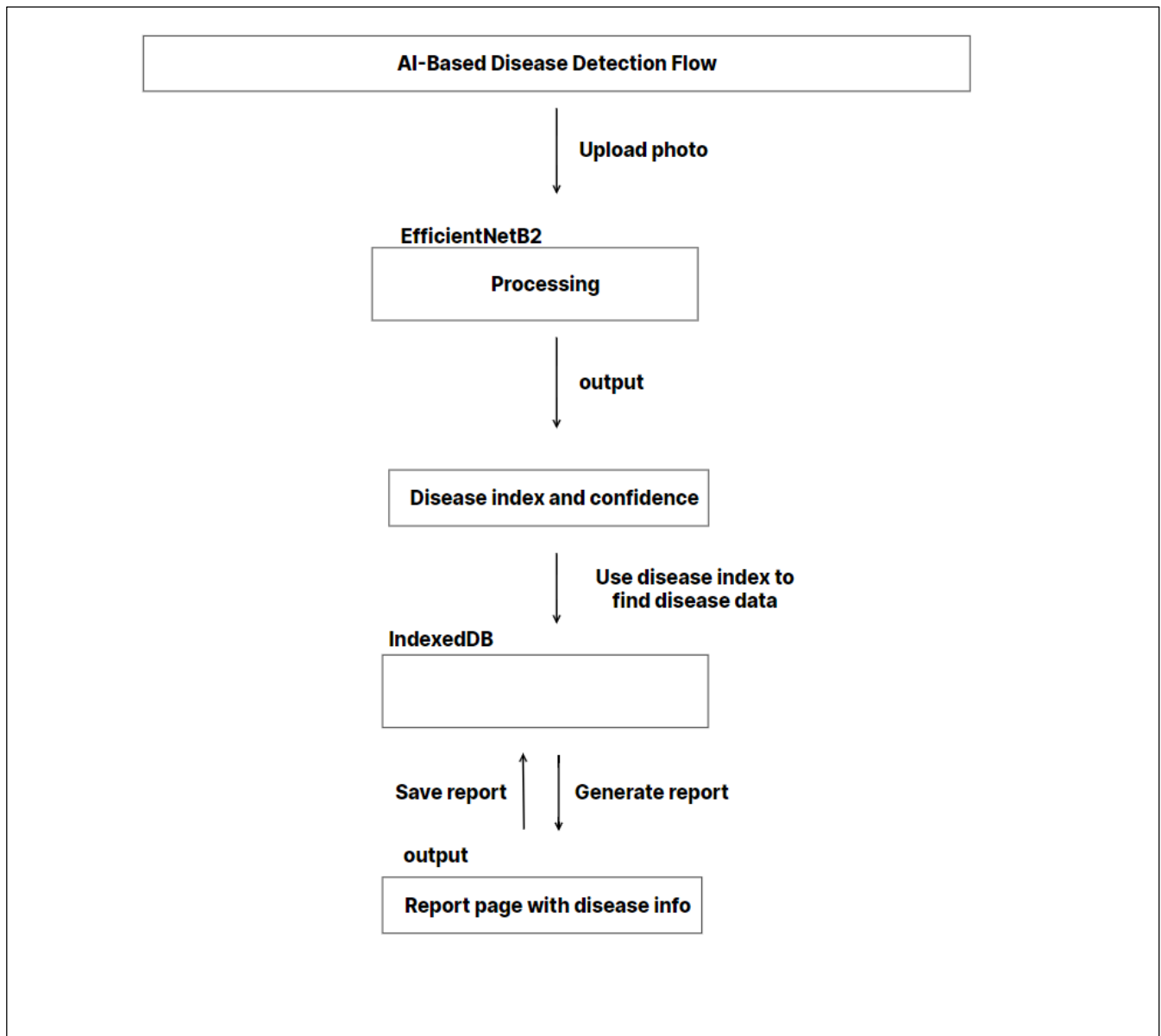
The AI model is integrated into the client side of the AgroSnap system and is executed directly within the browser using TensorFlow.js. The model receives the uploaded plant image as input and performs the disease prediction locally without requiring the image to be sent to the backend server. The prediction result is then used by the SPA to retrieve the corresponding disease information from IndexedDB and display the results to the user.

The trained Keras model was converted into TensorFlow.js format to enable its execution directly in the browser.



6.2.2 AI model – Tensorflow.js – continue

The prediction workflow begins when the user uploads a plant image. The EfficientNetB2 model processes the image and produces a disease ID along with its confidence score. The disease ID is then used to retrieve the corresponding disease information from IndexedDB, which is used to generate and display the final disease report.



6.2.3 IndexedDB

IndexedDB is a browser-based NoSQL database used for storing application data locally on the user's device. In AgroSnap, it is used to store data retrieved from the server-side database, allowing the application to access the required information without continuously relying on an Internet connection. This supports the application's offline functionality by allowing users to:

- 1- **Access required data while offline**, such as saved reports and personal user data.
- 2- **Store newly created reports locally** until an Internet connection becomes available, after which the data can be synchronized with the server.

Browser



6.2.4 Progressive Web Application - PWA

Progressive Web Application (PWA): AgroSnap is implemented as a Progressive Web Application (PWA) to provide a web application experience with support for offline access and locally cached resources. The PWA allows the application to remain accessible and provide supported functionality even when an Internet connection is unavailable. This is achieved through two main components: the **Web App Manifest** and the **Service Worker**.

➤ **Web App Manifest**

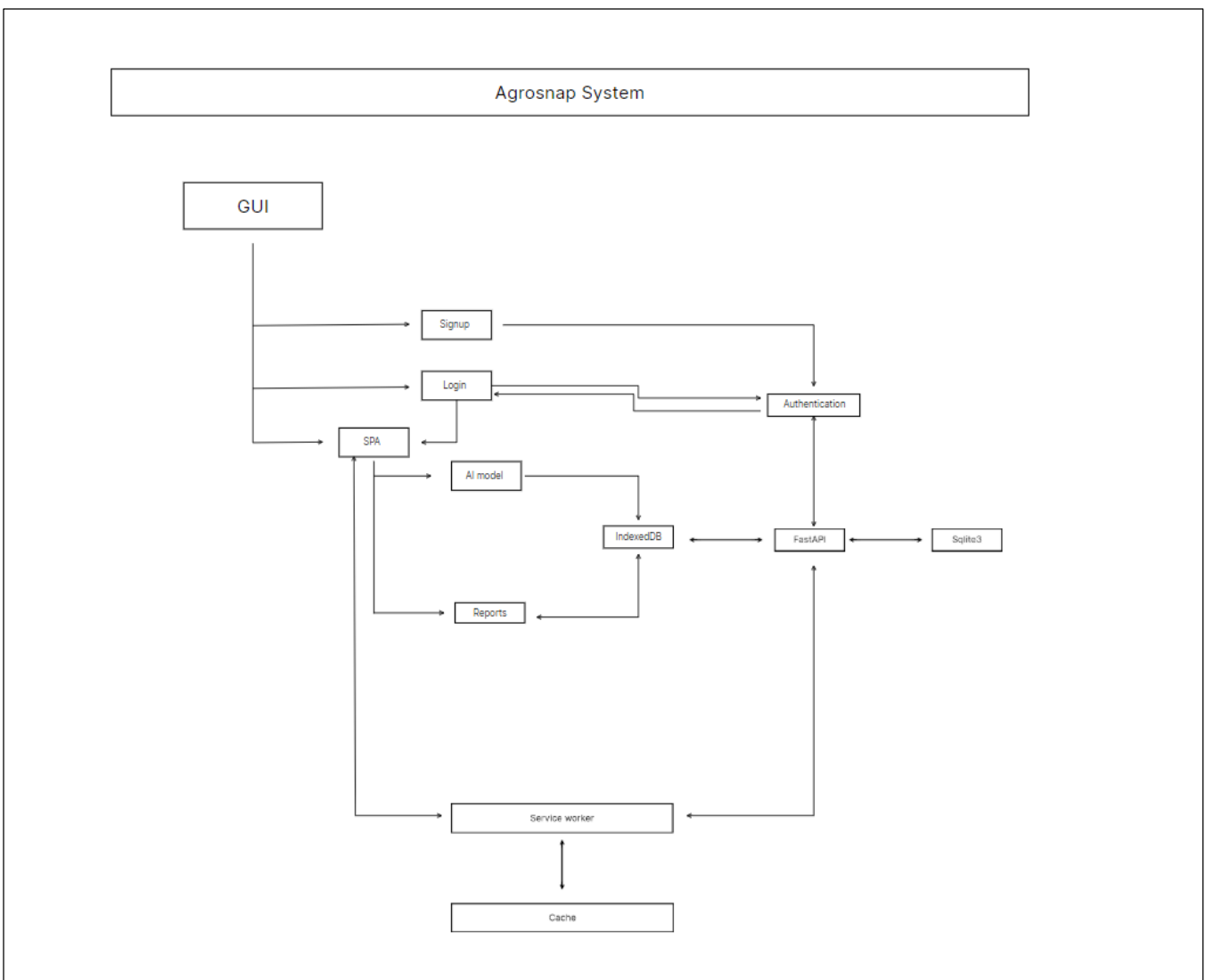
The Web App Manifest defines the application's metadata and configuration, such as its name, icons, theme, and display behavior. It allows AgroSnap to be recognized and launched as an installable web application, providing a more app-like experience for users.

➤ **Service Worker**

The Service Worker operates in the background and manages network requests and cached application resources. It allows previously cached resources to be served locally when they are available, reducing the application's dependency on the network and supporting offline operation.

6.3 Offline & Synchronization Architecture

AgroSnap supports offline operation by using IndexedDB to store application data and newly generated reports locally on the user's device. When an Internet connection is unavailable, the data remains stored locally and the supported application functionality can continue to operate. Once the connection is restored, the locally stored data is synchronized with the backend through the FastAPI API and persisted in the SQLite3 database.



6.4 Server - Side Architecture

6.4.1 System component & Structure

- **database.py:** Manages the SQLite3 connection lifecycle, handles session creation, and defines the context manager for database operations.
- **database.db:** The single, embedded disk file storing all persistent relational tables.
- **schemas.py:** Contains Pydantic models to validate API requests and serialize backend responses.
- **auth.py:** Contains FastAPI authentication routes (`/signup`, `/login`), password hashing, and JWT token issuance.
- **report.py:** Contains API routes to generate, fetch, save, and delete disease reports.
- **Sync.py:** Contains API Function to synchronization the data between database and indexed.DB .

6.4.1 Data Flow Execution Sequence

- **Request & Payload Validation:** Client calls an API endpoint (`auth.py` or `report.py`). Incoming JSON payloads are parsed and validated via Pydantic classes defined in `schemas.py`.
- **Authentication Guard:** Endpoints extract and decode the JWT payload to confirm user identity and authorization (`user_id`).
- **Database Connection:** The route calls dependency routines in `database.py` to open a database transaction.
- **SQL Execution & Persistence:** Queries run against `database.db` (e.g., verifying credentials in `USERS_INFO` or committing entries into `SAVE_REPORT`)
- **Response Handling:** Validated output models from `schemas.py` return formatted JSON responses to the client UI or offline sync handler.

FastAPI & Backend Layered Architecture Diagram



6.5 Database Architecture

6.5.1 Definition & Technical Overview

- **Formal definition of the database**

“Sqlite3 is C-language library (Embedded library/Code library) that implements inside app, server-less, Zero-configuration, transection depending on SQL database engine unlike traditional database system such as(MYSQL) that is run as separate background server processes, sqlite3 reads and write directly to disk files, the entire database- including tables, indices, trigger, and schema definitions- is stored in a single cross-platform file”.

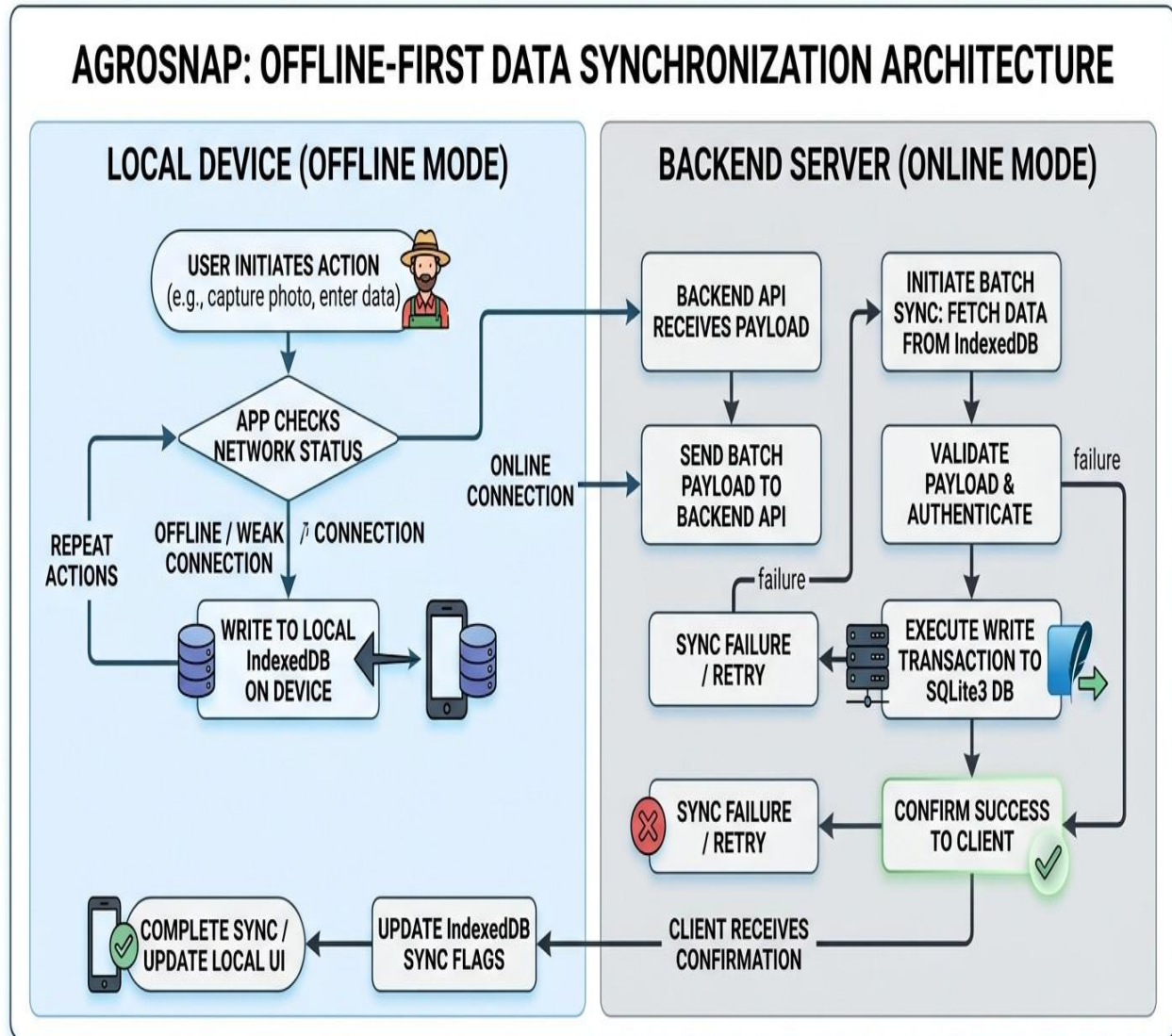
6.5.2 Key Properties & Rationale for

- **Embedded & Server-less Architecture:** reduces operational complexity because there is no separate database process to install.
- **zero-configuration Setup:** Eliminates user management, port binding, or network configuration overheads.
- **Lightweight Footprint:** Requires minimal memory and CPU resources.

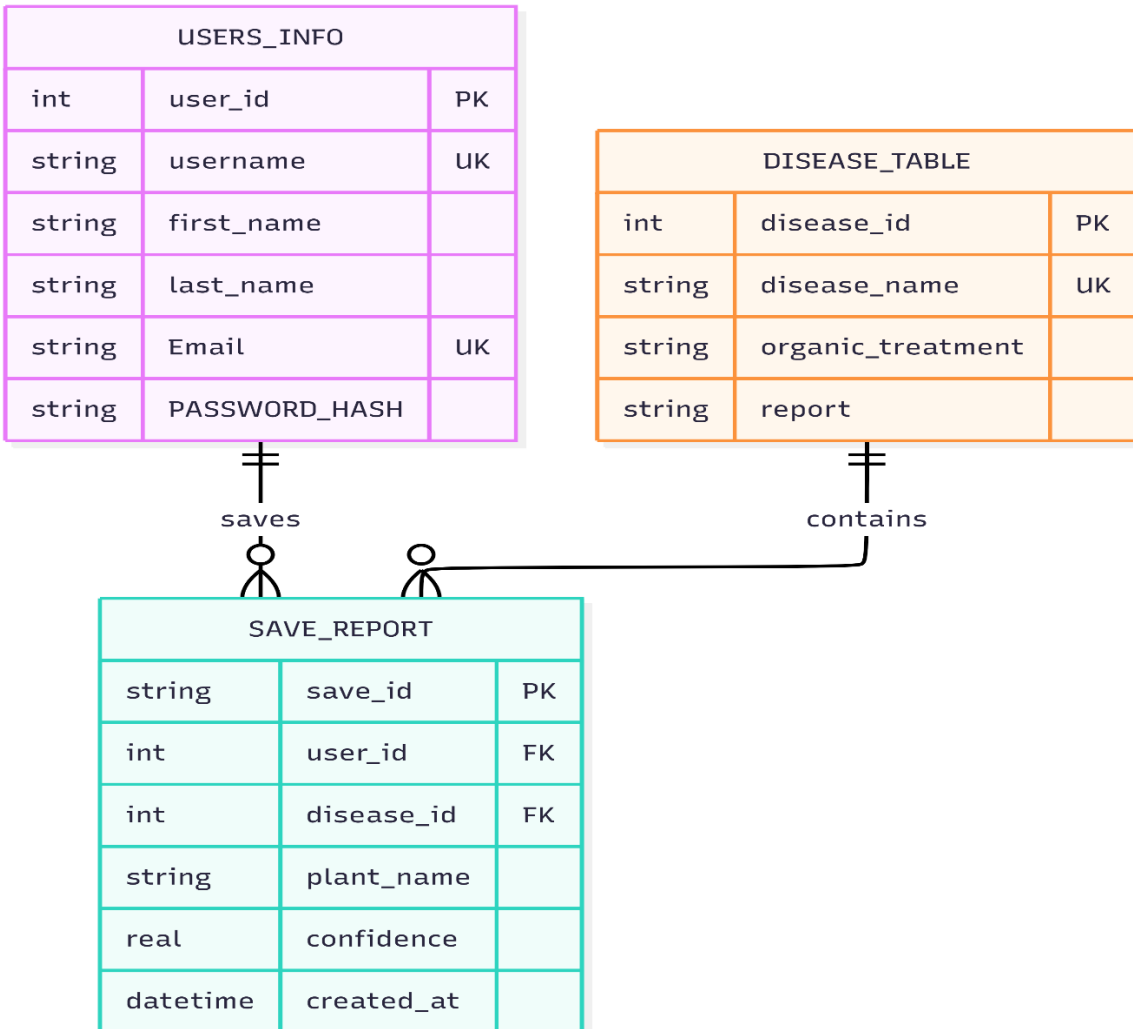
6.5.3 Data Flow Execution Steps

- **Local Capture (Offline Mode):** The farmer captures plant images or enters observational data in the field. The client application writes this data directly to **IndexedDB** on the device.
- **Connectivity Check:** The app continuously checks network status. If offline, data remains safely persisted inside IndexedDB and operations proceed seamlessly.
- **Batch Transmission (Online Mode):** Upon detecting an active internet connection, the app initiates a sync routine, retrieving queued local data from IndexedDB and sending it in a structured batch payload to the backend API.
- **Server Persistence:** The backend service validates the incoming payload and executes a write transaction to update the local **SQLite3** database instance, confirming completion back to the client to update local sync flags.

6.5.4 Data Flow Execution Flow-chart



6.5.5 Database Class Diagram



PK : PRIMARY-KEY

FK : FOREIGN KEY

UK : UNIQUE KEY

6.5.6 Database Schema

A.Introduction to the Schema

- **Users_info** : Holds registered user and profile details .
- **Disease_Table** : is read-only dictionary containing identified plant disease and their organic treatment .
- **Save_Report** : functions as a relational link table connecting users to their saved diagnosis report , storing the prediction metadata(confidence, plant_name,created_at)

B.Relational integrity & Foreign Keys

- **Users_info** to **save_report** (1:N) : one user can save zero ,one, or many reports(user_id as foreign key)
- **Disease_Table** to **Save_Report** (1:N):one disease entry can be referenced across multiple saved reports (disease_id as foreign key).

C.Data Dictionary Tables

Include detailed specification tables for each of your three entities:

➤ Users Table

DATABASE SCHEMA DOCUMENTATION

Table: Users (User Accounts Data)

Field Name	Data Type	Constraints	Description
 user_id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique identifier for each user
username	STRING	 UNIQUE, NOT NULL	Unique account handle
first_name	STRING	—	User's given name
last_name	STRING	—	User's surname
Email	STRING	 UNIQUE, NOT NULL	Registered contact email address
 PASSWORD_HASH	STRING	NOT NULL	Securely hashed user password

➤ Disease Table

DATABASE SCHEMA DOCUMENTATION

Table: PlantDiseases (Disease Information Data)

Field Name	Data Type	Constraints	Description
 disease_id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Primary key, uniquely identifying each plant disease record.
disease_name	STRING	 * UNIQUE, NOT NULL	The full name of the detected plant disease (e.g., 'Leaf Rust'). Must be unique and is mandatory.
organic_treatment	STRING	—	
report	STRING	—	A field for storing detailed reports or observations related to this disease.

➤ Save Reports

DATABASE SCHEMA DOCUMENTATION

Table: SavedReports (AI Diagnosis History)

Field Name	Data Type	Constraints	Description
 save_id	STRING / INT	PRIMARY KEY	Unique ID for the saved report record
 user_id	INTEGER	 FOREIGN KEY (USERS_INFO.user_id)	Link to the user who saved the report
 disease_id	INTEGER	 FOREIGN KEY (DISEASE_TABLE.disease_id)	Link to the identified disease
plant_name	STRING	* NOT NULL	Identified plant species
confidence	REAL	* NOT NULL	AI model classification probability score
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Timestamp of when report was saved

6.6.1 Schemas Layer Documentation

The **`schemas classes.py`** module defines data structures using **Pydantic**. It serves as the primary data validation and serialization layer between HTTP requests, backend routes, and database operations.

- **Request Payload Validation:** Enforces exact data types, required fields, and format rules for incoming API JSON bodies before they reach route handlers.
- **Password Strength Verification:** Incorporates custom validation routines (via `@field_validator`) directly within user creation schemas to ensure passwords meet security standards prior to hashing.
- **Response Serialization:** Formats database query results into structured JSON objects sent back to the client, preventing internal schema or sensitive fields (like password hashes) from leaking.

Password Validation & Schema Breakdown

- **User Signup:** Validates registration payloads. It contains custom field validation logic to check password complexity:
 - **Minimum Length:** Enforces at least 8 characters.
 - **Character Variety:** Ensures the inclusion of uppercase letters, lowercase letters, numbers, and special characters.
- **User Login:** Validates authentication payloads (`username/email` and `password`).
 - **Token:** Defines the payload returned upon successful login, including `access token` and `token type` (e.g., `bearer`).
 - **Token Data:** Maps decoded JWT token claims (such as `user_id` or `username`) for internal route dependency injection.
- **Report Schemas:**
 - **Report Create:** Validates incoming disease report payloads generated by the application.
 - **Report Response:** Defines the response structure returned to the client when reading or listing saved reports from `database.db`.

6.7.1 Authentication

Handles user authentication workflows. It exposes `/api/auth/signup` for account creation and `/api/auth/login` to authenticate credentials. Upon successful validation, it generates and signs a JSON Web Token (JWT) using a secret key, returning the access token to the client for session authorization.

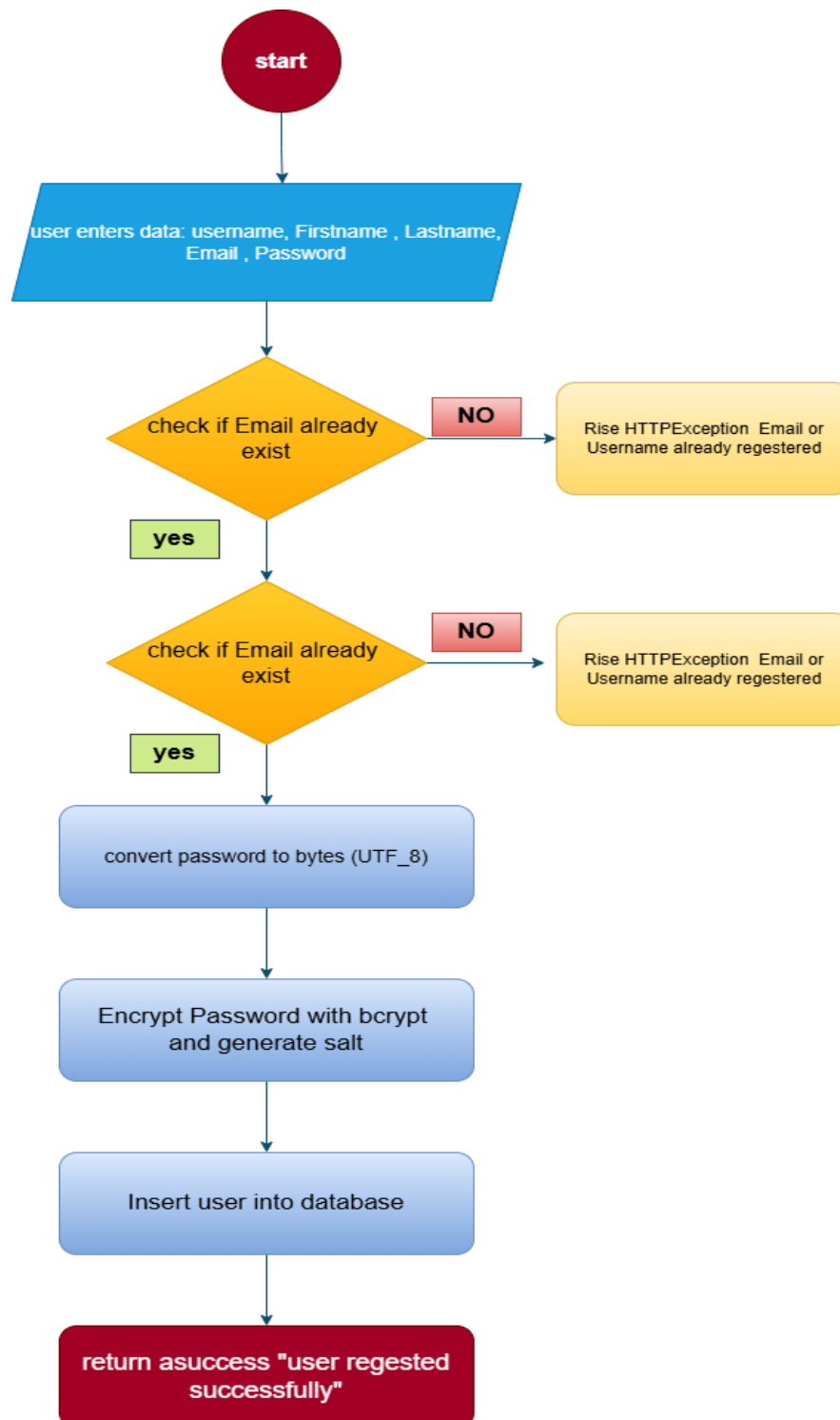
➤ Authentication Core Function

- **Signup (POST `/api/auth/signup`):** Receives user registration data via `schemas.py` validation. Hashes the user password using `bcrypt` and stores the credentials inside the **USERS_INFO** table in `database.db`.
- **Login (POST `/api/auth/login`):** Verifies the submitted username/email and compares the hashed password against the record in `database.db`.
Token Generation (`create_access_token`): Converts user identity payload schemas (`Data_of_Token`) into dictionaries via `.model_dump()`, attaches a UTC expiration timestamp (`exp`), and signs the token with `jwt.encode()` using `SECRET_KEY` and `ALGORITHM`.
- **Token Decoding Endpoint (POST `/decoding`):** Accepts raw token strings, decodes and verifies claims using `jwt.decode()`, and returns user identity fields (`user_id`, `username`, `Email`, `first_name`, `last_name`). It explicitly captures `ExpiredSignatureError` (raising 401 Token has expired!) and general `JWTErrors` exceptions (raising 400 Invalid Token).

➤ Token Verification & Protected Routes

- **Bearer Token Header:** The client includes the token in subsequent HTTP requests inside the `Authorization: Bearer <JWT_TOKEN>` header.
- **Dependency Injection (`HTTPBearer`):** Protected modules (such as `report.py`) use FastAPI dependencies to decode and verify the JWT signature.
- **Access Control:** If the token is valid and unexpired, the endpoint extracts the user identity (`user_id`) to process database operations; otherwise, it returns an HTTP 401 Unauthorized response.

6.7.2 Signup Flowchart



6.7.3 Login Flowchart



6.8 Report Management Subsystem Documentation

The **report.py** module handles all application logic for managing plant disease scan reports. It serves as the bridge between client scan actions, user authorization dependencies, and persistent storage in **database.db**.

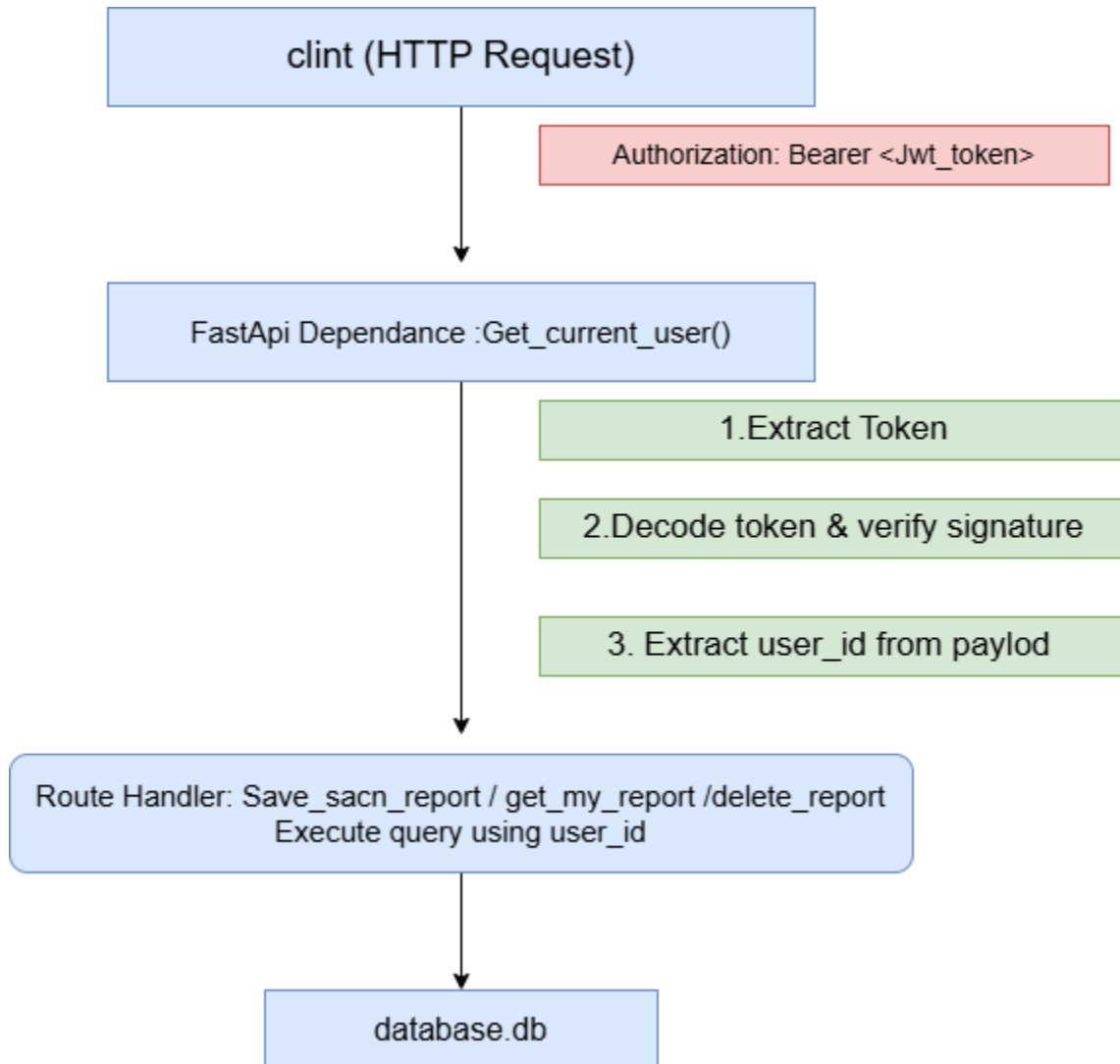
➤ 6.8.1 Authentication & Dependency Injection Layer Documentation

To keep route handlers clean and secure, the backend uses FastAPI's **Dependency Injection system (Depends)** to validate tokens and extract user identities before reaching endpoint logic.

➤ 6.8.1 Key Responsibilities `get_current_user()` :

- **Token Extraction:** Intercepts incoming HTTP requests and extracts the `Authorization: Bearer <token>` header.
- **Decoding & Verification:** Cryptographically decodes the JWT token using the secret key and verification algorithms.
- **Payload Parsing:** Extracts the `user_id` claim stored inside the token payload.
- **Route Context Injection:** Passes the validated `user_id` directly to downstream functions like `save_scan_report`, `get_my_reports`, and `delete_report`.

➤ Request Data Flow



w

➤ 6.8.1 Core Functions

- **save_scan_report** (POST /api/reports/save): Receives validated scan data (plant name, detected disease, confidence score, created_at ,) via **Pydantic** schemas. It extracts the authenticated user's ID from the JWT token and writes a new report record to the database.
- **get_my_reports** (GET /api/reports): Retrieves all historical scan reports associated specifically with the logged-in user. It filters records in **database.db** using the token's embedded **user_id** to ensure user data isolation.
- **delete_report** (DELETE /api/reports/{report_id}): Removes a specific scan record from the database. It verifies both the existence of the report and that the requesting user owns the record before executing the deletion query

➤ Security & Authorization Layer

All functions inside report.py are declared as protected routes. They rely on FastAPI dependency injection to extract and decode the Bearer JWT token from HTTP headers before execution:

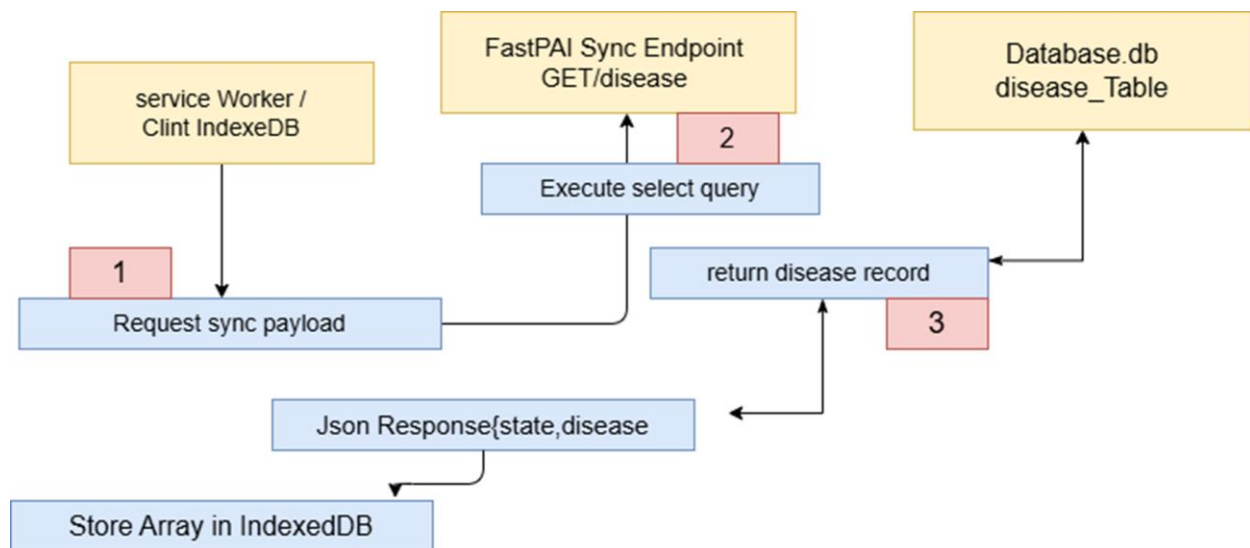
- **Token Authentication:** Unauthenticated requests missing a valid token are blocked immediately with an HTTP 401 `Unauthorized` status code.
- **Data Ownership Enforcement:** SQL operations for fetching and deleting reports explicitly enforce `WHERE user_id = :current_user_id` to prevent unauthorized cross-user data access.

8.9 Offline Data Synchronization Subsystem

To support offline functionality and fast client-side lookup within Progressive Web App (PWA) environments, the backend exposes the `sync_diseases_to_indexeddb` endpoint.

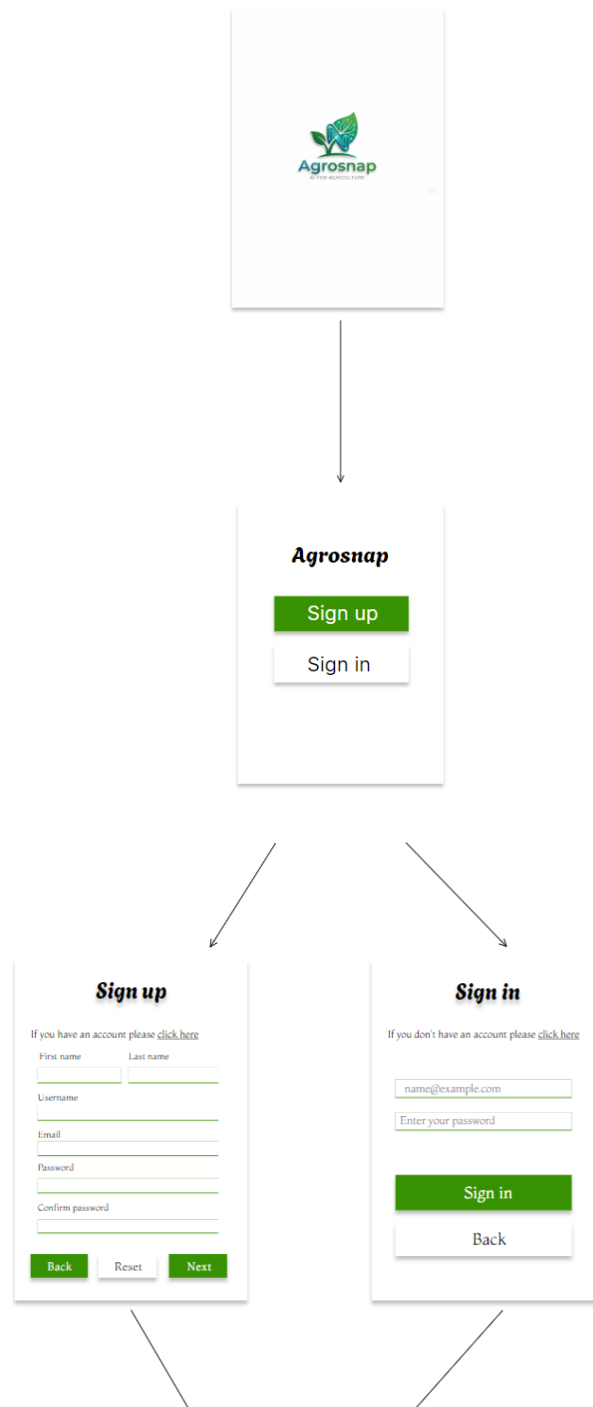
- **Purpose:** Provides client service workers with complete plant disease reference datasets (`disease_id`, `disease_name`, `organic_treatment`, `report`) to populate local browser storage (**IndexedDB**).
- **Data Extraction Logic:** Connects directly to `database.db`, executes a targeted query against `disease_Table`, and utilizes `sqlite3.Row` object mapping to guarantee standard key serialization.
- **Reliability & Exception Control:** Features robust connection lifecycle handling via `try-except-finally` execution blocks. Database-level exceptions are intercepted and re-thrown as structured HTTP 500 errors, while connection termination (`conn.close()`) is enforced to keep SQLite file locks minimal and thread-safe.

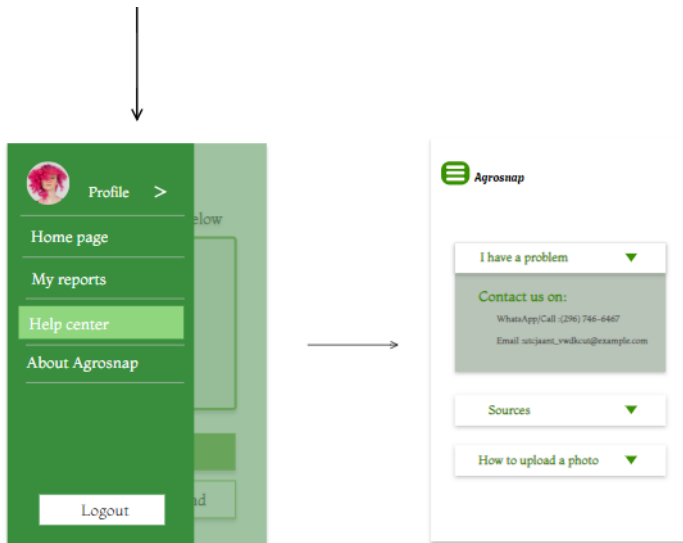
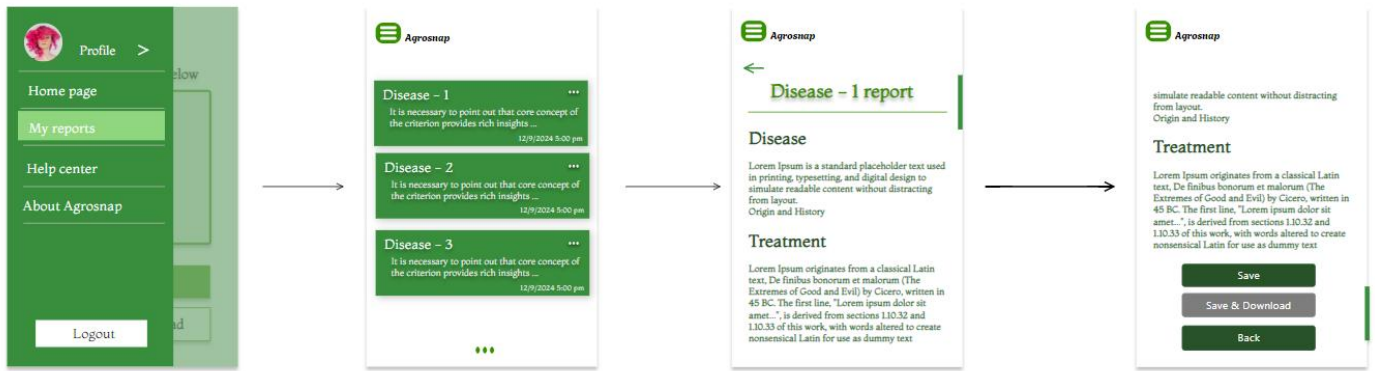
➤ Data Flow for Client Caching



7.UI/UX Design and User Flow

Navigation diagram







6.1 How to use the website

- 1- The user logs in or creates an account.
- 2- The user navigates through the application's main pages.
- 3- The user uploads a photo for leaf to detect if there any disease possible.
- 4- The system sends the data to the AI model for prediction.
- 5- The predicted disease and detailed report are displayed.
- 6- The user can save or download the generated report.
- 7- The report is stored in the database for future access.

8.Features

8.1 Authentication features

8.1.1 Log-in

Description :

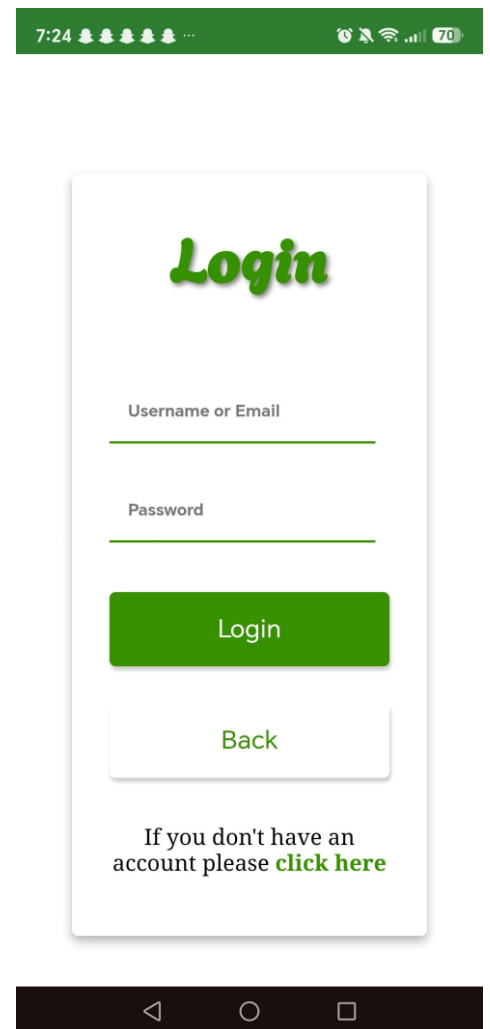
Users who already have an account can sign in using either their email address or username.

Purpose :

1. Authenticate registered users.
2. Provide secure access to personal data.
3. Allow users to access their saved reports and prediction history.

Main function:

1. Sign in using an email address or username.
2. Validate user credentials.
3. Redirect users to the home page after successful authentication.
4. Display an error message if the entered credentials are incorrect.



The image shows a mobile application interface for a login screen. At the top, there is a status bar with the time 7:24, several notification icons, and battery/charging indicators. The main content area has a white background with a green border. The word "Login" is displayed in a large, bold, green font. Below it are two input fields: "Username or Email" and "Password", each with a green underline. A green "Login" button is positioned below the password field. Below the button is a "Back" link in green text. At the bottom, there is a message: "If you don't have an account please [click here](#)". The bottom of the screen features a black navigation bar with three white icons: a back arrow, a circle, and a square.

8.1.2 Registration

Description :

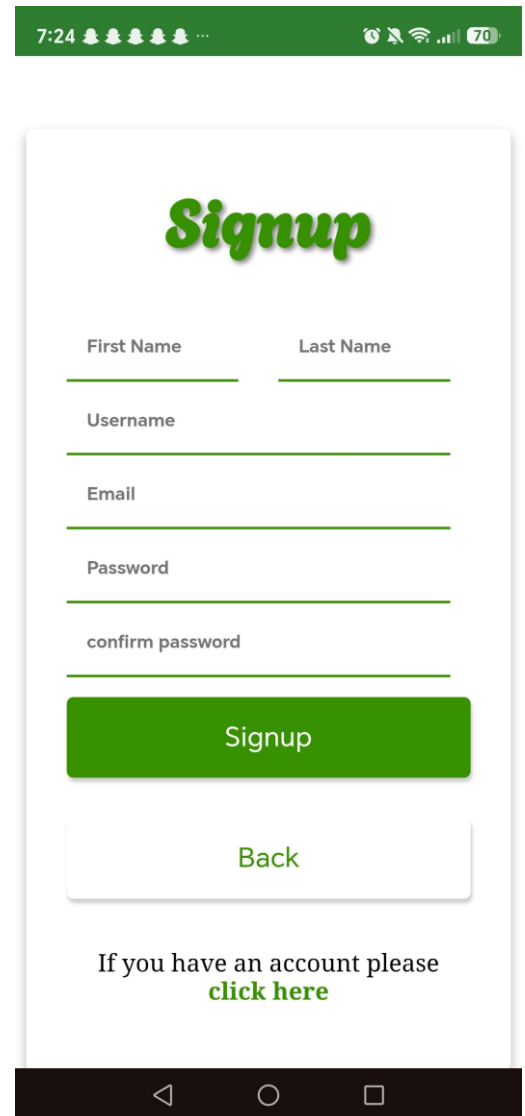
The Registration feature allows new users to create an account and access the AgroSnap application. It collects the required user information, validates the input, and creates a new user account.

Purpose :

To provide users with a secure and simple way to create an account and gain access to AgroSnap's personalized features, such as saving and managing their plant disease diagnosis reports.

Main function:

1. Collect user registration information.
2. Validate required fields and user input.
3. Check whether the provided email/username is already registered.
4. Send the registration data to the backend.
5. Create a new user account in the database.
6. Handle registration errors and display appropriate feedback to the user.
7. Redirect the user to login page after successful registration.



The screenshot shows a mobile application interface for the 'Signup' screen. At the top, there is a green header bar with the time '7:24', several notification icons, and system status icons (alarm, Wi-Fi, signal, battery at 70%). The main content area is white and features the title 'Signup' in a large, bold, green font. Below the title are five input fields, each with a green underline: 'First Name', 'Last Name', 'Username', 'Email', and 'Password'. A 'confirm password' label is positioned below the 'Password' field. A prominent green button labeled 'Signup' is located below the input fields. Underneath the button is a white button with a green border labeled 'Back'. At the bottom of the screen, there is a line of text: 'If you have an account please [click here](#)'. The very bottom of the image shows a black navigation bar with three white icons: a back arrow, a circle, and a square.

8.1.3 Log-out

Description :

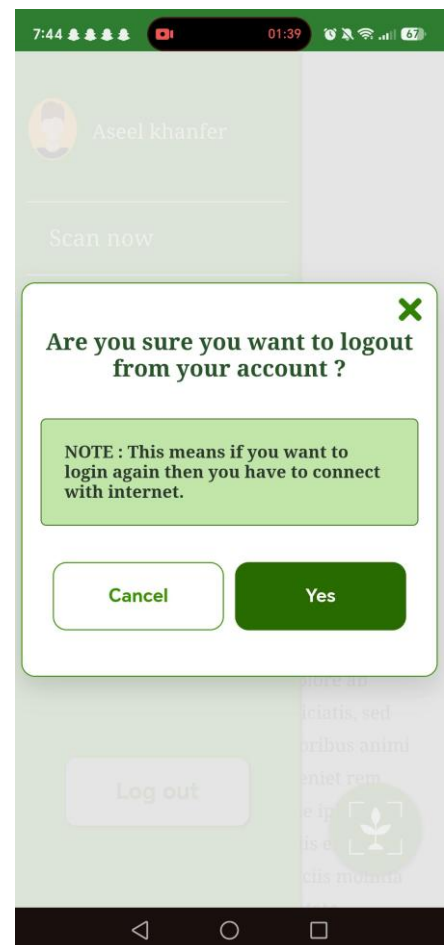
The Logout feature allows authenticated users to securely end their current session and exit their AgroSnap account.

Purpose :

To prevent unauthorized access to the user's account and ensure that the authentication session is properly terminated when the user chooses to log out.

Main function:

1. Allow the authenticated user to log out of their account.
2. Remove the stored authentication token from the browser.
3. Clear relevant session data.
4. Reset the user's authenticated state.
5. Redirect the user to the login page after logout.
6. Prevent access to authenticated pages after logging out.



8.1.4 User Session

Description :

The User Session feature manages the authenticated user's session while using the AgroSnap application.

Purpose :

To maintain the user's authenticated state and ensure that authenticated users can access protected application features during their session.

Main function:

1. Maintain the user's authenticated state after successful login.
2. Store the authentication token locally..
3. Check whether a valid authentication token is available.
4. Allow the application to identify the currently authenticated user.
5. Preserve the user's login state when navigating between application pages.

8.1.5 Token Expiration Handling

Description :

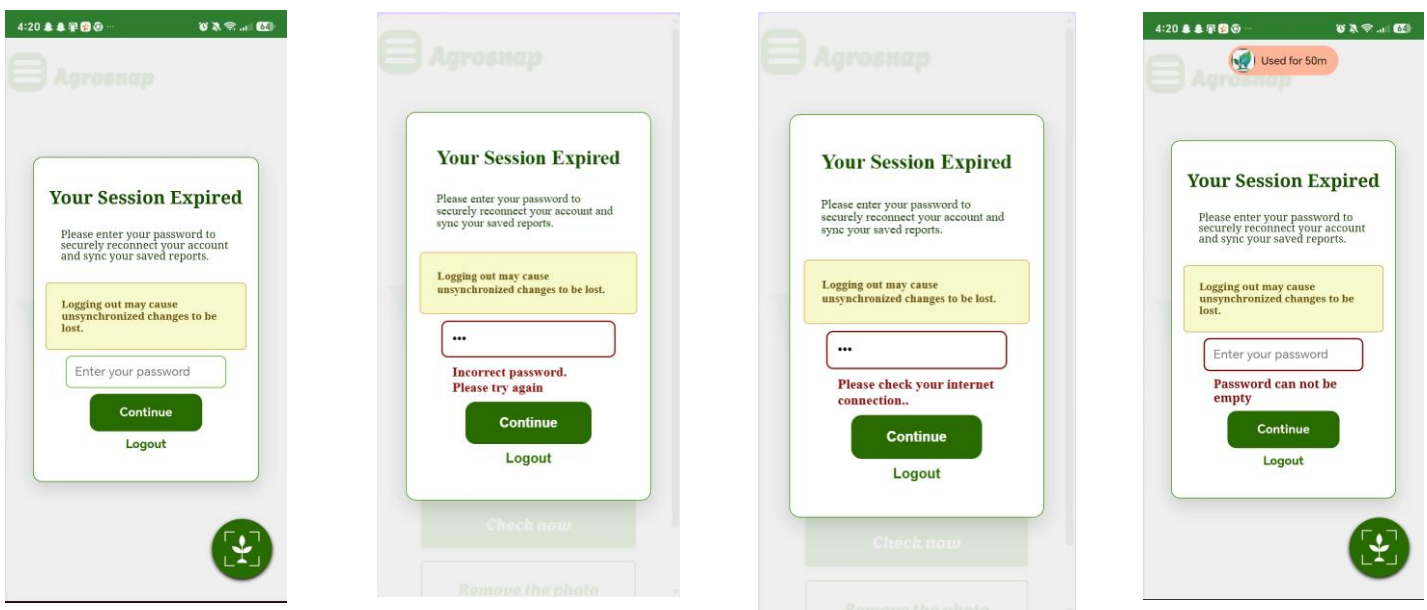
The Token Expiration Handling feature manages expired authentication tokens and ensures that users cannot continue accessing protected features after their authentication period has ended.

Purpose :

To maintain application security by detecting expired tokens and requiring the user to authenticate again when necessary.

Main function:

1. Validate the authentication token and detect when it has expired.
2. Handle re-authentication by securely requesting the user's password and providing
3. appropriate feedback for invalid, missing, or unavailable credentials.
4. Allow the user to continue their current session after successful re-authentication without losing their data or interrupting their work.
5. Provide a manual logout option for users who want to end their session.



8.2 User Management Features

8.2.1 View Profile

Description :

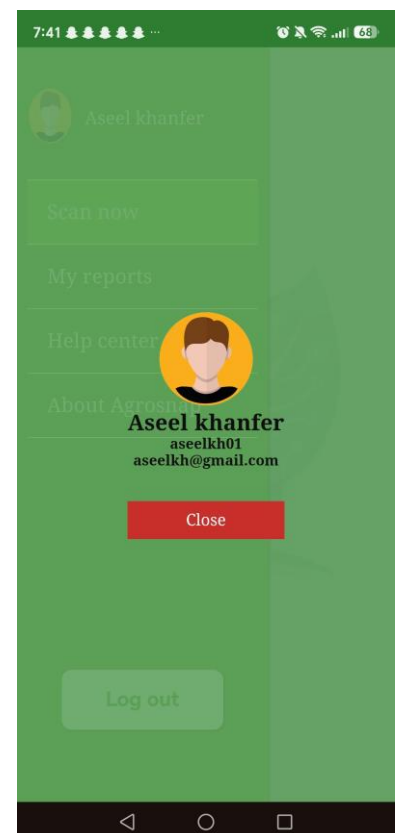
The View Profile feature allows authenticated users to view their personal account information within the AgroSnap application.

Purpose :

To provide users with easy access to their account information and allow them to verify the details associated with their account.

Main function:

1. Display the authenticated user's profile information.
2. Retrieve the user's information from the local database.
3. Display the user's name and other available account details.
4. Ensure that profile information is associated with the currently authenticated user.
5. Provide access to the user's profile through the Profile page.



8.2.2 User Session Persistence

Description :

Allows users to remain logged in when they close and reopen the application.

Purpose :

To provide a seamless experience by preserving the user's authenticated state between application sessions.

Main function:

1. Preserve the user's login state when the application is closed and reopened.
2. Automatically restore the authenticated state when a valid authentication token is available.
3. Allow the user to continue using the application without logging in again.

8.3 Plant Diagnosis Features

8.3.1 Image Upload

Description :

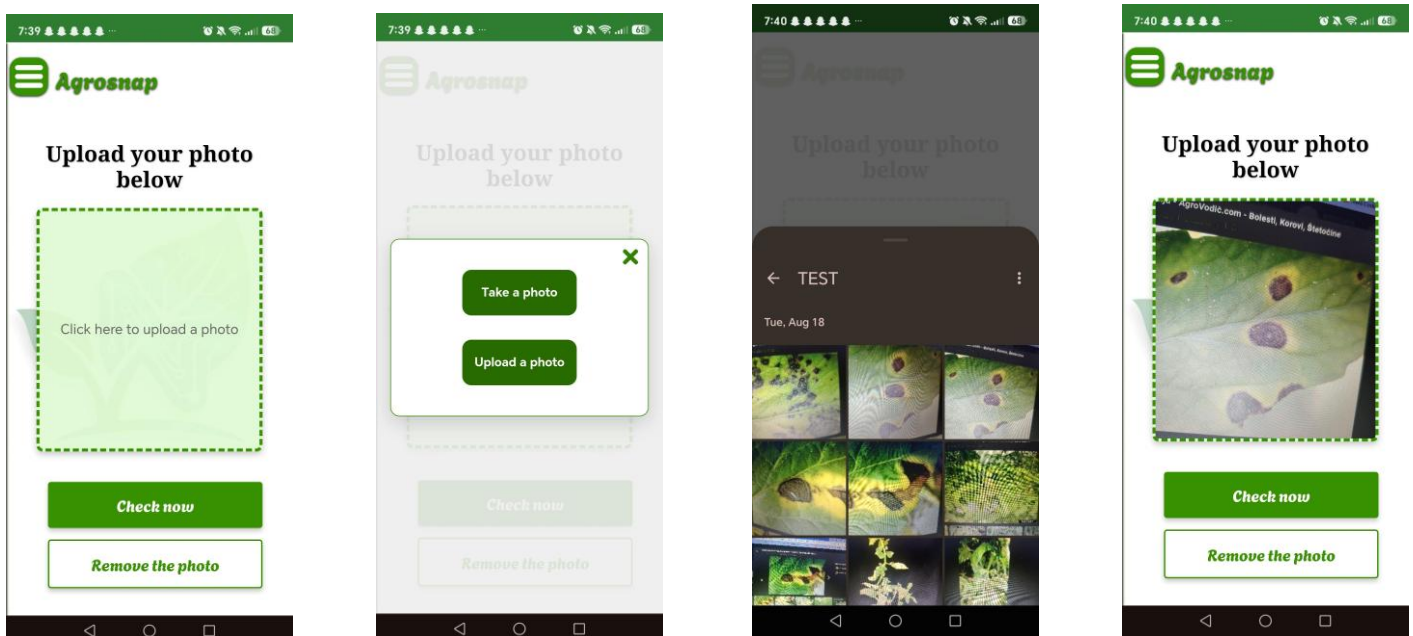
Allows users to provide a plant image for diagnosis by uploading an image from their device or capturing one using the camera.

Purpose :

To provide the image input required by the AI diagnosis system.

Main function:

1. Upload an image from the device.
2. Capture an image using the camera.
3. Preview or remove the selected image.
4. Check if there an image uploaded before the AI model start running.



8.3.2 Disease Diagnosis

Description :

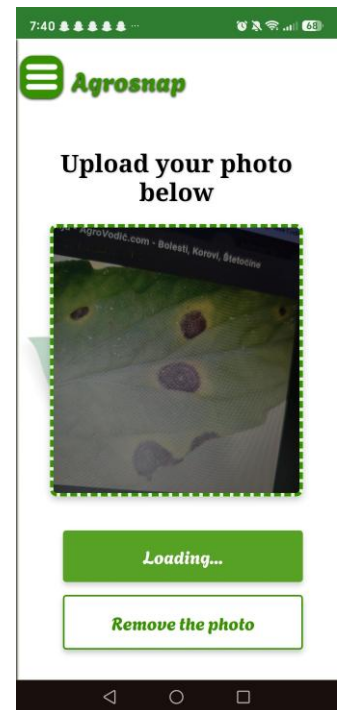
Uses an AI-based image classification system to identify the disease affecting the plant shown in the uploaded image.

Purpose :

To provide users with an automated plant disease diagnosis based on the submitted image.

Main function:

1. Process the plant image using the AI model.
2. Predict the most likely disease.
3. Calculate and display the prediction confidence.
4. Present the diagnosis result to the user.



8.3.3 Responsive AI Processing

Description :

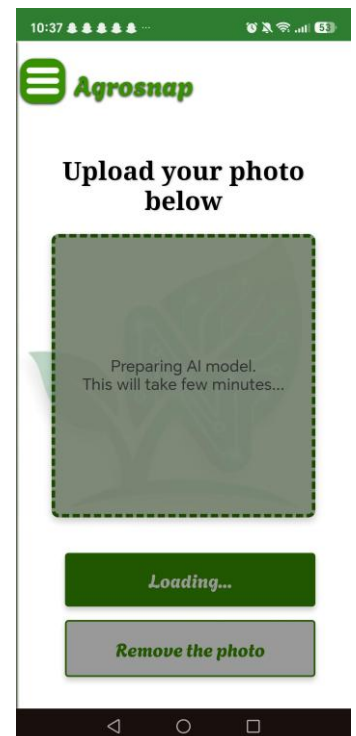
Maintains application responsiveness while the AI model is being prepared or processing the submitted image.

Purpose :

To provide a smooth user experience during AI processing without blocking the application's interface.

Main function:

1. Perform AI processing without blocking the user interface.
2. Keep the application responsive during diagnosis.
3. Provide processing feedback while the diagnosis is being performed.



8.4.1 Generate Report

Description :

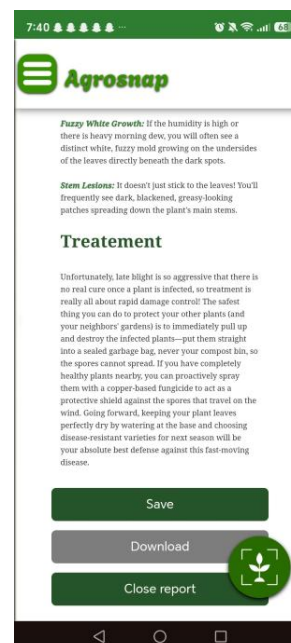
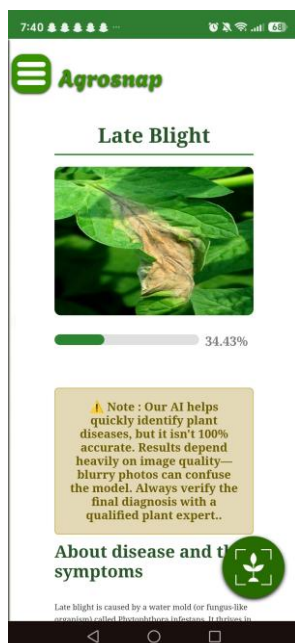
Generates a detailed disease diagnosis report based on the analyzed plant image.

Purpose :

To present the diagnosis results and relevant disease information in a clear and structured format.

Main function:

1. Display a representative image of the identified disease.
2. Display the predicted disease name.
3. Display the disease prediction confidence percentage.
4. Provide information about the disease, including symptoms and treatment recommendations.



8.4.2 Save Report

Description :

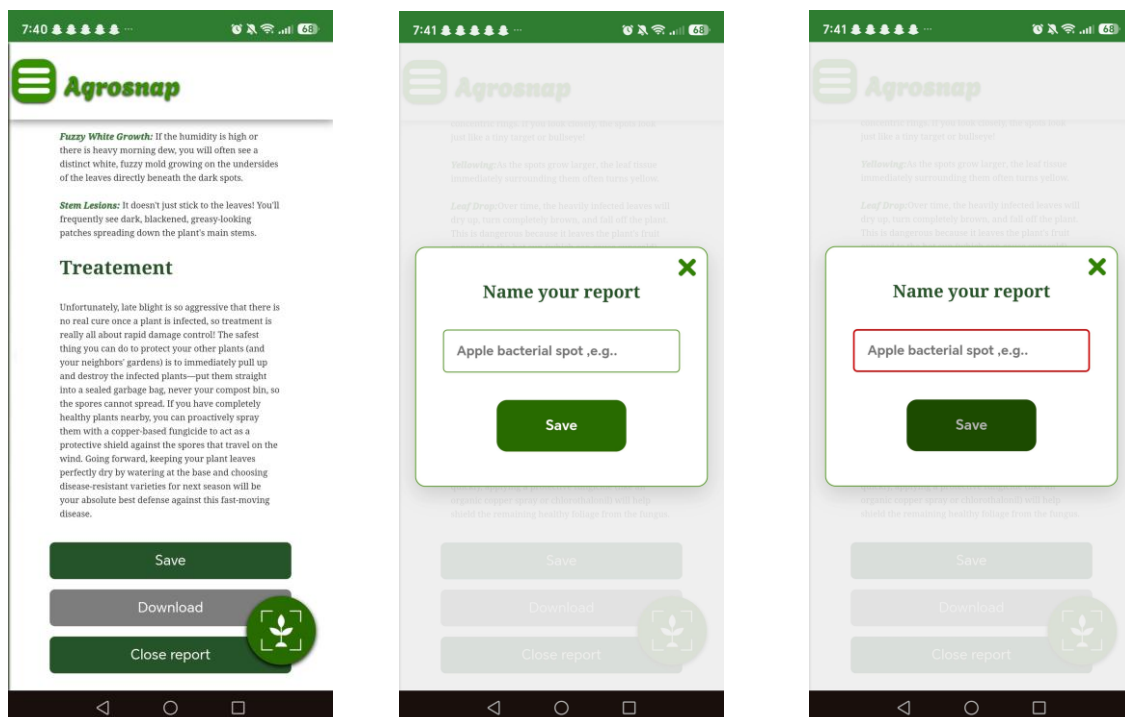
Allows users to save a generated diagnosis report for future access.

Purpose :

To allow users to keep and review their diagnosis results later.

Main function:

1. Allow the user to choose a name for the report.
2. Save the report with its associated diagnosis data.
3. Store the report for later access.



8.4.3 View Saved

Description :

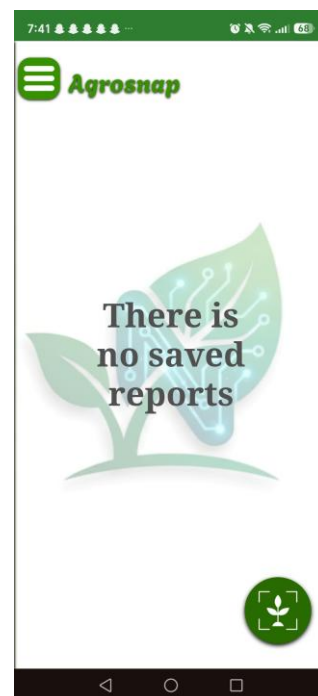
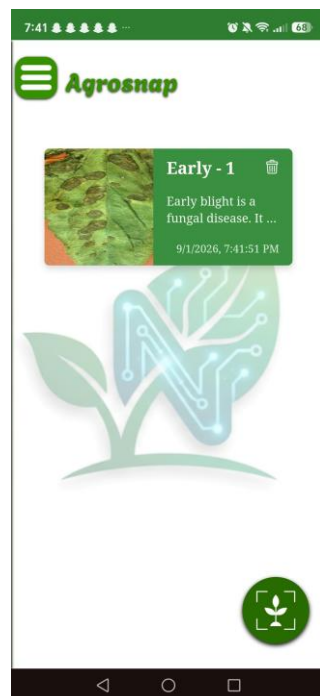
Allows users to access their previously saved diagnosis reports.

Purpose :

To provide users with convenient access to their previous diagnosis results.

Main function:

1. Display the names of saved reports.
2. Display the date and time each report was saved.
3. Open a saved report to view its complete details.



8.4.4 Download

Description :

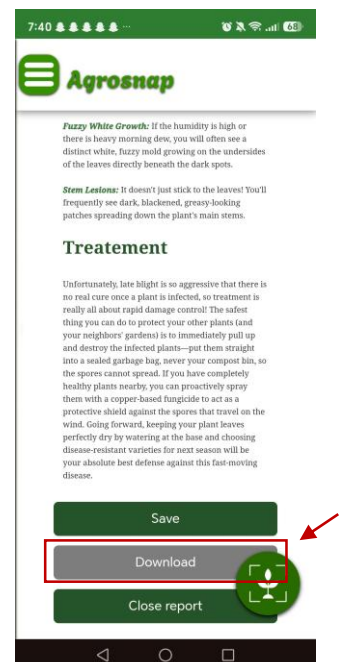
Allows users to download a generated diagnosis report for offline access or external use.

Purpose :

To provide users with a portable copy of their diagnosis report.

Main function:

1. Generate the report in a downloadable format.
2. Allow the user to save the report to their device.
3. Use the disease name as the file name if the report has not been saved.
4. Use the saved report name as the file name if the report has been saved.



8.4.5 Delete Report

Description :

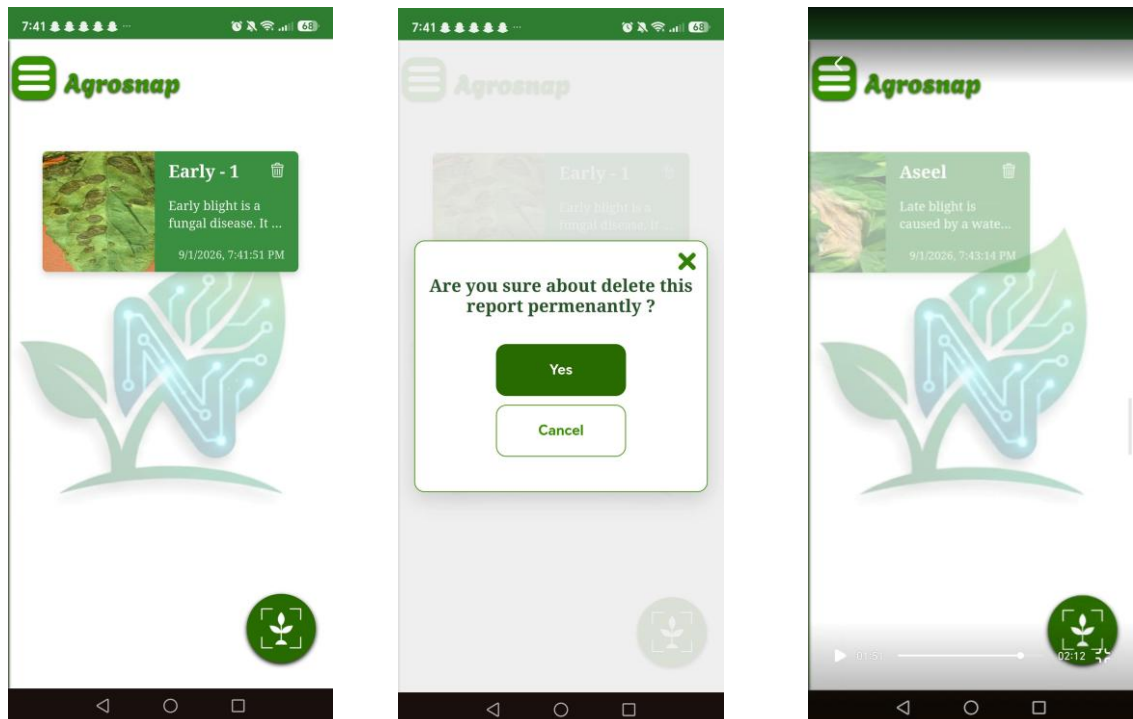
Allows users to remove reports they no longer need.

Purpose :

To give users control over their saved reports and allow them to manage their stored data.

Main function:

1. Select a saved report for deletion.
2. Confirm deletion.
3. Remove the selected report from the user's saved reports.



8.5 Navigation Features

Description :

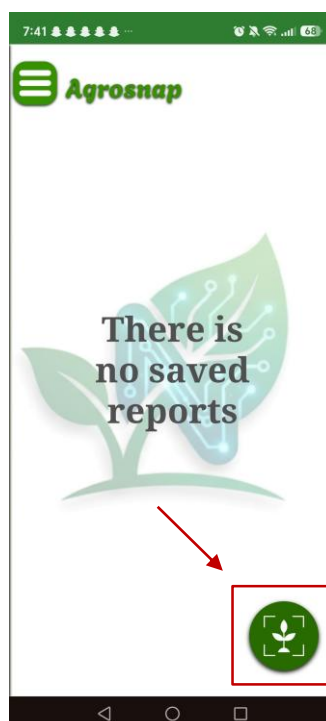
Provides a consistent way for users to navigate between the different pages and features of the AgroSnap application.

Purpose :

To make the application easy to navigate and provide users with quick access to its main features.

Main function:

1. Navigate between the main application pages using the navigation sidebar.
2. Provide access to the Scan page from different sections of the application.
3. Highlight or indicate the currently active page.
4. Support navigation between the available application sections.
5. Provide appropriate navigation options based on the user's authentication state.



8.6.1 Offline Usage

Description :

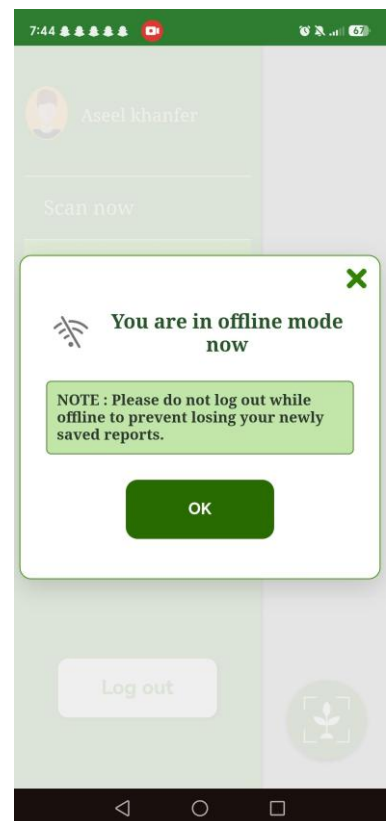
Allows users to access and use the main AgroSnap features without an active internet connection.

Purpose :

To provide users with continuous access to essential plant diagnosis and report management features in environments with limited or unavailable internet connectivity.

Main function:

1. Use the plant diagnosis feature while offline.
2. Access previously saved reports without an internet connection.
3. Store relevant data locally for offline use.
4. Synchronize locally stored data with the server when an internet connection is restored



8.6.2 Install as a Web Application – PWA

Description :

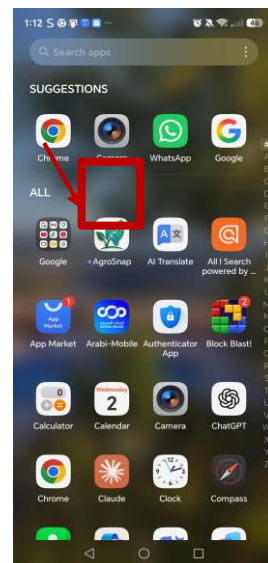
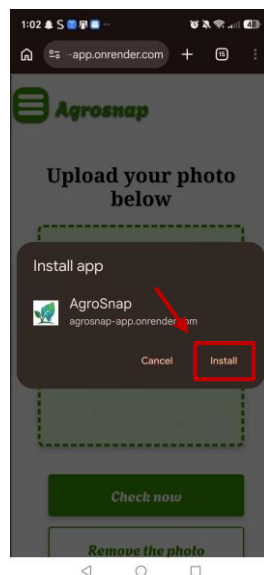
Allows users to install AgroSnap on their device and access it as a web application.

Purpose :

To provide a convenient, app-like experience and allow users to access AgroSnap directly from their device.

Main function:

1. Allow users to install AgroSnap on supported devices.
2. Launch AgroSnap as an installed application.
3. Provide an app-like experience without requiring the user to open the website through the browser.
4. Allow access to the application from the device's home screen or application menu.



9. Implementation

9.1 Front-end Implementation

9.1.1 Single Page Application – SPA

The client-side of AgroSnap was implemented using HTML, CSS, and JavaScript. The application follows a modular structure in which different modules are responsible for page management, user interaction, AI inference, local data storage, synchronization, and offline functionality. The following sections describe the main client-side implementation components and their responsibilities.

9.1.1 Front – End Structure

The frontend resources are organized under the **static** and **templates** directories. The **static** directory contains client-side stylesheets, **JavaScript modules**, **images**, **machine learning** model files, **PDF resources**, and **PWA-related** files. The **templates** directory contains the **HTML templates** used by the application pages.

```
app/  
├─ static/  
│   ├─ css/  
│   ├─ images/  
│   ├─ js/  
│   ├─ models/  
│   │   └─ web_model/  
│   ├─ ReportsPdf/  
│   ├─ manifest.json  
│   └─ service-worker.js  
└─ templates/
```

Directory /Resource	Responsibility
css/	Page-specific and shared styling.
js/	Client-side application logic.
images/	Static images and UI resources.
models/	Tensorflow.js model files.
ReportsPDF/	Disease reports PDF files.
manifest.json/	PWA configuration.
service-worker.js	Caching and offline functionality.
templates/	HTML page templates

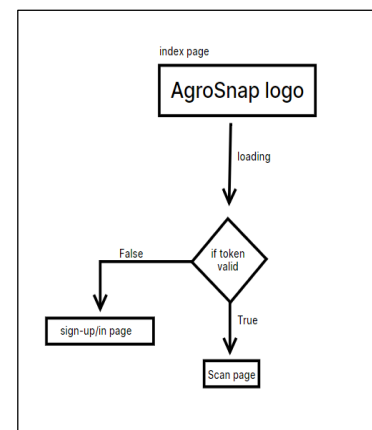
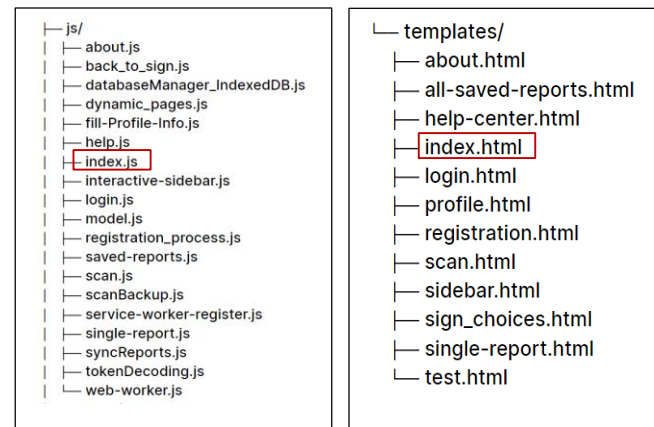
9.1.2 Application Initialization

The application initialization process is responsible for preparing the required client-side components when AgroSnap is launched. The **initial HTML page** includes the **PWA manifest** and registers the **Service Worker**. It also initializes the client-side **IndexedDB database** and creates the required object stores if they do not already exist. These stores are initially created without user-specific data and are populated when users register and interact with the application.

At the same time, the client-side initialization logic checks whether any token exists in the browser's local storage. Based on the token state, the application determines whether a user session is available and redirects the user to the appropriate page.

Controls the initial client-side flow by displaying the application logo, **checking** for an existing **authentication token**.

If a **token is present**, the application proceeds to the main application flow. If **no token** is found, the user is redirected to the sign choices page, where they can either log in or register.



Index.js

```
23
24 async function check_if_user_logged_in(){
25     let user_token = localStorage.getItem("user_token");
26     if(user_token){
27         console.log("check any update .....");
28         window.location.href="/sidebar";
29     }
30     else{
31         move_to_sign_page();
32     }
33
34 }
35
```

Index.html

```
<link rel="stylesheet" href="/static/css/index.css">
<link rel="stylesheet" href="/static/css/general_style.css">
<link rel="manifest" href="/static/manifest.json">
<script src="/static/js/index.js" defer></script>
<script src="/static/js/service-worker-register.js" defer></script>
<script type="module" src="/static/js/databaseManager_IndexedDB.js" defer></script>
```

Provides the initial application page and references the **PWA manifest** and **Service Worker**. It also initializes the **IndexedDB database** and creates the required object stores.

9.1.3 Authentication and User Management

The authentication component is responsible for managing user **access** to the AgroSnap application. It provides separate interfaces for user registration and login, allowing users to create an account and subsequently access the application using their credentials. The client side handles user input, communicates with the **backend authentication endpoints**, and manages the authentication token returned after successful login.

The authentication flow was implemented using **HTML, CSS**, and **JavaScript**. JavaScript modules are responsible for handling form interactions, sending authentication requests to the backend, processing the returned responses, and storing the authentication token in the **browser's local storage**. The token is subsequently used to identify the authenticated user when accessing protected application functionality.

➤ Registration Process

```
3 document.getElementById("submit").addEventListener('click',complete_registration_process);
4
5 > async function complete_registration_process(event) { ...
6 }
7
8
9
10 > function get_user_info(){ ...
11 }
12
13
```

The **registration process** is implemented using two main functions responsible for collecting user input and handling the registration request. First, the `get_user_info()` function retrieves the data entered through the registration form. The collected data is then passed to `complete_registration_process()`, which sends it to the server side through an HTTP request and waits for the response returned by the FastAPI backend.

The returned response is then processed to determine whether the registration was successful or whether the submitted data violates one or more registration requirements. The registration data is considered valid when the required fields are not empty, the username is not already registered in the database, the email address satisfies the required format, the password meets the required strength criteria, and the password and confirmation password match.

```
| | js/
| | | about.js
| | | back_to_sign.js
| | | databaseManager_Indexedl
| | | dynamic_pages.js
| | | fill-Profile-Info.js
| | | help.js
| | | index.js
| | | interactive-sidebar.js
| | | login.js
| | | model.js
| | | registration_process.js
| | | saved-reports.js
| | | scan.js
| | | scanBackup.js
| | | service-worker-register.js
| | | single-report.js
| | | syncReports.js
| | | tokenDecoding.js
| | | web-worker.js
```

```
| | templates/
| | | about.html
| | | all-saved-reports.html
| | | help-center.html
| | | index.html
| | | login.html
| | | profile.html
| | | registration.html
| | | scan.html
| | | sidebar.html
| | | sign_choices.html
| | | single-report.html
| | | test.html
```

Get data from UI

The `get_user_info()` function retrieves the values entered by the user from the corresponding form elements, check any empty input and then prepares the data for the registration request.

```
function get_user_info(){
  const user_data = {
    "username":document.getElementById("username").value,
    "first_name": document.getElementById("first_name").value,
    "last_name" : document.getElementById("last_name").value,
    "email" : document.getElementById("email").value,
    "PASSWORD_HASH": document.getElementById("password").value,
    "confirm_password":document.getElementById("confirm-password").value
  };
  // create an array of all values to check whether any of them is empty later
  const all_values = Object.values(user_data);

  //remove all spaces at the end and start
  const anyValueIsEmpty = all_values.some(value => value.trim() === "");
```

Send data to server as a request

The `complete_registration_process()` function handles the client-side registration request and communication with the FastAPI backend. First, the function prevents the default form submission behavior to avoid reloading the page and retrieves the entered user information using `get_user_info()`. If the collected data is available, it is converted to JSON and sent to the `/signup` endpoint using an asynchronous POST request.

After sending the request, the function waits for the server response and processes the returned JSON data. If the response indicates an error, the returned error message is extracted and displayed to the user. The function also handles validation errors returned by the backend, including structured validation errors generated by Pydantic.

When the registration is completed successfully, the interface is updated to indicate that the process is loading, a success message is displayed, and the user is automatically redirected to the login page. If the request fails because the server cannot be reached, the function catches the resulting error and informs the user that an Internet connection is required.

```
try{
  let user_data = get_user_info();
  if (user_data != null){
    const response = await fetch(based_url,
    {
      method:"POST",
      headers:{"Content-type": "application/json","ngrok-skip-browser-warning": "true"},
      body:JSON.stringify(user_data)
    });

    const process_results = await response.json();

    if (!response.ok) {
      // const the_msg = process_results.detail;
      let errorMessage;
      if(Array.isArray(process_results.detail)){ //pydantic errors
        errorMessage=process_results.detail[0].msg;
      }
      else{
        errorMessage=process_results.detail;
      }
      document.getElementById("msg").style.visibility="visible";
      document.getElementById("msg").innerHTML = errorMessage;
      return;
    }
  }
}
```

```
if(process_results.status=="success"){
  document.getElementById("submit").innerHTML="Loading...";

  appear_Successfully();
  setTimeout(function(){
    window.location.href="/loginForm";
  }, 500);
}
```


➤ Log – in Process

The login process is implemented to authenticate the user and initialize the client-side application data after a successful login. The process **collects the credentials** entered by the user, sends them to the FastAPI backend through a **POST** request, and processes the **returned** authentication response. When authentication is successful, the returned **JWT token** and user information are stored locally, and the token is decoded to obtain the required user identification data. The application then **synchronizes** the user's saved reports and disease information from the server with the **local IndexedDB** database before redirecting the user to the main application interface.

```
import handleData from "../tokenDecoding.js";
import DB from "../databaseManager_IndexedDB.js";
import { getSavedReportsFromMainDB , syncDiseases } from "../syncReports.js";

document.addEventListener('click',check_user_data);

> async function check_user_data(element) { ...
}

> function get_values_from_fields(){...
}
```

Get data from UI

```
function get_values_from_fields(){
  const collected_data = {
    "identifier": document.getElementById("username").value,
    "password" : document.getElementById("password").value
  };
  return collected_data;
}
```

The `get_values_from_fields()` function collects the username and password entered by the user and returns them as an object. The collected credentials are then sent to the `/login` endpoint using an **asynchronous POST** request with a **JSON request body**.

Send data to server as a request

The login process first collects the credentials entered by the user through `get_values_from_fields()` and sends them to the `/login` endpoint using an asynchronous **POST request**. After receiving the response from the FastAPI server, the returned **JSON data** is checked to determine whether the authentication request was **successful**. If the response is **unsuccessful**, the returned error information is extracted and displayed to the user, allowing authentication or validation errors to be communicated through the login interface.

```
const user_data = get_values_from_fields();
try{
  if (user_data){
    console.log("user data is not empty \n " ,user_data );

    const response = await fetch("/login",{
      method:"POST",
      headers:{"Content-Type": "application/json"},
      body:JSON.stringify(user_data)
    }); //end of response

    const result = await response.json(); //get result in json
  }
}
```

```
//check the response
if (!response.ok) {
  console.log("the response is not ok ");

  let errorMessage;
  if(Array.isArray(result.detail)){ //pyndatic errors
    errorMessage=result.detail[0].msg;
    console.log("this is error");
  }
  else{
    errorMessage=result.detail;
  }
  document.getElementById("msg").style.visibility="visible";
  document.getElementById("msg").innerHTML = errorMessage;
  return;
}
```



```

console.log("THIS IS THE RESULT OF TOKENS AFTER LOGIN: ", result);
if(result.create_token){
    localStorage.setItem("user_token",result.create_token);
    localStorage.setItem("other_info",JSON.stringify(result.user_info));

    const decodedData = handleData.decodeToken(result.create_token);
    console.log("this is the decoded data from create_token : ",decodedData);
}

```

```

//sync data from sqLite3 to indexedDB
const allSavedReports = await getSavedReportsFromMainDB(localStorage.getItem("user_token"));

if(allSavedReports.success){
    console.log("all saved reports that fetched from sqLite3 ", allSavedReports.data );
    let resultOfSync = await syncDiseases();
    console.log(resultOfSync);
    await DB.prepareDataAndStoreIt(allUserData,resultOfSync); //store all user data in indexedDB
}

//store all user data in indexedDB
window.location.href="/sidebar";

```

When the authentication request is successful, the client checks whether a token has been returned by the server. The **received JWT token** and additional user information are then stored in **localStorage** for subsequent authenticated operations. The token is **decoded** to extract the required user information, which is prepared for local storage. Before entering the main application interface, the client retrieves the user's saved reports and **synchronizes** the required disease information with **IndexedDB**. Once the initial data preparation is completed, the user is redirected to the main application page.

➤ Authentication Token Management

The `tokenDecoding.js` module is responsible for **handling the JWT authentication token** on the client side. It provides functions for decoding the token and retrieving user-related information stored within **its payload**, such as the **user ID and username**. The decoded token data is also **stored locally** to make the required user information available to other parts of the application.



```

import { jwtDecode } from "jwt-decode/build/esm/index.js";
function decodeToken(encodedToken) {
    // decode the token
    try{
        if(!encodedToken)return;
        const decodedData = jwtDecode(encodedToken);
        console.log(decodedData);
        localStorage.setItem("user_data",JSON.stringify(decodedData));
        return decodedData;
    }
    catch(error){
        console.log(error);
        return error;
    }
}

function getUserIdFromToken(token) {
    try{
        const userData = decodeToken(token);
        return userData.user_id;
    }
    catch(e){
        return error;
    }
}

function getUsernameFromToken(token){
    try{
        const userData = decodeToken(token);
        return userData.username;
    }
    catch(e){
        return error;
    }
}

```

The token decoding process uses the `jwt-decode` library to decode the JWT received from the server and access the information contained in its payload. The library is used on the client side to extract the data encoded within the token without manually parsing its structure. The `decodeToken()` function serves as the main decoding function; it receives the **encoded JWT**, **decodes** it using the `jwtDecode()` function provided by the library, **stores** the decoded user data in **localStorage**, and returns the decoded information for further use. Two additional functions, `getUserIdFromToken()` and `getUserUsernameFromToken()`, use the decoded token data to retrieve specific user information. The first extracts the user's ID, while the second extracts the username. This approach allows other client-side components to retrieve the required user information through dedicated functions rather than directly processing the JWT structure.

➤ Logout Process

The **logout process** is implemented to **terminate the user's session** and **remove locally stored user-related data**. When the logout function is triggered, the application first **clears** the relevant **IndexedDB tables**, followed by clearing the authentication token and other **locally stored data** from **localStorage**. The active session data stored in **sessionStorage** is also removed. Finally, the user is redirected to the sign page, preventing further access to the authenticated application flow until the user logs in again.

```
3 async function logout_process(){
4   await DB.clear_DB_tables(); // clear indexedDB
5   localStorage.clear(); // clear local storage
6   sessionStorage.clear(); // clear session
7   window.location.href="/sign"; //back to sign page
8 }
9
```

➤ Token Expiration Handling

```
export async function mainSync(){
  const syncStatus = await sync.syncReports();
  if(syncStatus.responseStatus == 401){
    showSessionExpiredWarning();
  }
  return syncStatus;
}
```

```
function checkUserPassword(event){
  event.preventDefault();
  hideFailedTokenCreationMsg();
  hideSuccessTokenCreationMsg();
  void sessionExpiredAlert.offsetwidth;
  let enteredPassword=userPassword.value.trim();
  if(!enteredPassword){showFailedTokenCreationMsg("Password can not be empty"); return;}
  RequestNewToken(enteredPassword);
}
```

The session expiration handling implemented in **dynamic_pages.js** begins in the **mainSync()** function, which checks the synchronization status returned by the synchronization process. If the returned status is **401 Unauthorized**, indicating that the authentication **session is no longer accepted** by the server, the function displays the session expiration warning. **The user can then either exit the account or continue the session by providing their password**. When the user chooses to continue, **checkUserPassword()** validates that a password has been entered and passes it to the token renewal process. This allows the user to re-authenticate without having to restart the application or manually return to the login page.

The **RequestNewToken()** function handles the process of obtaining a new authentication token after the user's session has expired. It retrieves the user's identifier from the existing token and sends it together with the entered password to the **/login endpoint** through a **POST** request. If the authentication fails, an error message is displayed to the user. When the server successfully returns a new token, the old token is **replaced in localStorage**, and the updated user information is stored locally. The new token is then decoded to obtain the required user data, which is updated in **IndexedDB**. Finally, the synchronization process is triggered again to restore the user's data synchronization before the session expiration warning is closed.

```
async function RequestNewToken(enteredPassword){
  try{
    const userID = handleToken.getUserUsernameFromToken(localStorage.getItem("user_token"));
    const userData = {
      identifier: userID,
      password: enteredPassword
    };
    const response = await fetch("/login",{
      method:"POST",
      headers:{"Content-Type": "application/json"},
      body:JSON.stringify(userData)
    });
    const result = await response.json();
    if(!response.ok){
      showFailedTokenCreationMsg("Incorrect password. Please try again");
    }
    if(result.create_token){
      console.log("The old token is: ", localStorage.getItem("user_token"));
      localStorage.setItem("user_token",result.create_token);
      localStorage.setItem("other_info",JSON.stringify(result.user_info));
      const decodedData = handleToken.decodeToken(result.create_token);
      console.log("The new token is: ", localStorage.getItem("user_token"));
    }
  }
}
```

```
//
await DB.prepareDataAndStoreIt(allUserData);
await mainSync();
showSuccessTokenCreationMsg("New token has been created ✅")
hideSessionExpiredWarning();
}
catch(e){
  showFailedTokenCreationMsg("Please check your internet connection..")
  hideOfflineModeDialog();
}
}
```

9.1.4 Page Management and SPA Navigation

The **AgroSnap client side** is implemented using a **Single Page Application (SPA)** approach, where the application **dynamically loads and displays different page templates without requiring a complete browser page reload** for each navigation. Page management is handled through JavaScript modules that control the loading of page content, initialization of page-specific functionality, and navigation between the different application views. The implementation also uses asynchronous operations and **ES module features** to organize and manage the client-side code. In addition, **sessionStorage** is used to preserve the **current page state** so that the selected page can be restored after a **browser refresh**.

```
js/
├── about.js
├── back_to_sign.js
├── databaseManager_IndexedDB.js
├── dynamic_pages.js
├── fill-Profile-Info.js
├── help.js
├── index.js
├── interactive-sidebar.js
└── ...
```

➤ Single Page Application – SPA

```
async function fit_the_page(curr_page,additionalData) {
  const allPages = [
    {path:"/scan",initJSfunction:initElements},
    {path:"/all_saved_reports",initJSfunction:startSavedReportsPage},
    {path:"/help",initJSfunction:slideAnimation},
    {path:"/about",initJSfunction:startAboutPage},
    {path:"/profile",initJSfunction:init},
    {path:"/single_report",initJSfunction:initReport}
  ];

  var clicked_page = allPages[curr_page]; //get part of name by the index -> curr_page

  fetch(clicked_page.path) //this method will go and search about this page
  .then(response => {
    return response.text();
  })
  .then(async HTML_data =>{
    void document.getElementById("content").offsetWidth;
    if (curr_page != 4){
      document.getElementById("content").classList.add("changePagesAnimation");
      document.getElementById("content").addEventListener("animationend",()=>{
        document.getElementById("content").classList.remove("changePagesAnimation");
      });
    }
    document.getElementById("main").scrollTop=0; //this to let the page go to top again , if user scrolled down
    document.getElementById("content").innerHTML=HTML_data; // put the new page here
    sessionStorage.setItem("current_page",curr_page);

    if(clicked_page.initJSfunction){ //if we have an exported functions that should work once the page called
      await clicked_page.initJSfunction(additionalData);//call the function that belong to specific page
    }
  })
}
```

The **SPA** structure allows the application to **load the required HTML template dynamically** into the main application interface instead of navigating to a completely new HTML document for every page. When the user selects a different page, the page management logic determines the requested view, loads its corresponding template, and **initializes the JavaScript functionality associated with that page**. This approach provides continuous navigation within the application while reducing the need for full page reloads.

Asynchronous operations are handled using JavaScript's **async/await syntax**, particularly when loading page templates and performing operations that require waiting for a response or completion before continuing the execution flow. This allows asynchronous tasks to be handled in a sequential and readable manner without blocking the main application flow.

The client-side code is **organized using JavaScript ES Modules** through **import and export**. Functions and modules are exported from their respective files and imported only where they are required. This modular structure separates page management, authentication, database operations, synchronization, and other client-side responsibilities into independent modules, improving code organization and maintainability.

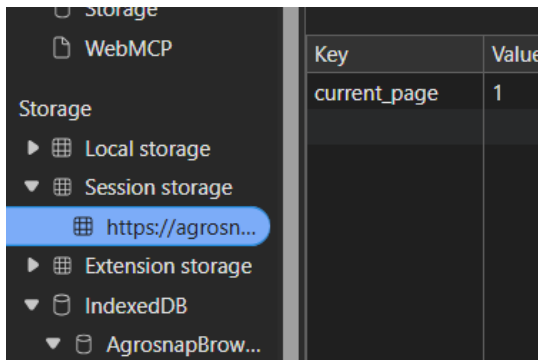
```
async function fit_the_page(curr_page,additionalData) {
```

```
  await clicked_page.initJSfunction(additionalData);
```

```
import {slideAnimation} from './help.js';
import { init } from './fill-Profile-Info.js';
import { initElements } from './scan.js';
import { startSavedReportsPage } from './saved-reports.js';
import handleToken from './tokenDecoding.js';
import DB from './databaseManager_IndexedDB.js';
import { initReport } from './single-report.js';
import { startAboutPage } from './about.js';
import sync from './syncReports.js';
```

```
export async function initElements(){
```

➤ SessionStorage



To preserve the user's current location within the application, the **current page** is stored in the browser's **sessionStorage**. Whenever the user navigates to a different view, the current page identifier is **updated** in **sessionStorage**. When the application is refreshed, the stored page information is retrieved and used to restore the previously selected view instead of always returning the user to the default page. Since sessionStorage persists while the browser tab or session remains active, the current page state is preserved across page refreshes.

9.1.5 Scan and Image Processing

The **scan.js** module is responsible for managing the client-side scanning process in **AgroSnap**. It handles the interaction between the user and the image scanning interface, including receiving an image from the available input methods, managing the selected image, and preparing it for the diagnosis process. The module also coordinates the communication with the **AI processing component** and updates the user interface according to the **returned diagnosis result**. This module therefore serves as the main client-side controller for the image scanning workflow, connecting the image input, user interface, and AI prediction process.

```
js/
├── about.js
├── back_to_sign.js
├── databaseManager_IndexedDB.js
├── dynamic_pages.js
├── fill_Profile-Info.js
├── help.js
├── index.js
├── interactive-sidebar.js
├── login.js
├── model.js
├── registration_process.js
├── saved-reports.js
├── scan.js
├── scanBackup.js
├── service-worker-register.js
├── single-report.js
├── syncReports.js
├── tokenDecoding.js
└── web-worker.js
```

➤ Image Acquisition

```
uploadBox.onclick = showPhotoUploadPopup;
cameraButton.onclick=openCamera;
fileManagerButton.onclick=openFileManager;
```

```
fileInput.addEventListener("change",showImage);
camera.addEventListener("change",showImage);
```

```
> function openFileManager(event){ ...
}
> function openCamera(event){ ...
}
> function openPopup(){ ...
}
```

The image acquisition process provides **two input methods** for the user: **capturing a new image** using the device camera or **selecting an existing image** through the file manager. When the user selects either option, the corresponding input element is triggered through `openCamera()` or `openFileManager()`. Both input methods are then **handled** through **the same image-processing** flow, allowing the application to use a unified process regardless of how the image was obtained.

➤ Image Selection and Preview

The `showImage()` function handles the selected image after it is received from either the file manager or the camera input. It first retrieves the selected file from the input event and stores it in `selected_image`. If no file is selected, the function terminates without making any changes. When a valid image is received, the upload popup is closed and a temporary object URL is created using `URL.createObjectURL()`. This URL is assigned to the preview image element to display the selected image within the upload area. The function also disables selecting another image until the current image is cleared, ensuring that only one image is processed at a time.

```
function showImage(event){
    void uploadBox.offsetWidth;
    //files always return a group of images so we always need the first photo has been selected
    selected_image = event.target.files[0]; //the selected file from fileManager / camera / gallery

    //if no photo selected , then don't do anything
    if(!selected_image){
        return;
    }

    uploadBox.classList.remove("error");//when the user upload a photo successfully
    closePopup();

    //when the photo has been uploaded and appeared inside the box , you can't upload another one until you
    canClick=false;
    convertedImg=URL.createObjectURL(selected_image); //make a temp url in browser
    previewImage.src = convertedImg; // fill the url inside 
    previewImage.style.display="block";//make the image visible
}
```

➤ Remove Image and Clear Memory

```
function clearBox(){
    if(convertedImg){
        URL.revokeObjectURL(convertedImg); //clear the memory from any temp url
        convertedImg = null;
    }
    selected_image=null;
    previewImage.src = "";
    previewImage.style.display="none";
    previewImage.classList.remove("checking-and-loading");
    checkButton.innerHTML="Check now";
    canClick=true; //user can upload images again after the box is cleared
}
```

The `clearBox()` function resets the image upload area after the user removes the selected image. It first **releases** the **temporary object URL created** for the image using `URL.revokeObjectURL()` to free the associated browser resource. The selected image and its preview are then cleared, and the preview element is hidden. The function also restores the check button to its initial state **and re-enables image selection**, allowing the user to choose a new image.

➤ Image Validation and Preparation

The `prepareSelectedImage()` function prepares the selected image for processing by converting it into an `ImageBitmap` using `createImageBitmap()`. An `ImageBitmap` provides a browser-native representation of the image that can be efficiently transferred to a Web Worker for further processing. This conversion allows the image to be passed from the main thread to the worker through `postMessage()`, so the AI processing can be performed separately without blocking the user interface. The function is asynchronous because creating the bitmap is an asynchronous operation and may require time to complete.

```
async function prepareSelectedImage(selectedImg){
    try{
        //convert file to bitmap , so the web worker can read it
        let imageBitmap = await createImageBitmap(selectedImg);
        if(!imageBitmap)return false;

        //clear the temp url from browser memory to prevent memory leak
        URL.revokeObjectURL(selectedImg);
        return imageBitmap;
    }
    catch(error){
        console.log("Something went wrong during preparing the image , ",error.message);
    }
}
```


➤ Send Image to AI model

```
async function checkThePlantLeaf(){
  try{
    closePhotoUploadPopup();
    uploadBox.classList.remove("error"); //remove red border
    //check if there any uploaded image before send it to AI
    if(!selected_image){
      /*we used it to let the browser see the changes
      so the animation can removed,added and worked again*/
      void uploadBox.offsetWidth;
      uploadBox.classList.add("error");
      return;
    }

    canClick=false;
    previewImage.classList.add("checking-and-loading");
    checkButton.innerHTML="Loading...";
    void uploadBox.offsetWidth;
    let bitmapImage = await prepareSelectedImage(previewImage);

    //send the converted image to web worker
    modelThread.postMessage({
      type:'predict',
      image:bitmapImage
    });

    //show_report(predictedDisease);
  }
  catch(error){
    alert("this is in scan.js , in checkThePlantLeaf(), " + error);
  }
}
```

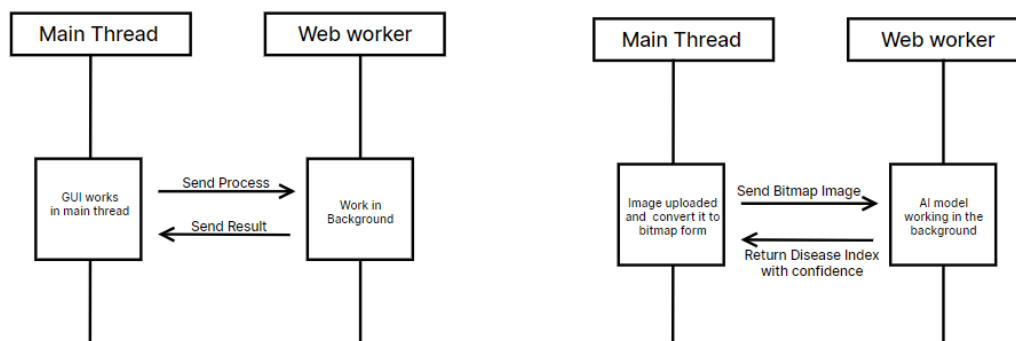
The `checkThePlantLeaf()` function starts the image diagnosis process when the user selects “**Check now**”. It first verifies that an image has been selected and stops the process if no image is available. For a valid image, it updates the interface to indicate that processing is in progress, prepares the image using `prepareSelectedImage()`, and sends the **resulting ImageBitmap to the Web Worker** through `postMessage()` for AI processing.

9.1.6 AI Model Integration

➤ Web Worker for AI Processing

A **Web Worker** was used in AgroSnap to execute **the AI processing in a separate background thread** from the **main thread**. The **main thread is responsible for handling the user interface and user interactions**, while the **Web Worker performs computationally intensive operations such as model processing and image inference**. This separation prevents the AI operations from blocking the main thread and keeps the application interface responsive during processing. Communication between the main thread and the Web Worker is implemented through message passing using `postMessage()` and `onmessage`. The main thread sends commands and image data to the worker, while the worker returns the model status and prediction results back to the main thread.

```
| |— js/
| |  |— about.js
| |  |— back_to_sign.js
| |  |— databaseManager_IndexedDB.js
| |  |— dynamic_pages.js
| |  |— fill-Profile-Info.js
| |  |— help.js
| |  |— index.js
| |  |— interactive-sidebar.js
| |  |— login.js
| |  |— registration_process.js
| |  |— saved-reports.js
| |  |— scan.js
| |  |— service-worker-register.js
| |  |— single-report.js
| |  |— syncReports.js
| |  |— tokenDecoding.js
| |  |— web-worker.js
```



➤ AI Model Loading and Warm – Up

The **model loading and warm-up** process starts when `initElements()` sends a `loadModel` message from the main thread to the **Web Worker** using `postMessage()`. The worker receives this message through its `onmessage` handler and calls `loadModel()`. The function loads the TensorFlow.js Graph Model from `model.json` using `tf.loadGraphModel()`, together with its associated weight files, and stores the loaded model in memory for subsequent predictions. After the model is successfully loaded, a warm-up operation is performed using a dummy input tensor with the same input dimensions expected by the model ($260 \times 260 \times 3$). This allows the model to perform an **initial prediction before it is used with a real image**. Once **loading and warm-up** are completed, the Web Worker sends a `modelReady` message back to the **main thread**. The `scan.js` module receives this message through `onmessage`, updates the model readiness state, and enables the scanning controls for the user.

```
//warmup
tf.tidy(() => {
  const dummy = tf.zeros([1, 260, 260, 3]);
  const output = model.predict(dummy);

  if (Array.isArray(output)) {
    output.forEach(t => t.dispose());
  } else {
    output.dispose();
  }
});

console.log("Model warm-up finished!");

return model;
```

```
//recieving area from main thread
self.onmessage =async(event)=>{
  try{
    /*=====
    Upload model and warm up
    =====*/

    if(event.data.type == "loadModel"){
      await loadModel();

      /*when model upload then send msg
      to main thread[scan] that model
      is ready*/
      self.postMessage({
        type: "modelReady"
      });
    }
  }
}
```

```
//send the converted image to web worker
modelThread.postMessage({
  type:'predict',
  image:bitmapImage
});
```

➤ Image Processing and Prediction

```
function preprocessTheImage(image){
  return tf.tidy(() => {
    const readableImage = tf.browser.fromPixels(image)
      .resizeBilinear([260,260])
    /*the image type that came from browser is int and
    we convert it to float32 because the weight, colors and
    operations in model are float*/
    .toFloat()
    /*if somehow the user selected group of images ,
    just take the first selected image , index 0*/
    .expandDims(0);
    // console.log("the image became: ", readableImage);
    return readableImage;
  });
}
```

The `preprocessTheImage()` function prepares the selected image for input into the AI model. It first converts the image into a **TensorFlow.js** tensor using `tf.browser.fromPixels()`, then **resizes it to 260 × 260 pixels** to match the model's required input dimensions. The pixel values are converted to floating-point values, and a batch dimension is added to produce the required **input shape of [1, 260, 260, 3]**. The operations are wrapped in `tf.tidy()` to automatically manage intermediate tensors and reduce unnecessary memory usage. This preprocessing step ensures that the image is converted into the correct format and dimensions before being passed to the model for prediction.

The `analyzeTheImage()` function performs the prediction using the loaded AI model. It first passes the preprocessed image tensor to `model.predict()` to obtain the prediction probabilities for all supported classes. The `argMax()` function is then used to identify the class with the highest probability, and its index is stored as the `disease_id`. The corresponding probability is retrieved and converted into a percentage with two decimal places to represent the **prediction confidence**. Finally, the function releases the tensors used during prediction and returns the detected **disease ID** and **confidence value** to the Web Worker for further processing.

```
async function analyzeTheImage(convertedImage){

    /*this will make a delay if the model is not
    uploaded before , while if it uploaded then no delay*/

    //predict by AI
    const result = model.predict(convertedImage);
    //Get the max number [higher probability]
    const maxPercentage = result.argmax(1);
    //Get the disease ID
    const disease_id = maxPercentage.dataSync()[0];

    //for confidence percentage [all confidences for all diseases]
    const probabilities = await result.data();

    //get the higher confidence
    let confidence = probabilities[disease_id];
    // convert it to number of 100 [44%]
    confidence = (confidence * 100).toFixed(2);

    console.log("the confidence is : ", confidence);

    //dispose is for clear the memory from tensor
    maxPercentage.dispose();
    result.dispose();
    return {disease_id, confidence};

}
```

```
if(event.data.type == "predict"){
    let theImage = event.data.image;
    if(!theImage) return
    console.log("The bitmap image has been recieved successfully ", theImage);

    //will return a readable image which model can handle and read it
    let preparedImage = preprocessTheImage(theImage);

    console.log("The readable image is :", preparedImage);

    const result = await analyzeTheImage(preparedImage);

    preparedImage.dispose();
    theImage.close();

    self.postMessage({
        type:"result",
        diseaseInfo:result
    });
}
```

When the **Web Worker receives** a predict message, it retrieves the **ImageBitmap** sent from the main thread and passes it to `preprocessTheImage()` to prepare it for the model. The resulting tensor is then passed to `analyzeTheImage()` to perform the prediction. After the prediction is completed, the function releases the allocated image and tensor resources using `dispose()` and `close()`. Finally, the **prediction result containing the disease_id and confidence** is sent back to the main thread using `postMessage()` for further handling and display.

➤ Prediction Result Handling

After the prediction is completed in the Web Worker, the resulting `disease_id` and `confidence` are **sent back to the main thread** using `postMessage()`. The `scan.js` module receives this result through the worker's `onmessage` handler and passes the returned diagnosis information to `sendDataToGUI()`. This function handles the prediction result at the application level by storing the required information locally and updating the interface according to the detected class. If the prediction corresponds to a healthy plant, a **healthy-result message is displayed**; otherwise, the user is directed to the **appropriate disease report**. This allows the **prediction generated by the AI model** to be converted into a result that can be presented and used by the rest of the application.

```
function sendDataToGUI(diseaseInfo){

    console.log("What will send to GUI : ", diseaseInfo);

    //if the value id null
    if(diseaseInfo.disease_id === undefined || diseaseInfo.disease_id === null){
        console.log("no result");
        return;
    }
    localStorage.setItem("predictedDiseaseIndex",diseaseInfo.disease_id);
    localStorage.setItem("confidence", diseaseInfo.confidence);
    sessionStorage.setItem("CallerPage",sessionStorage.getItem("current_page"));
    clearBox();
    if(diseaseInfo.disease_id === 2){ // if the result is healthy , then show the healthy popup
        showHealthyPopup();
        return;
    }
    show_report(diseaseInfo);

}
```

```
//Result area after the worker predicted the image
if(event.data.type=='result'){
    console.log("The result of prediction is: ", event.data.diseaseInfo);
    sendDataToGUI(event.data.diseaseInfo);
}
```


9.1.7 Report Management

The report management process handles the diagnosis results after the AI prediction is completed. It allows the user to generate and view the corresponding report, **save** it for later access, **delete saved reports** when they are no longer needed, and **download reports as PDF** files. These operations provide the main client-side functionality for managing the reports generated by the application.

```
├── js/
│   ├── about.js
│   ├── back_to_sign.js
│   ├── databaseManager_IndexedDB.js
│   ├── dynamic_pages.js
│   ├── fill_Profile_Info.js
│   ├── help.js
│   ├── index.js
│   ├── interactive-sidebar.js
│   ├── login.js
│   ├── registration_process.js
│   ├── saved-reports.js
│   ├── scan.js
│   ├── service-worker-register.js
│   ├── single-report.js
│   ├── syncReports.js
│   ├── tokenDecoding.js
│   └── web-worker.js
```

➤ Generate Report

```
//get the disease info from indexedDB
let the_disease = await db.get_diseases_info(index);
await setDiseaseImage(index);

//fill the elements with data of disease to represent it in user interface
reportTitle.innerHTML = the_disease.disease_name;
diseaseContent.innerHTML = the_disease.disease_info;
treatmentContent.innerHTML = the_disease.treatment;
progPercent.innerHTML = `${confidence}%`;
progLine.style.width = `${confidence}%`;
```

The `fillReportStructure()` function generates and **populates the disease report based on the prediction result**. It uses the predicted `disease_id` to retrieve the corresponding disease information from **IndexedDB**, including the disease name, description, and treatment. It also sets the appropriate disease image and displays the prediction confidence in the report. This function therefore connects the AI prediction with the stored disease information and presents the complete diagnosis report to the user.

➤ Save Report

The report saving process is handled by the `saveReportProcess()` function, which prepares the required report data before **storing it locally**. A **unique identifier** is generated for each report using `crypto.randomUUID()`, allowing each saved report to be uniquely identified. The function then collects the user ID, predicted disease ID, report name, prediction confidence, synchronization status, and creation date into a report object. This object is stored in the `saved_reports` object **store in IndexedDB**, providing persistent local storage for the user's reports. The report is initially marked as unsynchronized, allowing it to be synchronized with the server later through the application's synchronization process.

```
let newUUID = crypto.randomUUID();
let userID = handleData.getUserIdFromToken(localStorage.getItem("user_token"));
let diseaseOtherInfo = await db.get_diseases_info(diseaseData.disease_id);
```

```
let isSaved = await db.store_report(reportData);

if(isSaved && !reportData.isSynced){ //check if saved in indexedDB first
  const saveStatus = await sync.sendReportToMainDB(localStorage.getItem("user_token"), reportData);
  if(saveStatus.success) //we have to put it as saveStatus.success
    reportData.isSynced=true;
  goToMyReports(reportData);
}

hideRenameDialog();
```

➤ Delete Report

```
async function deleteReport(reportId){  
  
    let isDeleted = await db.add_deleted_reports(reportId);  
    if(!isDeleted) return isDeleted;  
    return isDeleted;  
}
```

The report deletion process is handled by the `deleteReport()` function. When a user chooses to delete a saved report, the report ID is passed to the function and stored in the `deleted_reports` object **store in IndexedDB** using `add_deleted_reports()`. The report is therefore **recorded as deleted** on the client side, **allowing the synchronization process to later propagate the deletion to the server**. The function also checks whether the deletion record was successfully stored before returning the resulting status.

➤ Download Report

The report download process retrieves the PDF file corresponding to the detected disease from the predefined report paths. The `downloadReport()` function first obtains the PDF URL based on the disease ID and sends an asynchronous `fetch()` request to retrieve the file. The returned response is then converted into a Blob, which represents the PDF data in the browser. A temporary object URL is created for the **Blob using `URL.createObjectURL()`**, and a temporary anchor element is used to initiate the download. The download attribute **specifies the name of the downloaded report**, while `URL.revokeObjectURL()` is finally used to release the **temporary object URL** after the download is triggered.

```
let pdfURL = reportsPDFs[diseaseID].pdf;  
const requestPDFReport = await fetch(pdfURL);  
  
if(!requestPDFReport.ok){  
    console.log("failed request ", requestPDFReport.status);  
}
```

```
//  
const blob = await requestPDFReport.blob();  
  
console.log("PDF blob:", blob);  
console.log("Blob type:", blob.type);  
console.log("Blob size:", blob.size);  
  
const url = URL.createObjectURL(blob);  
  
console.log("this is url after creating it: ",url);  
  
const reportLink = document.createElement("a");  
  
reportLink.href=url;  
reportLink.download = reportName;  
// localStorage.removeItem("openedReport");  
reportLink.click();  
URL.revokeObjectURL(url);
```

9.1.8 Browser Database – IndexedDB Implementation

```
├─ js/  
│  ├─ about.js  
│  ├─ back_to_sign.js  
│  ├─ databaseManager_IndexedDB.js  
│  ├─ dynamic_pages.js  
│  ├─ fill-Profile-Info.js  
│  ├─ help.js  
│  ├─ index.js  
│  ├─ interactive-sidebar.js  
│  └─ login.is
```

IndexedDB is NoSQL database , was used to provide **persistent client-side storage** for AgroSnap. It allows the application to **store user data**, disease information, and saved reports **locally in the browser**, **supporting offline access** and reducing the need to repeatedly retrieve data from the server.

➤ Database Structure

The IndexedDB database is initialized using the **idb library**, which provides a simplified interface for working with **IndexedDB**. The database contains separate **object stores** for user **information**, **disease information**, **saved reports**, and **deleted reports**. The database connection is created when required and reused for subsequent operations, while the upgrade callback is responsible for creating the required object stores when the database is initialized.

```
//these are the tables names we will create it
const user_table = "user_info";
const disease_table = "diseases_info";
const saved_reports_table = "saved_reports";
const deleted_report_table = "deleted_reports";
```

```
if(!dbConnection){
    dbConnection = await openDB("AgrosnapBrowserDatabase", 1,{
        upgrade(db){
            // here where we create our tables
            create_users_table(db);
            create_disease_table(db);
            create_saved_reports(db);
            create_deleted_reports(db);
        }
    });
}

} //end if
```

➤ Adding Data to Object Stores

```
async function add_new_user(newData){...
}

async function add_disease_info(all_diseases) {
    //After got the disease from fastAPI we add it to indexedDB
    try{
        if(all_diseases){
            const db = await openConnection();
            const trans = db.transaction(disease_table,"readwrite");
            const current_table = trans.objectStore(disease_table);
            await Promise.all( all_diseases.map( row => current_table.put(row) ));
            trans.done;
        }
    }
    catch(error){
        console.log("Adding disease failed , " , error.message);
    }
}

async function add_deleted_reports(reportID){...
}

async function store_report(report_data){...
}
```

The client-side database provides methods for adding, retrieving, and deleting data from the different object stores. These **methods use IndexedDB** transactions to access the required object store and perform the corresponding operation. As shown in the figure, the **add_disease_info()** method uses a **readwrite** transaction and **put()** to store user data. The other data insertion methods follow the same approach for their respective object stores.

➤ Data Retrieval from indexedDB

The client-side database also provides methods for retrieving stored data from the object stores. These methods use **readonly** transactions to access the required data **without modifying the database**. As shown in the figure, the `getUserByID()` method retrieves a specific user record **using its key**, while other retrieval methods follow the same approach to access reports and disease information from their respective object stores.

```
async function get_all_reports_from_indexedDB(){
  /*will return all saved reports from
  indexedDB and will be used in the saved reports page*/
  const db = await openConnection();
  const trans = db.transaction(saved_reports_table, "readonly");
  const current_table = trans.objectStore(saved_reports_table);
  const allReports = await current_table.getAll();
  trans.done;
  return allReports;
}

async function get_report_by_id(report_id){
  const db = await openConnection();
  const trans = db.transaction(saved_reports_table, "readonly");
  const current_table = trans.objectStore(saved_reports_table);
  const report = await current_table.get(report_id);
  trans.done;
  return report;
}

async function get_all_deleted_reports() { ...
}

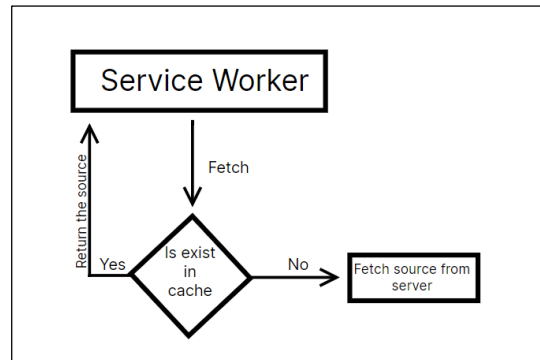
async function prepareDataAndStoreIt(user_info,all_disease){ ...
}

async function getUserByID(userID) { ...
}
```

9.1.9 PWA and Offline Support

```
| | | js/
| | | | about.js
| | | | back_to_sign.js
| | | | databaseManager_IndexedDB.js
| | | | dynamic_pages.js
| | | | fill-Profile-Info.js
| | | | help.js
| | | | index.js
| | | | interactive-sidebar.js
| | | | login.js
| | | | registration_process.js
| | | | saved-reports.js
| | | | scan.js
| | | | service-worker-register.js
```

```
| | | ReportsPdf/
| | | | manifest.json
| | | | service-worker.js
| | | templates/
```



AgroSnap combines **three client-side technologies** to provide **PWA and offline** capabilities. The **Web App Manifest** defines the application information required for installation as a **PWA**, such as its name, icons, and display settings. The **Service Worker** manages cached application resources and handles network requests, allowing required files to remain available offline. **IndexedDB** provides persistent local storage for application data such as user information, disease information, and saved reports. Together, these technologies allow AgroSnap to be installed as a web application and continue providing access to **cached resources and locally stored** data when an Internet connection is unavailable.

➤ Service Worker Registration

The **Service Worker** is registered when the application starts using the browser's `navigator.serviceWorker` API. The implementation first checks whether Service Workers are supported by the browser, then registers `service-worker.js` to enable the application's **caching and offline functionality**.

```
if("serviceWorker" in navigator){
  navigator.serviceWorker.register("/service-worker.js");//add the json file in navigator
  .then(() => {
    console.log("Service Worker registered");
  })
  .catch(error => {
    console.log(error);
  });
}
```

➤ Cache Configuration and Installation

```
const CACHE_NAME = "Agrosanp-1";
> const FILES_TO_CACHE = [ ...
];
```

```
self.addEventListener("install", event => {
  console.log("Installation started , service-worker.js");
  self.skipWaiting();
  console.log("SW installed");
  event.waitUntil(
    caches.open(CACHE_NAME).then(async cache => {
      for (const file of FILES_TO_CACHE) {
        try {
          await cache.add(file);
          console.log("Cached:", file);
        } catch (e) {
          console.log("Failed:", file, e.message);
        }
      }
    })
  );
  console.log("All files installed , service-worker.js ");
});
```

▼ Cache storage
Agrosanp-1 - https://agrosnap-app.o...

A dedicated **cache named Agrosanp-1** is used to store the resources required by the application. The `FILES_TO_CACHE` array defines the resources to be cached, including application pages, CSS and JavaScript files, images, PDF reports, PWA resources, IndexedDB dependencies, and TensorFlow.js model files. This allows the main application resources and AI model files to be **available locally for offline use**.

During the `install` event, the **Service Worker opens the specified cache and iterates through the resources defined in `FILES_TO_CACHE`**, adding each resource to the cache. The installation process is managed using `event.waitUntil()` to ensure that the caching operation is completed as part of the Service Worker installation. The implementation also uses `skipWaiting()` to allow the newly installed Service Worker to become active without waiting for existing Service Worker instances to finish.

➤ Fetch Handling and Offline Access

During a **fetch event**, the Service Worker first **checks whether the requested resource is available in the Cache Storage**. If a cached response is found, it is returned directly without accessing the network. If the resource is not cached, the Service Worker attempts to retrieve it from the network. When both the cache and network are unavailable, a **503 Service Unavailable response is returned**, indicating that the requested resource is **not available offline**.

```
event.respondWith(
  caches.match(event.request, { ignoreVary: true })
    .then(response => {
      // If the file in cache then return it without connecting to FastAPI and network
      if (response) {
        console.log("FROM CACHE:", event.request.url);
        return response;
      }

      // If not then go and take it from network
      console.log("Trying network:", event.request.url);
      return fetch(event.request)
        .then(response => {
          console.log("Network SUCCESS:", event.request.url);
          return response;
        })
        .catch(error => {
          console.log("Network FAILED:", event.request.url, error.message);
          return new Response("You are offline and the resource is not cached.", {
            status: 503,
            statusText: "Service Unavailable",
            headers: new Headers({ "Content-Type": "text/plain" })
          });
        });
    })
);
```

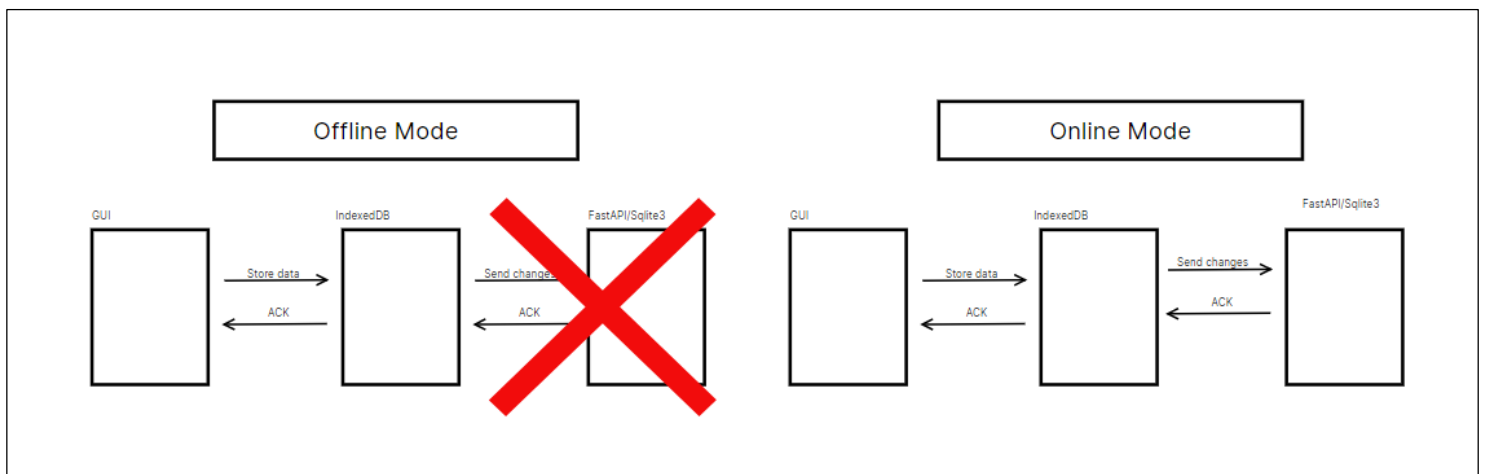
➤ Web Application Manifest

The **Web Application Manifest** defines the basic information and configuration of AgroSnap as a **Progressive Web App (PWA)**. It specifies properties such as the application name, icons, start URL, and display mode, allowing the application to be installed and launched as a **standalone web application** on supported devices.

```
{
  "name": "AgroSnap",
  "id": "/",
  "short_name": "AgroSnap",
  "start_url": "/",
  "scope": "/",
  "display": "standalone",
  "background_color": "#ffffff",
  "orientation": "any",
  "theme_color": "#2E7D32",
  "icons": [
    {
      "src": "/static/images/agrosnap_logo192x192.png",
      "sizes": "192x192",
      "type": "image/png"
    }
  ]
}
```

```
<link rel="stylesheet" href="/static/css/general_style.css">
<link rel="manifest" href="/static/manifest.json">
<script src="/static/js/index.js" defer></script>
```

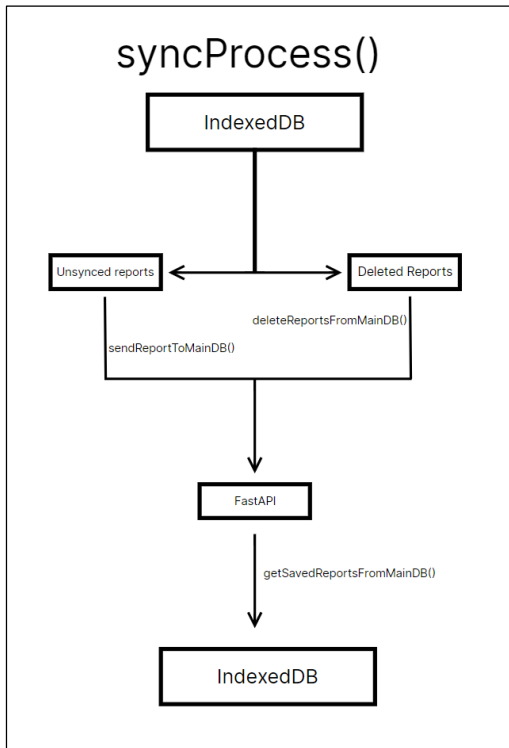
9.1.8 Data Synchronization



Data synchronization is implemented in AgroSnap to maintain consistency between the data stored locally in IndexedDB and the data stored on the server. **The synchronization mechanism handles both sending local changes to the server and retrieving server data for local storage**, which supports the application's offline functionality. The `isSynced` property is used to identify reports that have already been synchronized. The main synchronization methods include `sendReportToMainDB()` for sending reports, `getSavedReportsFromMainDB()` for retrieving server reports, `deleteReportsFromMainDB()` for **synchronizing deleted reports**, and `syncDiseases()` for retrieving disease information. These operations are coordinated by `syncProcess()`, which also compares local and server reports to maintain data consistency.

```
| | | js/
| | |   |-- about.js
| | |   |-- back_to_sign.js
| | |   |-- databaseManager_IndexedDB.js
| | |   |-- dynamic_pages.js
| | |   |-- fill-Profile-Info.js
| | |   |-- help.js
| | |   |-- index.js
| | |   |-- interactive-sidebar.js
| | |   |-- login.js
| | |   |-- registration_process.js
| | |   |-- saved-reports.js
| | |   |-- scan.js
| | |   |-- service-worker-register.js
| | |   |-- single-report.js
| | |   |-- syncReports.js
| | |   |-- tokenDecoding.js
| | |   |-- web-worker.js
```

➤ Synchronization Process



```

//checking the out of synced reports
let outOfSyncedReport=[];
let allReports = await browserDB.get_all_reports_from_indexedDB();
if(allReports.length<=0) //no reports saved yet
  console.log("No saved Reports");

//loop for sync any out of sync reports and store it in sqlite3
for(let reportNum=0;reportNum< allReports.length;reportNum++){

  //skip any synced reports , which means this is already stored in sqlite3
  if(allReports[reportNum].isSynced==true)continue;
  outOfSyncedReport.push(allReports[reportNum]);
  const isSent = await sendReportToMainDB(userCurrent,allReports[reportNum]);

  // 401 means unauthorized , so the token could be expired or not exist
  if(!isSent.success && isSent.responseStatus===401)
    return{success:false , details:isSent.details , responseStatus:isSent.responseStatus }
}
  
```

The synchronization process first retrieves all saved reports from IndexedDB and checks their synchronization status. Reports with `isSynced` set to true are skipped because they have already been stored on the server. Reports that are not synchronized are passed to `sendReportToMainDB()` to be sent to the server. If the request fails due to an unauthorized response (401), the synchronization process stops and returns the corresponding status so that the application can handle the expired or invalid authentication token.

➤ Server and Local Data Consistency

After synchronizing the local changes, the application compares the reports stored in IndexedDB with the current reports retrieved from the server. This comparison is needed because a report that was previously synchronized may later be deleted from the server, for example, when the same account is used on another device. In this case, the report would still exist in the local IndexedDB and would become outdated. Therefore, `creatSetOfReports()` is used to collect the IDs of the server reports and compare them with the local reports. If a report is marked as synchronized but its ID is no longer found on the server, the local copy is `deleted using delete_report()`. This ensures that outdated reports are removed from IndexedDB and that the local data remains consistent with the server.

```

let serverReports = await getSavedReportsFromMainDB(userCurrent);
let indexedDBReports = await browserDB.get_all_reports_from_indexedDB();

//check before complete the sync
if (!serverReports.success) {
  return {
    success:false,
    details:serverReports.details,
    responseStatus:serverReports.responseStatus};
}

console.log("SERVER REPORTS : " , serverReports.data);
console.log("INDEXEDDB REPORTS : " , indexedDBReports);
//send server reports [ sqlite3 reports ] and get the set of reports with ID only
let setOfReportsID = creatSetOfReports(serverReports.data);

//compare the differences of indexedDB reports and sqlite3 reports
for (let reportIndex =0;reportIndex<indexedDBReports.length;reportIndex++){
  let this_report = indexedDBReports[reportIndex]
  if(setOfReportsID.has(this_report.save_id))continue;
  if(this_report.isSynced===true) await browserDB.delete_report(this_report.save_id);
}
  
```


➤ Saved Reports Synchronization

```
export async function getSaverReportsFromMainDB(token){ //
}

async function sendReportToMainDB(token , savedReports){...
```

```
> async function deleteReportsFromMainDB(token,reportID){ ...
}

> async function storeInIndexedDB(allSavedReportsInDB) { ...
}

> function markedReportisSynced(perReport){ ...
}
```

The synchronization of saved reports works in both directions. The `sendReportToMainDB()` method sends locally stored reports that have not yet been synchronized to the server and, when the operation succeeds, the report is marked as synchronized. In the opposite direction, `getSaverReportsFromMainDB()` retrieves the user's reports from the server, while `markedReportisSynced()` prepares the returned data with **isSynced set to true** before it is stored in IndexedDB.

➤ Prevent Multiple Synchronization

To **prevent multiple synchronization processes** from running simultaneously, the application stores the currently running synchronization operation in `syncPromise`. If another **synchronization request is triggered while a process is already running**, the **existing promise is returned** instead of starting a new process. This prevents overlapping synchronization operations and unnecessary duplicate requests.

```
let syncPromise = null; //to store the current sync process here to prevent multi call
//The actual sync method
> async function syncProcess() { ...
}
//this to prevent calling the sync method many time at the same time , so the sync meth
> async function syncReports(){ ...
}

//this method to check if any report has been deleted by other devices for the same acc
> function creatSetOfReports(serverReports){ ...
}
```

➤ Deleted Reports Synchronization

The synchronization process also handles reports that were deleted while the application was offline. The client first retrieves the report IDs stored in the `deleted_reports` object store and sends each ID to the server through `deleteReportsFromMainDB()`. If the deletion is **successful**, the report **is removed from the local database**. The implementation also handles the case where a report was created and deleted while offline; since it may not exist on the server, **the server returns a 404 response**, after which the corresponding deletion record is removed locally.

```
//check if there any deleted reports stored in delete table
let allDeletedReports = await browserDB.get_all_deleted_reports();

for(let reportNum =0 ; reportNum<allDeletedReports.length; reportNum++){

    let isDeleted = await deleteReportsFromMainDB(userCurrent,allDeletedReports[reportNum].save_id);

    /*=====
    This when the user create and delete the report in offline mode , so this deleted report will not reach the
    server , so when fastAPI try to find it in sqlite3 it will not be found to delete , so just
    delete it from indexedDB in delete table
    =====*/

    if(!isDeleted.success && isDeleted.responseStatus === 404){
        await browserDB.deleteReportPermanently(allDeletedReports[reportNum].save_id);
        // return{success:false , details:isDeleted.details , responseStatus:isDeleted.status }
    }
}
```


9.2 Back-End Database Implementation

The Purpose of the implementation section is to explain how the code operates, execution, and manages the database in real-time.

➤ Implementation Document Structure

1. System Overview & Technology Stack

- **Language & Framework: Python, FastAPI**
- **Database: SQLite3 (database.db)**
- **Authentication: PyJWT, Passlib/Bcrypt (Password Hashing)**
- **Validation: Pydantic schemas**
- **Environment: pycharm**

2. Project & Module Structure

Map out the project directory and explain the role of each file:

📁 Directory Structure & Key Modules		
FILE NAME	DESCRIPTION	TYPE
└─ <code>database.py</code>	<i>SQLite connection & session management</i>	Database
└─ <code>database.db</code>	<i>Persistent SQLite database file</i>	Data Store
└─ <code>schemas.py</code>	<i>Pydantic models & password validation logic</i>	Validation
└─ <code>auth.py</code>	<i>Authentication routes & JWT generation/decoding</i>	Security
└─ <code>report.py</code>	<i>Protected report management endpoints</i>	API Router
└─ <code>sync.py</code>	<i>Offline PWA sync endpoint (/diseases)</i>	API Router
└─ <code>main.py</code>	<i>FastAPI entry point & CORS configuration</i>	Core

9.2.1 Database layer sqlite3

This section details the database implementation using **SQLITE3**, focusing on schema setup, tables relationships, and key constrain (such as primary key , foreign key) that maintain the data integrity , it is also explain the Object-Oriented Design patterns, custom helper functions, and context managers used to manage database connections and safely handle execution errors.

9.2.1.1 context manager pattern: is implementation using python's **with** statement it 's manages the database connection dynamically-opening them and closing them automatically when operation or methods complete .

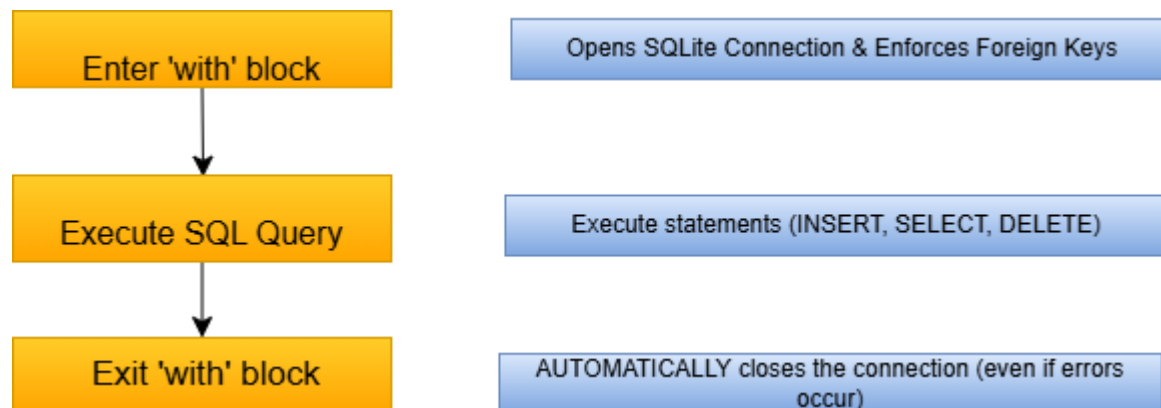
```
336
337 query = ("SELECT user_id, username, first_name, last_name, Email, PASSWORD_HASH FROM users_info "
338          " WHERE username =? or email =? ")
339
340
341 try:
342     with self._get_connection() as conn:
343         cursor = conn.cursor()
344         cursor.execute(query, parameters(identifier, identifier))
345         row = cursor.fetchone()
346         # return dictionary not tuple
347         if row:
348             return {
349                 "user_id": row[0],
350                 "username": row[1],
351                 "first_name": row[2],
352                 "last_name": row[3]
```

What the problem it solves

Without context manager, database connection must be opened and closed manually, if exception occurs mid-execution, **conn.closed()** is skipped .leading to locked database files and memory leaks.

How with **self. _get connection () as conn:**

When using the with keyword, is like manager wrap the operation inside this function automatic lifecycle manager.



Also it enforces Resource safety:

- **Just-In-Time (JIT) Connections:** Connections exist only for the precise duration of the query, preventing idle resource usage.
- **Leak Prevention:** If `cursor.execute()` fails (e.g., duplicate username), execution jumps directly to `except`, but the `with` block guarantees the connection is closed first.

How the transaction control handling?

is implemented using Python's explicit `conn.commit()` calls and SQLite's default transaction behavior inside `try-except` blocks.

How They Work in Code:

Commit (Saving Changes Permanently) saves all pending database modifications (such as `INSERT`, `UPDATE`, or `DELETE`) to the physical `.db` disk file. `conn.commit()` is explicitly called right after `cursor.execute()` inside the `with` block:

```
336
337
338 query = ('SELECT user_id, username, first_name, last_name, Email, PASSWORD_HASH FROM users_info "
339         " WHERE username =? or email =? ")
340
341 try:
342     with self._get_connection() as conn:
343         cursor = conn.cursor()
344         cursor.execute(query, parameters (Identifier, Identifier))
345         row = cursor.fetchone()
346         # return dictionary not tuple
347         if row:
348             return {
349                 "user_id": row[0],
350                 "username": row[1],
351                 "first_name": row[2],
352                 "last_name": row[3],
353                 "Email": row[4],
354                 "PASSWORD_HASH": row[5]
355             }
```

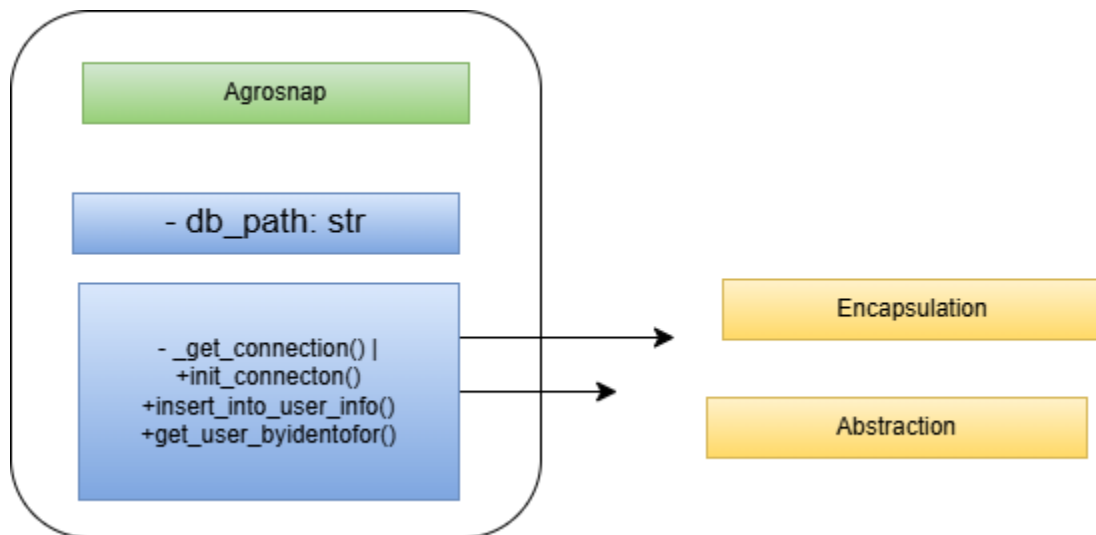
Rollback (Undoing Failed Operations):

A **Rollback** cancels any new queries made during an uncommitted transaction, and reverts the database to the state before the failed operation, rollback occurs via **SQLite's automatic connection cleanup**. Using `except`

```
285
286
287 with self._get_connection() as conn:
288     cursor = conn.cursor()
289     cursor.execute(query, parameters (email,))
290     return cursor.fetchall()
291 except sqlite3.Error as e:
292     return {"status": "error", "message": f"Database error: {e}"}
293 except Exception as e:
294     return {"status": "error", "message": f"Database error: {e}"}
```

9.2.1.2 Object-Oriented Programming (OOP) Implementation:

The **AgrosnapDatabase** class is designed around core Object-Oriented Principles to encapsulate data management, promote code reusability, and isolate database interactions from the rest of the application.



Encapsulation (Hiding Internal Mechanics)

Encapsulation bundles data (attributes) and methods that operate on that data into a single class, while restricting direct access to internal components.

```
17
18     def _get_connection(self): 15usages &user
19         # this def return connection with database.db file , conn is the object of connection
20         # this connection factory also this '_' mean the def is privet you can't call from API
21         # PRAGMA this :
22         conn = sqlite3.connect(self.db_path)
23         conn.execute('PRAGMA foreign_keys = ON')
24         return conn
```

Abstraction (Simplifying Complex Interfaces)

Abstraction have complex internal SQL queries and connection handling behind simple, clean method calls. The FastAPI application layer calls readable methods like **get_user_by_identifier()** or **insert_into_users_info()** without needing to write raw SQL or manage database cursors.

Dictionary data mapping:

Methods like above we convert the low-level SQLite tuple outputs into structured python dictionary that abstract the database format and ready for Json format.



```
309 def get_user_by_id(self, user_id: int) -> dict: &user*
310
311 query = "SELECT user_id , username, first_name ,last_name, Email ,PASSWORD_HASH FROM users_info WHERE user_id = ?;"
312
313 try:
314     with self._get_connection() as conn:
315         cursor = conn.cursor()
316         cursor.execute(query, parameters=(user_id,))
317         row = cursor.fetchone()
318         if row:
319             return {
320                 "user_id": row[0],
321                 "username": row[1],
322                 "first_name": row[2],
323                 "last_name": row[3],
324                 "Email": row[4],
325             }
326         return None
327
328 except sqlite3.Error as db_error:
329     print(f"Database Error: {db_error}")
330     raise db_error
331
332
```

Single Responsibility & Method Reusability

each method within the class has a clear single responsibility, prevent the code duplication across database operation. Like function read the physical files from the disk, then call in database insertion like

self.insert_into_diseas_table().

Database Error handling

- **Method 1: Status Dictionaries:** The database function handles error completely inside itself, it catches the SQLITE error, print a log and return a dictionary to caller.



```
return cursor.fetchall()
except sqlite3.Error as e:
    return {"status": "error", "message": f"Database error: {e}"}
except Exception as e:
    return {"status": "error", "message": f"Database error: {e}"}
```

➤ Method 2: Exception Propagation (`raise`):

When fetching critical data (like user profiles during login), returning a simple dictionary might be dangerous or awkward. Instead, your code catches the error to print a log, but then uses `raise` to **pass the error up to FastAPI**.

```
        return None
    except sqlite3.Error as db_error:
        print(f"Database error: {db_error}")
        raise db_error
```

How fastApi use it:

methods catch database-level errors, log technical details internally, and return sanitized dictionary status payloads to prevent internal schema exposure. Critical identity lookups re-raise caught exceptions, enabling FastAPI to translate low-level persistence failures into appropriate HTTP status codes (400, 404, 409, 500).

Password Hashing & Encryption Strategy (**Bcrypt**):

User passwords are never stored in plain text inside the `users_info` table. Instead, the system enforces non-reversible, salted password hashing using the **Bcrypt** algorithm (via the `bcrypt` Python module).

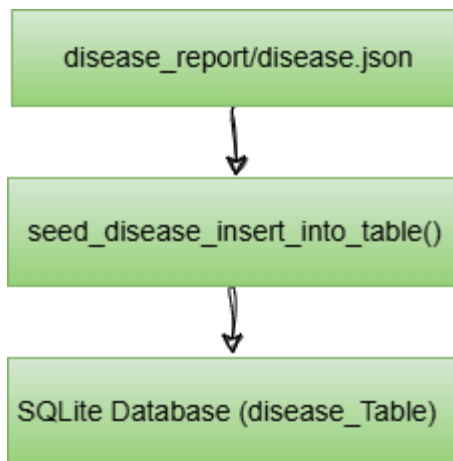


- **No Plaintext Storage:** Raw passwords are never saved in the `users_info` table.
- **Unique Salting (`bcrypt.gensalt()`):** Generates a random 128-bit salt for each user so identical passwords produce completely different database hashes, neutralizing rainbow table attacks.
- **Bcrypt Hashing (`bcrypt.hashpw()`):** Uses an adaptive cost factor (Eksblowfish algorithm) that makes brute-force cracking attempts computationally expensive and slow.
- **Constant-Time Verification (`bcrypt.checkpw()`):** During `/login`, the server hashes the input password with the stored salt and compares it in constant time, preventing timing side-channel attacks.

How we store data in database:

Data Seeding & Ingestion Pipeline (`disease.json`):

To initialize the backend database with disease descriptions, organic treatments, and diagnostic reports, the system uses a **JSON-driven seeding pipeline**. Pre-defined disease knowledge is structured inside `disease.json` and loaded into the SQLite `disease_Table` via `seed_disease_insert_into_table()`.



```
{
  "disease_name": "early blight",
  "organic_treatment": "Spray copper soap or Bacillus subtilis. Prune lower branches to improve airflow and apply mulch to prevent",
  "report": "Concentric rings (\"target spots\") with yellow halos starting on older, lower foliage"
},]
```

Path Resolution

```
base_dir = Path(__file__).resolve().parent.parent
json_path = base_dir / "disease_report"/"disease.json"

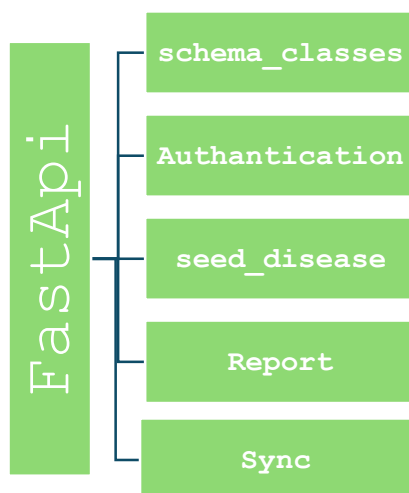
# Build reliable paths using os.path.join
db_path = os.path.join(base_dir, "database.db" )
```

9.2 .2 FastAPI Application & Router Setup

This layer details the implementation of the web framework using FastAPI. It explains how server endpoints are organized into modular routers (such as authentication, reporting, data synchronization, and database seeding) and how Pydantic schema models validate incoming client requests and format outgoing API responses.

Modular Router Architecture

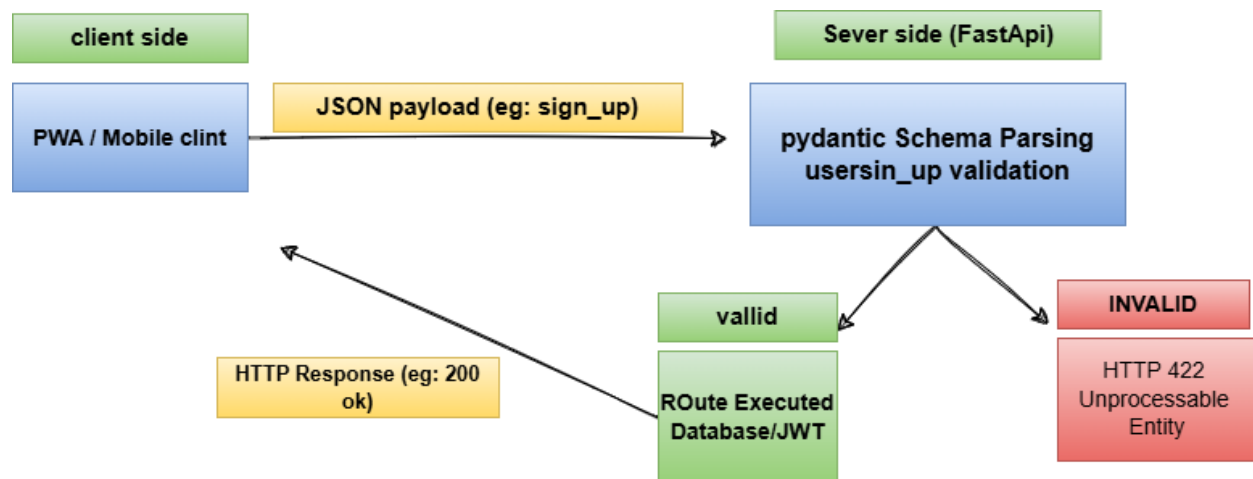
FastAPI's **APIRouter** enables Separation of Concerns by decoupling the global web server from individual feature modules. Instead of placing every HTTP endpoint inside a single monolith file, the backend splits routes into isolated sub-applications based on domain responsibility: **Authentication** (`auth`), **Reports** (`report`), **Offline Sync** (`sync`), and **Database Seeding** (`seed_disease`).



Schema_classes:

Request & Response Validation Layer (Pydantic Schemas):

The backend uses **Pydantic BaseModel** classes to enforce strict data contracts, perform payload parsing, and execute automatic request validation before controller logic runs.



How Schemas Interact Between Client and Server:

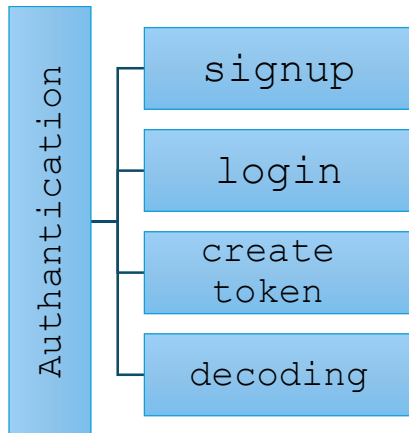
- **Client-Side Contract (API Interface):** The Pydantic classes define the expected JSON request body structure. Frontend developers know precisely which fields, data types, and required formats must be sent in API calls.
- **Automatic Server-Side Validation:** When a request hits a FastAPI route, FastAPI deserializes the incoming JSON body into the specified Pydantic model. If a field type is wrong (e.g., sending an integer instead of an `EmailStr`), FastAPI automatically rejects the request with an `HTTP 422 Unprocessable Entity` before executing route logic.

```
class UserSign_Up(BaseModel): 2 usages  👤 user

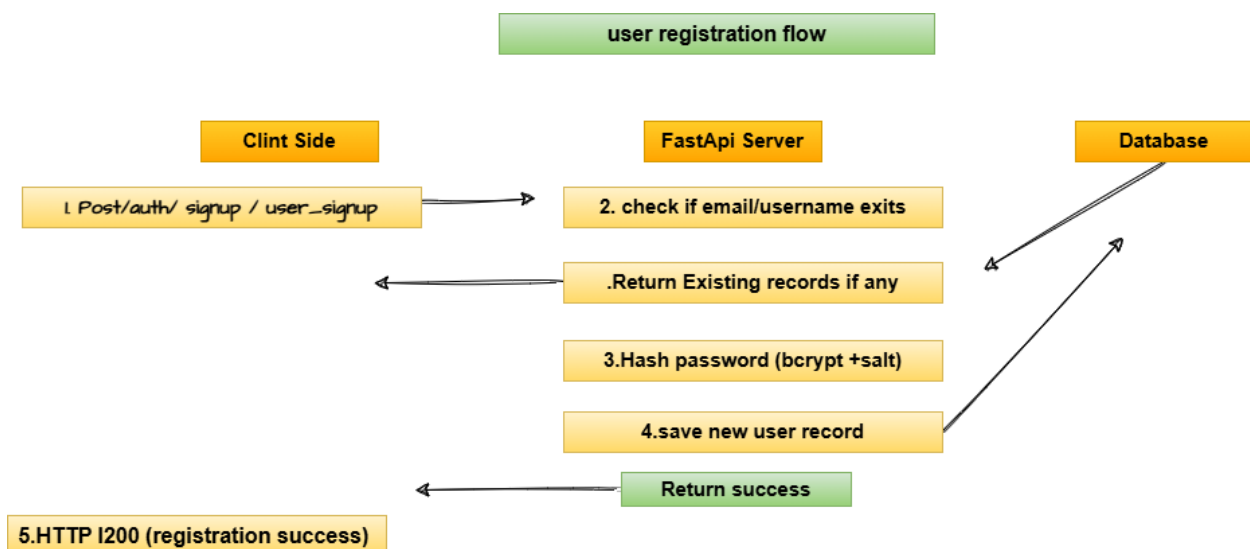
# pydantic class that will tell Aseel what data to be passed
username: str
first_name: str
last_name : str
email : EmailStr
PASSWORD_HASH: str
confirm_password: str
```

Authentication:

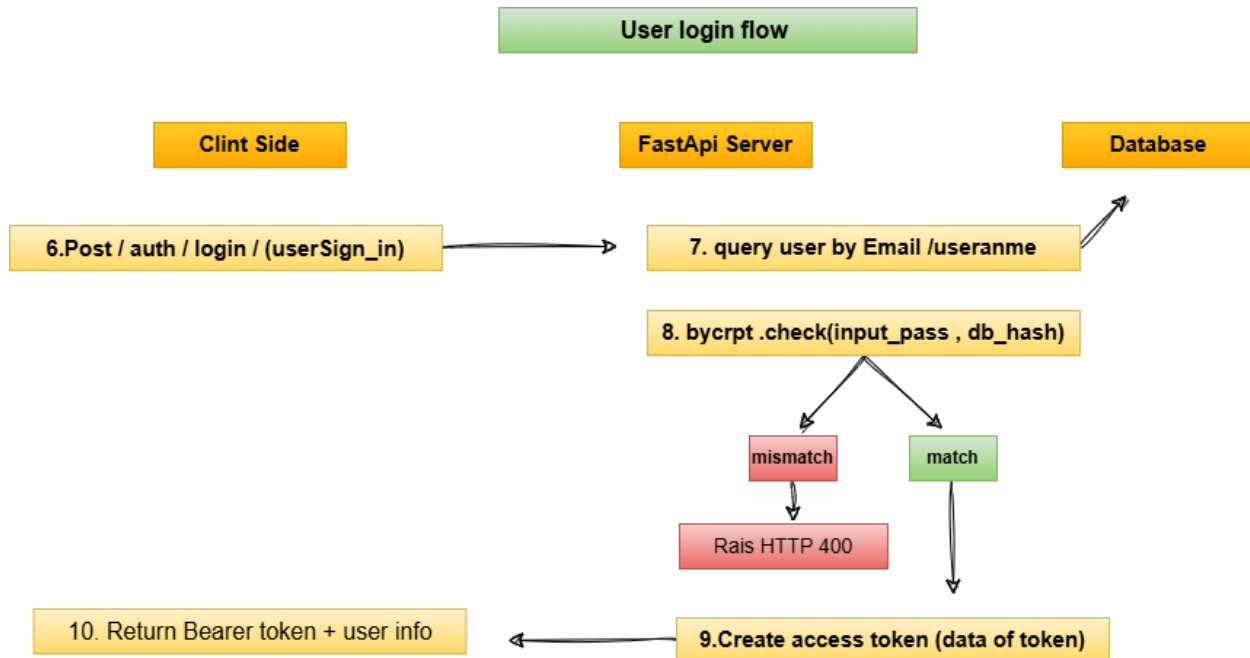
This authentication module manages user identity verification and account provisioning within the system. It enforces data validation rules during user registration to ensure payload integrity, validates credentials during login against secure data stores, and governs session lifecycle management to maintain secure user access across application boundaries.



➤ SignUp Registration flow :



➤ Login flow:



SignUp(user_data: UserSign_Up):

Purpose: Registers a new user account, verifies uniqueness, hashes the raw password, and saves the user record to the database.

Input Payload: `UserSign_Up` Pydantic model (`username`, `first_name`, `last_name`, `email`, `PASSWORD_HASH`).

Workflow:

1. Calls `database_instance.get_user_by_email()` and `get_user_by_username()` to ensure no existing user shares the same identifier. If found, raises an `HTTPException(400 Bad Request)`.
2. Converts the plain-text password string into UTF-8 bytes (`password.encode('utf-8')`).
3. Executes `bcrypt.gensalt()` and `bcrypt.hashpw()` to generate a secure password hash.
4. Inserts the user record into the SQLite `users_info` table via `database_instance.insert_into_users_info()`.

login(login_data: userSign_in):

Purpose: Authenticates returning users, verifies credentials against database hashes, and issues a JWT access token upon success.

Input Payload: `userSign_in` Pydantic model (`identifier` [email or username], `password`).

Workflow:

1. Queries the database using `get_user_by_identifier()` to look up the account by either email or username.
2. If the user does not exist, raises an `HTTPException(400 Bad Request)`.
3. Converts both the user-entered password and stored database hash into byte arrays.
4. Verifies the password using `bcrypt.checkpw()`.
5. Instantiates a `Data_of_Token` payload and delegates token creation to `create_access_token()`.
6. Returns a bearer token payload alongside non-sensitive user profile details (`first_name`, `last_name`).

Algorithm:

- **Function:** `bcrypt.checkpw(login_password, password_hash_db)`
- **Mechanic:** Extracts the salt embedded within the stored hash string, hashes the incoming `login_password` using that salt, and compares the results.
- **Security Guarantee:** Uses a **constant-time byte comparison** algorithm to prevent **timing side-channel attacks** (preventing attackers from guessing passwords based on how fast the server responds).

```
# check if password that user enter matching what was sorted in database
if not bcrypt.checkpw(login_password, password_hash_db):
    raise HTTPException(status_code=400, detail=f"Invalid Email /UserName or Password ")
```

Create_access_token(token_payload: Data_of_Token):

Purpose: Generates a signed, stateless JSON Web Token (JWT) containing user identification claims and an expiration timestamp.

Input Payload: Data_of_Token Pydantic object (user_id, username, Email).

Workflow:

1. Converts the Pydantic schema model into a standard Python dictionary using `token_payload.model_dump()`.
2. Calculates the token expiration time by adding `ACCESS_TOKEN_EXPIRE_MINUTES` (60 minutes) to the current UTC timestamp (`datetime.now(timezone.utc)`).
3. Appends the "exp" claim to the payload dictionary.
4. Encodes and signs the token using `jwt.encode()`.

Algorithm:

- **Algorithm:** HS256 (HMAC using SHA-256).
- **Mechanic:** A symmetric cryptographic hash function that combines the JWT Header, Payload, and `SECRET_KEY` (`SUPER_SECRET_KEY...`) to produce a digital signature.
- **Security Guarantee:** Guarantees token integrity—if a client alters their `user_id` inside the token payload, the signature check fails during decoding.

```
data_to_but_in_token = token_payload.model_dump()
print(data_to_but_in_token)

# compute expire time of JWT depending on UTC
expire = datetime.now(timezone.utc) + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)

#merge expire time with token
data_to_but_in_token.update({"exp": expire})

# encode the token
create_encod_token = jwt.encode( data_to_but_in_token, SECRET_KEY, algorithm=ALGORITHM)

return create_encod_token
```

Token_decoding(data_token: str = Body(..., embed=True))

Purpose: Validates, unpacks, and verifies incoming client tokens to extract authenticated identity claims.

Input Payload: Raw JWT string embedded within the HTTP request body.

Workflow:

- Passes the raw token string, `SECRET_KEY`, and `ALGORITHM` to `jwt.decode()`.
- Cryptographically verifies the signature against `SECRET_KEY`.
- Checks whether the current time exceeds the embedded "exp" timestamp.
- Unpacks and returns the extracted user fields (`user_id`, `username`, `Email`, `first_name`, `last_name`).

Exception & Edge Case Handling:

- **`jwt.ExpiredSignatureError`**: Caught when the token age exceeds 60 minutes. Converts to **HTTP 401 Unauthorized** ("Token has expired!").
 - **`jwt.JWTError`**: Caught when signature verification fails due to token tampering or invalid formatting. Converts to **HTTP 400 Bad Request** ("Invalid Token").
-

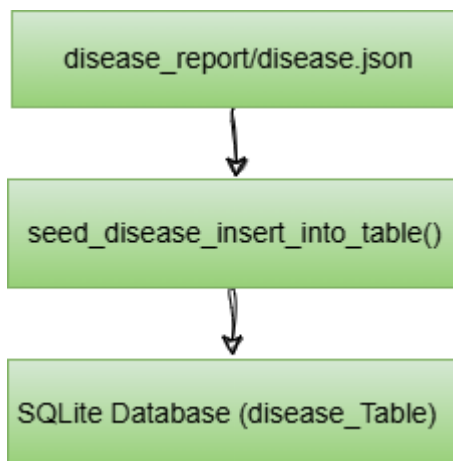
Seed_disease:

Technical of storing data in databases Sqlite3

Approach-sorting all the disease data in a central **`disease.json`** array and iterating through it with Python's built-in **`json`** module.

To initialize the backend database with disease descriptions, organic treatments, and diagnostic reports, the system uses a **JSON-driven seeding pipeline**. Pre-defined disease knowledge is structured inside **`disease.json`** and loaded into the SQLite **`disease_Table`** via

`seed_disease_insert_into_table()`.



Key technical:

➤ Structured Data Format (disease.json):

Store disease records as an array of **json object** , key attribute include **disease_name**, **organic_treatment** and **report**.

```
{
  "disease_name": "Early Blight",
  "organic_treatment": "Treating early blight is all about managing the fungus and keeping it from splashing back onto the leaves.",
  "report": "Early blight is a fungal disease. It is most commonly caused by the fungus Alternaria solani. Despite its name, early blight can affect both tomatoes and potatoes."
},
{
  "disease_name": "Late Blight",
  "organic_treatment": "Late blight is caused by the fungus Phytophthora infestans. It is characterized by dark, necrotic lesions on the leaves and stems. Organic treatments include removing infected plants, using fungicides, and ensuring proper drainage and air circulation.",
  "report": "Late blight is a serious fungal disease that can cause significant damage to potato and tomato crops. It is caused by the fungus Phytophthora infestans. The disease is characterized by dark, necrotic lesions on the leaves and stems. Organic treatments include removing infected plants, using fungicides, and ensuring proper drainage and air circulation."
}
```

➤ Path Resolution:

this technical ensure paths resolve correctly across different operating system .

```
base_dir = Path(__file__).resolve().parent.parent
json_path = base_dir / "disease_report"/"disease.json"
```

➤ Safe Database Seeding & transition:

- Uses `json.load()` to parse the entire array into memory.
- Iterates through each item and uses **parameterized queries (?)** to insert data into `disease_Table`.

```
# 2. Read and parse the JSON file
with open(json_path , "r" , encoding="utf-8") as file:
    diseases_data = json.load(file)
```

```
cursor.execute(
    """
    INSERT INTO disease_Table (disease_name, organic_treatment, report)
    VALUES (?, ?, ?)
    """
    for disease in diseases_data:
        cursor.execute(
            """
            INSERT INTO disease_Table (disease_name, organic_treatment, report)
            VALUES (?, ?, ?)
            """
            , (disease['disease_name'], disease['organic_treatment'], disease['report'])
        )
    cursor.commit()
```

Reports Module Implementation:

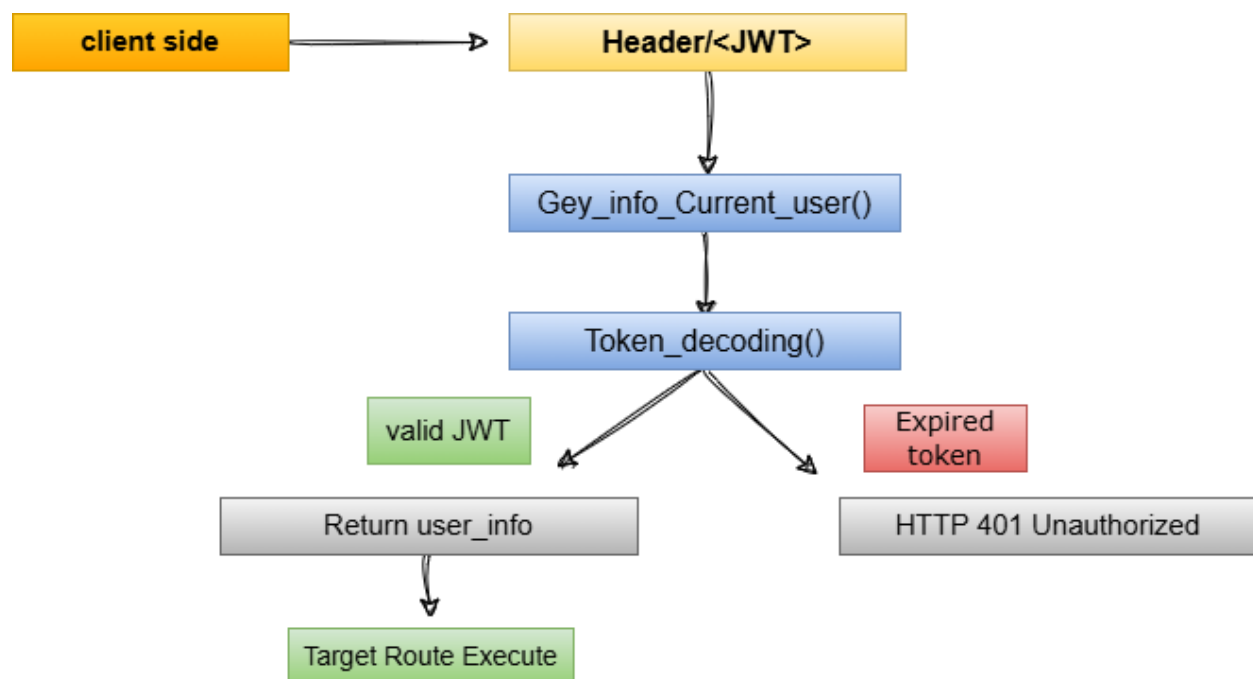
The reports module manages user diagnostic histories, allowing authenticated users to store, retrieve, and delete plant disease identification scans. Access control across these endpoints is enforced using a centralized FastAPI dependency.

Authentication Dependency (`get_info_current_user`): Before a request reaches any report route, FastAPI executes the `get_info_current_user` dependency. This acts as an authorization guard that extracts, decodes, and validates bearer tokens from incoming HTTP headers.

```
# this depends function by this def we get the data of User via "token_decoding"
def get_info_current_user(credentials : HTTPAuthorizationCredentials = Depends(security) ): 3 use

    # this will return the token
    token = credentials.credentials

    try:
        user_info= Token_decoding(token)
        return user_info
    except Exception :
        raise HTTPException(status_code=401, detail="Invalid or expired token")
```



workflow:

1. **Header Extraction:** Utilizes FastAPI's `HTTPAuthorizationCredentials = Depends(security)` to extract the raw bearer token string from the `Authorization` request header.
 2. **Payload Decoding:** Passes the token to `Token_decoding()`, which cryptographically checks the signature (HS256) and expiration (`exp`).
 3. **Identity Injection:** Returns the decoded payload dictionary (`user_id`, `username`, `email`) directly to the target endpoint, or raises an `HTTP 401 Unauthorized` exception if verification fails.
-

Save scan Report Endpoint:

Save

This endpoint allows an authenticated user to save a plant disease diagnosis scan directly to their account history.

Endpoint Specifications:

URL Path & security& HTTP Method:

```
@router.post("/save")  @ user
def Save_scan_report(request_data : SaveReportRequest , user_info : dict = Depends(get_info_current_user)) -> dict:
```

Request Schema:

```
# this schema for what data should send to save report
class SaveReportRequest(BaseModel): 2 usages  @ user +1
    save_id:str
    disease_id: int #this is the index of disease that store already in database
    plant_name :str
    confidence : float
    created_at: datetime
```

Workflow & processing logic

1. **User Identification:** Extracts the authenticated user's `user_id` from the injected `user_info` dependency dictionary.

```
# get the user_id from the token we had decode
user_id = user_info.get("user_id")
```

2. **Fallback Value Assignment:** Evaluates the incoming `plant_name` field. If omitted by the user, it defaults to "Disease Report".
3. **Data Formatting:** Formats the AI detection floating-point confidence score to a clean 2-decim representation (`f"{request_data.confidence:.2f}"`).
4. **Connect with database:** Connects to `database.db` and executes a parameterized `INSERT INTO Save_report` query to prevent SQL injection:

```
try:
    query = "INSERT INTO Save_report(save_id,user_id ,disease_id ,plant_name ,confidence , created_at ) VALUES (?,?,,?,?,,?)"
```

User Scan History Endpoint (GET /report)

My reports

This endpoint retrieves all saved diagnostic reports for the currently authenticated user, joining individual user scan logs with the global disease reference catalog to construct complete detailed reports.

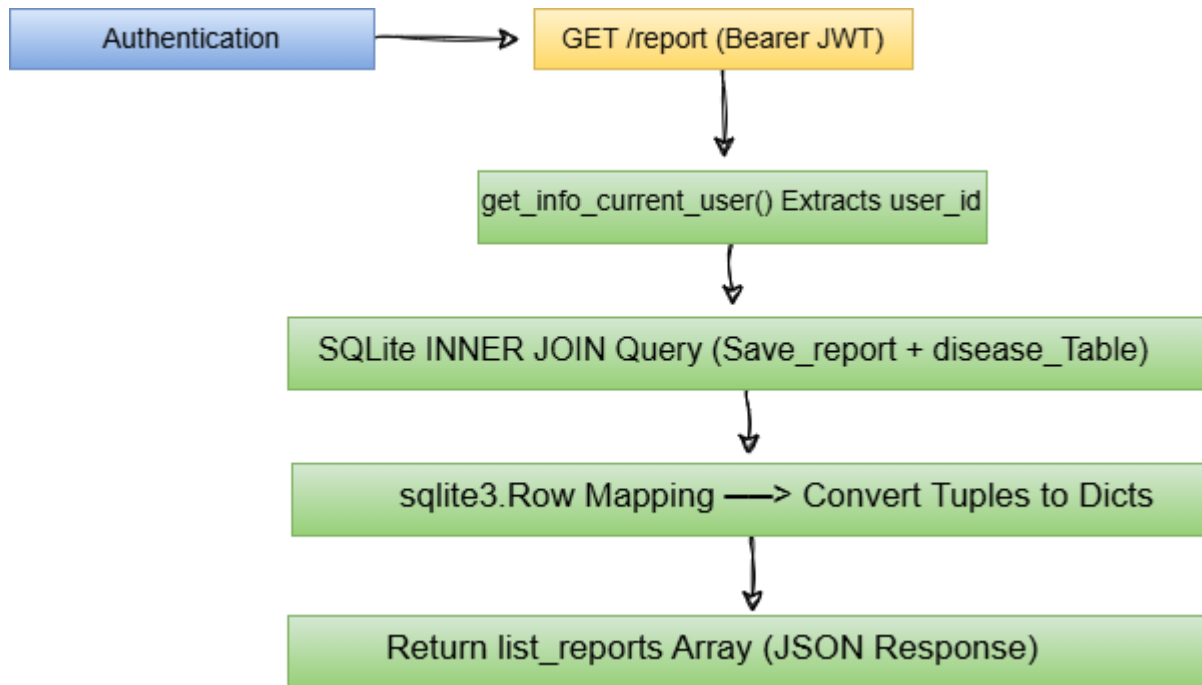
Endpoint Specifications:

URL Path & security & HTTP Method:

```
@router.get("/report")  @user
def my_Report(user_info : dict = Depends(get_info_current_user)) ->list
```

Return Type: `list[dict]` (Array of detailed report JSON objects)

```
{
  "save_id": row["save_id"],
  "plant_name": row["plant_name"],
  "confidence": row["confidence"],
  "disease_id": row["disease_id"],
  "created_at": row["created_at"],
  "disease_name": row["disease_name"],
  "organic_treatment": row["organic_treatment"],
  "disease_description": row["disease_description"]
}
```



Workflow:

1. **User Scope Isolation:** Extracts `user_id` from the JWT token so users can only retrieve their own saved reports.
2. **Relational Mapping (INNER JOIN):** Combines `Save_report` scan metadata with `disease_Table` details (`disease_name`, `organic_treatment`, `disease_description`) via `disease_id`.
3. **Dictionary Serialization (sqlite3.Row):** Converts raw SQL tuples into structured Python dictionaries using `conn.row_factory` for clean JSON serialization.
4. **Resource Safety:** Runs inside a `try-except-finally` block to handle `sqlite3.Error` exceptions and guarantee connection closing.

Delete Saved Report Endpoint (DELETE /delete-report) :

This endpoint allows an authenticated user to permanently remove a specific saved report from their personal history while enforcing strict ownership verification.

Endpoint Specifications:

URL Path & security& HTTP Method:

```
# Function to let users delete their own saved report
@router.delete("/delete-report/{save_id}") @user
def delete_save_report(
```

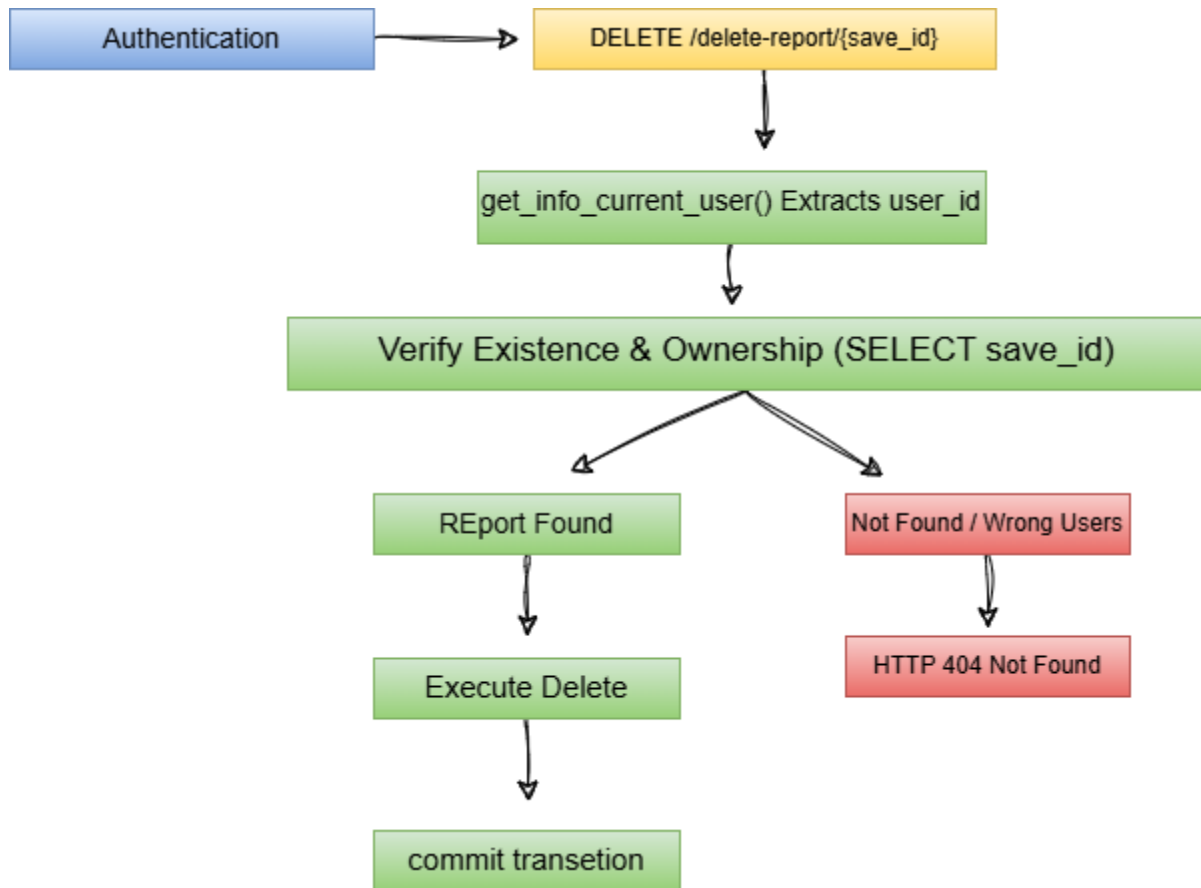
Path Parameter: `save_id` (string UUID representing the report to be deleted)

Security Guard: Depends (`get_info_current_user`)
(Requires valid JWT Bearer token)

```
def delete_save_report(
    save_id: str,
    user_info: dict = Depends(get_info_current_user)
) -> dict:
    # 1. Extract the user_id from the authenticated token
    user_id = user_info.get("user_id")
```

Return Type: `dict` (Status and confirmation message):

```
return {
    "status": "success",
    "message": f"Successfully deleted report with ID {save_id}"
}
```



Workflow:

1. The route first executes a parameterized `SELECT` query matching both `save_id` and `user_id`. This double-clause check ensures users cannot delete reports that belong to another account.

```
check_query = "SELECT save_id FROM Save_report WHERE save_id = ? AND user_id = ?"
cursor.execute(check_query, parameters: (save_id, user_id))
report = cursor.fetchone()
```

2. If no matching record is found, the route raises an `HTTP 404 Not Found` exception ("Report not found to delete it"), halting execution before any deletion query runs.

```
# If the report doesn't exist
if not report:
    raise HTTPException(
        status_code=404,
        detail="Report not found to delete it"
    )
```

3. **Atomic Deletion & Rollback:** Deletion is performed via `DELETE FROM Save_report WHERE save_id = ? AND user_id = ?`. If successful, `conn.commit()` persists the deletion. If a database error occurs, `conn.rollback()` restores state, and an HTTP 500 Internal Server Error is raised.

```
# 4. Execute the deletion query
delete_query = "DELETE FROM Save_report WHERE save_id = ? AND user_id = ?"
cursor.execute(delete_query, parameters: (save_id, user_id))
conn.commit()
```

4. **Connection Lifecycle Safety:** Database execution is wrapped in a `try-except-finally` block to guarantee that `conn.close()` executes regardless of success or failure.

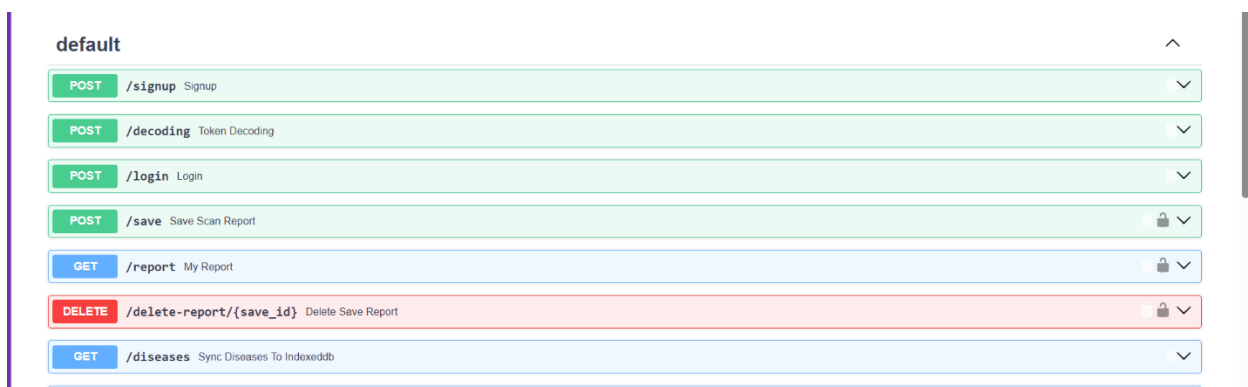
```
except sqlite3.Error as db_error:
    conn.rollback()
    raise HTTPException(status_code=500, detail=f"Database error: {str(db_error)}")

finally:
    conn.close()
```

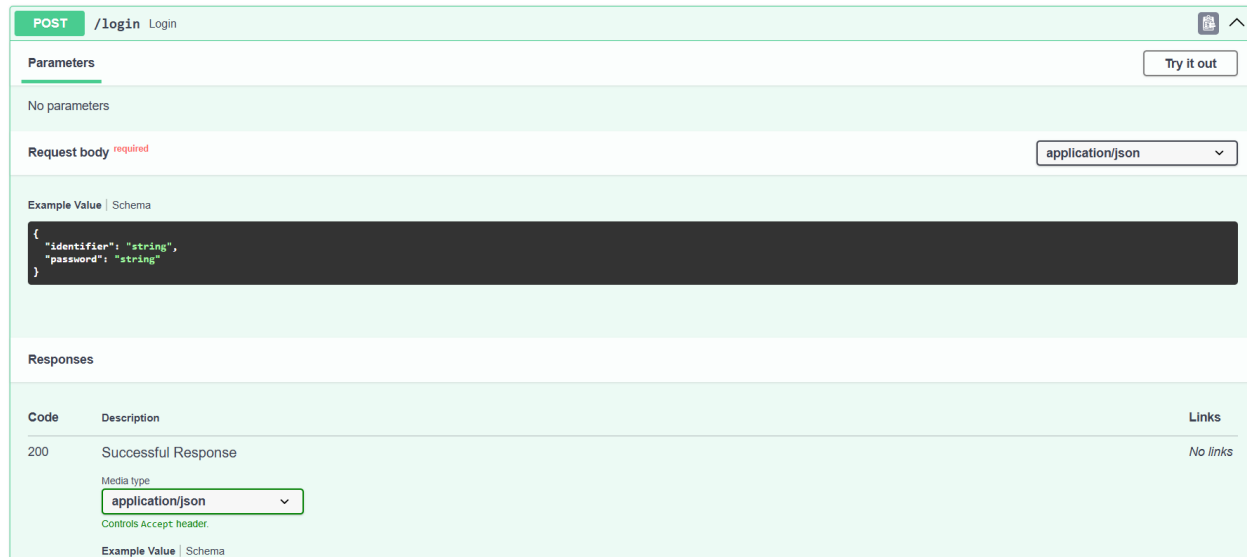
API Testing, Tooling & Interactive Documentation

To guarantee reliability, streamline client integration, and simplify backend validation, the system incorporates automated interactive documentation alongside manual endpoint testing workflows.

Swagger UI Interface (/docs): Provides an interactive web console for exploring every exposed route, path parameter (e.g., `{save_id}`), and request/response schema in real time.



Integrated Authentication Support: Includes a native **Authorize** dialog button. Testers paste their JWT token once, and Swagger automatically injects the `Authorization: Bearer <token>` header into subsequent request execution flows.



The image shows the Swagger UI for the `POST /login` endpoint. The interface is light green with a dark green header bar. The header bar contains the method `POST` and the path `/login` with a sub-label `Login`. On the right of the header bar are icons for a document, a lock, and a chevron. Below the header bar, there are sections for **Parameters**, **Request body**, **Example Value**, and **Responses**. The **Parameters** section shows "No parameters". The **Request body** section is marked as "required" and has a dropdown menu set to `application/json`. Below this is a dark grey box containing a JSON example:

```
{  "identifier": "string",  "password": "string"}
```

. The **Responses** section shows a table with one response: `200` with description "Successful Response". Below the table, there is a "Media type" dropdown set to `application/json` and a "Controls Accept header" link. At the bottom, there is a link for "Example Value | Schema".

POST /login Login

Parameters Try it out

No parameters

Request body *required* application/json

Example Value | Schema

```
{  "identifier": "string",  "password": "string"}
```

Responses

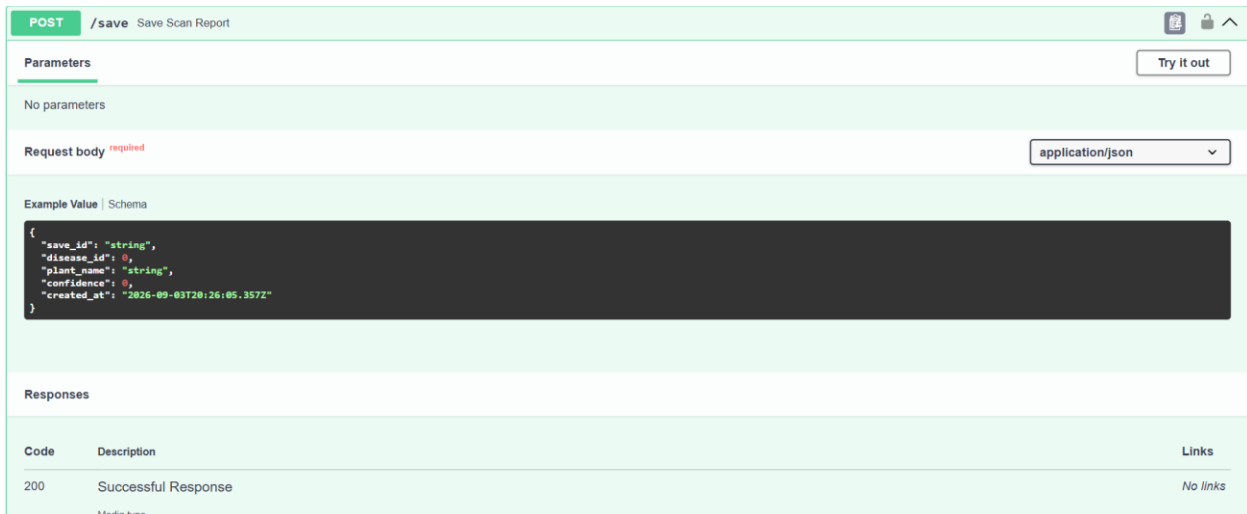
Code	Description	Links
200	Successful Response	No links

Media type application/json

Controls Accept header.

Example Value | Schema

Interactive Endpoint Execution: Developers can test end-to-end functionality—such as submitting scan reports to `POST /save` or running `DELETE /delete-report/{save_id}`—directly inside the browser without executing custom script clients.



The image shows the Swagger UI for the `POST /save` endpoint. The interface is light green with a dark green header bar. The header bar contains the method `POST` and the path `/save` with a sub-label `Save Scan Report`. On the right of the header bar are icons for a document, a lock, and a chevron. Below the header bar, there are sections for **Parameters**, **Request body**, **Example Value**, and **Responses**. The **Parameters** section shows "No parameters". The **Request body** section is marked as "required" and has a dropdown menu set to `application/json`. Below this is a dark grey box containing a JSON example:

```
{  "save_id": "string",  "disease_id": 0,  "plant_name": "string",  "confidence": 0,  "created_at": "2026-09-03T20:26:05.357Z"}
```

. The **Responses** section shows a table with one response: `200` with description "Successful Response". Below the table, there is a "Media type" dropdown set to `application/json` and a "Controls Accept header" link. At the bottom, there is a link for "Example Value | Schema".

POST /save Save Scan Report

Parameters Try it out

No parameters

Request body *required* application/json

Example Value | Schema

```
{  "save_id": "string",  "disease_id": 0,  "plant_name": "string",  "confidence": 0,  "created_at": "2026-09-03T20:26:05.357Z"}
```

Responses

Code	Description	Links
200	Successful Response	No links

Media type application/json

Controls Accept header.

Example Value | Schema

9.3 AI Model Development

9.3.1 Introduction

The AI component of AgroSnap was developed to identify plant diseases from leaf images. A deep learning image classification model was trained to classify the input image into one of six classes, including five supported plant diseases and a healthy class. To improve the training process and benefit from features learned from large-scale image datasets, transfer learning was adopted using the EfficientNetB2 architecture as the base model. The trained model was later converted to TensorFlow.js format so that it could be integrated into the client-side application and perform inference directly in the browser.

9.3.1.1 What is Transfer Learning?

Transfer learning is a machine learning technique that uses knowledge learned by a model from a large dataset and applies it to a different but related task. Instead of training a model from scratch, a pretrained model is adapted to the target problem by retraining some of its layers using the new dataset. This can reduce training time and improve performance when the available target dataset is

9.3.1.2 What is Transfer Learning?

EfficientNetB2 is a **convolutional neural network (CNN)** architecture designed for image classification. It belongs to the EfficientNet family, which improves the balance between model accuracy and computational efficiency by scaling the network's depth, width, and input image resolution in a coordinated way. EfficientNetB2 provides a suitable trade-off between classification performance and computational requirements, making it appropriate for image-based tasks such as plant disease classification.

9.3.1.3 Object Classification vs Object Detection

Object classification identifies the class or category of an object present in an image, without determining its exact location. In contrast, **object detection** identifies the object **class and also determines its location** within the image using bounding boxes.

In AgroSnap, **image classification** is used because the objective is to classify the input leaf image into one of the supported disease classes or the healthy class. The application does not need to locate the leaf or disease within the image; therefore, object detection is not required.

9.3.1.4 Why EfficientNetB2?

EfficientNetB2 was selected for AgroSnap because it provides a suitable balance between classification performance and computational efficiency. Compared with larger and more computationally demanding models, its relatively lightweight architecture makes it more suitable for running directly in the browser, where computational and memory resources are more limited than in server-based environments. This contributes to faster model loading and inference, which is important for maintaining a responsive user experience. In addition, the model can be converted to TensorFlow.js and stored through the application's caching mechanism, allowing it to be loaded and used locally without continuously depending on a server connection. Therefore, EfficientNetB2 supports the application's PWA and offline objectives while providing the required performance for plant disease classification.

9.3.2 Dataset Preparation

9.3.2.1 Dataset Organization

The dataset was organized into six classification classes, where each class represents a specific plant disease or the healthy condition. The classes include **Bacterial Spot**, **Early Blight**, **Healthy**, **Late Blight**, **Leaf Mold**, and **Yellow Leaf Curl Virus**. The images were organized into separate folders according to their corresponding class, allowing the dataset to be used directly for supervised image classification. The dataset was further divided into separate **training** and **testing** sets, with the same six classes represented in each set. This organization ensured that the model could learn the visual characteristics associated with each disease during training and later evaluate its classification performance on unseen test images.

```
dataset/
├── archive/
│   ├── train/
│   │   ├── bacterial_spot
│   │   ├── early_blight
│   │   ├── healthy
│   │   ├── late_blight
│   │   ├── leaf_mold
│   │   └── yellow_leaf_curl_viruse
│   └── test/
│       ├── bacterial_spot
│       ├── early_blight
│       ├── healthy
│       ├── late_blight
│       ├── leaf_mold
│       └── yellow_leaf_curl_viruse
```

9.3.2.2 Number of Images per Class

class	Train	Test
Bacterial spot	1,837	459
Early Blight	1,690	464
Healthy	2,049	455
Late Blight	1,901	450
Leaf Mold	1,721	479
Yellow Leaf Curl Virus	1,798	517

The dataset contains a total of **13,820 images** distributed across six classification classes. The training set contains **10,996 images**, while the testing set contains **2,824 images**. Each set includes the same six classes: Bacterial Spot, Early Blight, Healthy, Late Blight, Leaf Mold, and Yellow Leaf Curl Virus. The number of images across the classes is relatively similar, providing a reasonably balanced distribution and reducing the risk of the model being strongly biased toward a particular class. The training set was used to learn the visual characteristics of each class, while the separate testing set was reserved for evaluating the model on unseen images.

9.3.2.3 Data Preprocessing and Augmentation

1. Image Resizing

```
val_ds = tf.keras.utils.image_dataset_from_directory(  
    TRAIN_DIR,  
    validation_split=0.2,  
    subset="validation",  
    seed=123,  
    image_size=IMAGE_SIZE,  
    batch_size=BATCH_SIZE,  
    label_mode='categorical'  
)
```

```
IMAGE_SIZE = (260,260)  
BATCH_SIZE = 16  
TRAIN_DIR = "/content/datas
```

All input images were resized to **260 × 260 pixels** as part of the preprocessing step before being provided to the EfficientNetB2 model.

3. Data Augmentation and Batch

```
# Augmentation  
data_augmentation = tf.keras.Sequential([  
    layers.RandomFlip("horizontal_and_vertical"),  
    layers.RandomRotation(0.3),  
    layers.RandomZoom(height_factor=(-0.2, 0.2), width_factor=(-0.2, 0.2), fill_mode='reflect'),  
    layers.RandomContrast(0.1),  
    layers.RandomBrightness(factor=0.1, value_range=(0, 255)),  
])
```

Data augmentation was applied to the training images using random **flipping, rotation, zooming, contrast**, and brightness adjustments to improve data diversity and reduce overfitting. The images were processed in batches of **16** during training to manage computational and memory requirements.

```
IMAGE_SIZE = (260,260)  
BATCH_SIZE = 16  
TRAIN_DIR = "/content/datas
```

2. Validation Split

```
Found 10991 files belonging to 6 classes.  
Using 8793 files for training.  
Found 10991 files belonging to 6 classes.  
Using 2198 files for validation.  
Found 2824 files belonging to 6 classes.
```

```
val_ds = tf.keras.utils.image_dataset_from_directory(  
    TRAIN_DIR,  
    validation_split=0.2,  
    subset="validation",  
    seed=123,  
    image_size=IMAGE_SIZE,  
    batch_size=BATCH_SIZE,  
    label_mode='categorical'  
)
```

To **evaluate the model during training**, the training dataset was further divided into training and validation subsets. A **20% validation split** was applied to the original training directory, leaving 80% of the images for **training and allocating 20%** for validation. The validation set was not used to update the model's weights; instead, it was used to monitor the model's performance during training and guide the selection of the best-performing model. The same random seed was used for both subsets to ensure that the training and validation images were consistently divided.

9.3.3 Model Architecture

9.3.3.1 Pretrained EfficientNetB2

```
# call the model
base_model = tf.keras.applications.EfficientNetB2(
    input_shape=(260, 260, 3),
    include_top=False,
    weights='imagenet'
)
base_model.trainable = False #no need of modification of the base weights
```

EfficientNetB2 was used as the base model with pretrained weights from **ImageNet**. The model was configured with an input size of **260 × 260 × 3**, while its original classification layer was removed using **include_top=False** so that custom classification layers could be added for **AgroSnap's** six classes. During the initial training stage, the EfficientNetB2 base layers were frozen to preserve the pretrained features while the newly added classification layers were trained for the plant disease classification task.

9.3.3.2 Custom Classification

```
# collect all info above and use it to create the model structure
model = models.Sequential([
    data_augmentation,
    base_model,
    layers.GlobalAveragePooling2D(),

    layers.Dense(256),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.Dropout(0.4),

    #last layer to take a decision
    layers.Dense(num_classes, activation='softmax',
                 #add new layer and this will be detect the disease of plants
                 kernel_regularizer=tf.keras.regularizers.l2(0.001))
])
```

After the EfficientNetB2 base model, additional layers were added to adapt the extracted features to AgroSnap's six-class classification task. **GlobalAveragePooling2D** reduces the feature maps into a compact feature representation. A Dense layer with **256 units** then learns task-specific features, followed by **BatchNormalization** to stabilize the training process and **ReLU** activation to introduce non-linearity. A **Dropout rate of 0.4** was applied to reduce overfitting by randomly disabling some neurons during training.

The final Dense layer contains **six units**, corresponding to the six classification classes. The **Softmax** activation converts the outputs into probabilities for each class, while **L2 regularization (0.001)** helps reduce overfitting by penalizing large model weights.

9.3.4 Model Training

9.3.4.1 Initial Training / Warm-up

```
base_model.trainable = False #no need of modification of the base weights

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.0001),
    loss=tf.keras.losses.CategoricalFocalCrossentropy(gamma=2.0),
    metrics=['accuracy']
)

#we just training the new layer
history_warmup = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=10,
    callbacks=[checkpoint_callback, early_stopping]
)
```

In the initial training stage, the pretrained **EfficientNetB2** base model was kept frozen, while only the newly added classification layers were trained. The model was compiled using the **Adam optimizer** with a learning rate of **0.0001** and **Categorical Focal Crossentropy** as the loss function. The model was initially trained for up to **10 epochs**, using the validation set to monitor its performance. **EarlyStopping** was also applied to stop the training when the validation loss stopped improving and restore the best-performing weights.

9.3.4.2 Fine-Tuning

```
base_model.trainable = True

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.00001), #adam is an algorithm
    loss=tf.keras.losses.CategoricalFocalCrossentropy(gamma=2.0),
    metrics=['accuracy']
)

history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=50,
    callbacks=[checkpoint_callback,early_stopping_for_the_whole_model]
)
```

After the initial training stage, **fine-tuning** was performed by making part of the pretrained EfficientNetB2 layers trainable. **Most of the earlier layers remained frozen to preserve the general visual features learned from ImageNet**, while the later layers were **allowed to adapt to the plant disease classification task**. Batch Normalization layers were kept frozen to maintain stable feature representations during fine-tuning. A smaller learning rate of **0.00001** was used to make gradual updates to the pretrained weights and avoid losing the previously learned features. The model was then trained for up to **50 epochs** using the same loss function and the Adam optimizer, with Early Stopping used to restore the best-performing weights.

9.3.4.3 Training Results

1. Results During the Fine-Tuning

```
Epoch 29: finished saving model to /content/drive/MyDrive/AgroSnapModel_EMB2.keras
550/550 — 151s 176ms/step - accuracy: 0.9794 - loss: 0.0146 - val_accuracy: 0.9704 - val_loss: 0.0192
Epoch 30/50
550/550 — 0s 139ms/step - accuracy: 0.9815 - loss: 0.0139
Epoch 30: val_accuracy did not improve from 0.97043
550/550 — 90s 164ms/step - accuracy: 0.9812 - loss: 0.0141 - val_accuracy: 0.9650 - val_loss: 0.0205
Epoch 31/50
550/550 — 0s 137ms/step - accuracy: 0.9817 - loss: 0.0139
Epoch 31: val_accuracy did not improve from 0.97043
550/550 — 88s 161ms/step - accuracy: 0.9790 - loss: 0.0142 - val_accuracy: 0.9645 - val_loss: 0.0202
Epoch 32/50
550/550 — 0s 138ms/step - accuracy: 0.9822 - loss: 0.0140
Epoch 32: val_accuracy did not improve from 0.97043
550/550 — 142s 161ms/step - accuracy: 0.9816 - loss: 0.0137 - val_accuracy: 0.9695 - val_loss: 0.0199
Epoch 33/50
550/550 — 0s 135ms/step - accuracy: 0.9807 - loss: 0.0136
Epoch 33: val_accuracy did not improve from 0.97043
550/550 — 141s 160ms/step - accuracy: 0.9815 - loss: 0.0136 - val_accuracy: 0.9672 - val_loss: 0.0192
Epoch 34/50
550/550 — 0s 136ms/step - accuracy: 0.9837 - loss: 0.0128
Epoch 34: val_accuracy did not improve from 0.97043
550/550 — 89s 161ms/step - accuracy: 0.9828 - loss: 0.0131 - val_accuracy: 0.9691 - val_loss: 0.0195
```

The training results during the fine-tuning stage show that the model maintained a high training accuracy, ranging from approximately **97.9% to 98.3%** across **epochs 31–34**. The validation accuracy remained between **96.45% and 96.95%**, while the validation loss remained relatively low. These results indicate that the model maintained stable performance during the later stages of fine-tuning, although the validation accuracy did not improve beyond the previously recorded best value of **97.04%**.

3. Evaluation Result

```
test_loss, test_accuracy = model.evaluate(test_ds)
print(f"Accuracy test result is: {test_accuracy * 100:.2f}%")

... 177/177 — 15s 82ms/step - accuracy: 0.9646 - loss: 0.0223
Accuracy test result is: 96.46%
```

The **trained model was evaluated using a separate test dataset** containing previously unseen images. The model achieved a **test accuracy of 96.46%** with a **test loss of 0.0223**. The high test accuracy indicates that the model was able to correctly classify the majority of unseen images across the six supported classes.

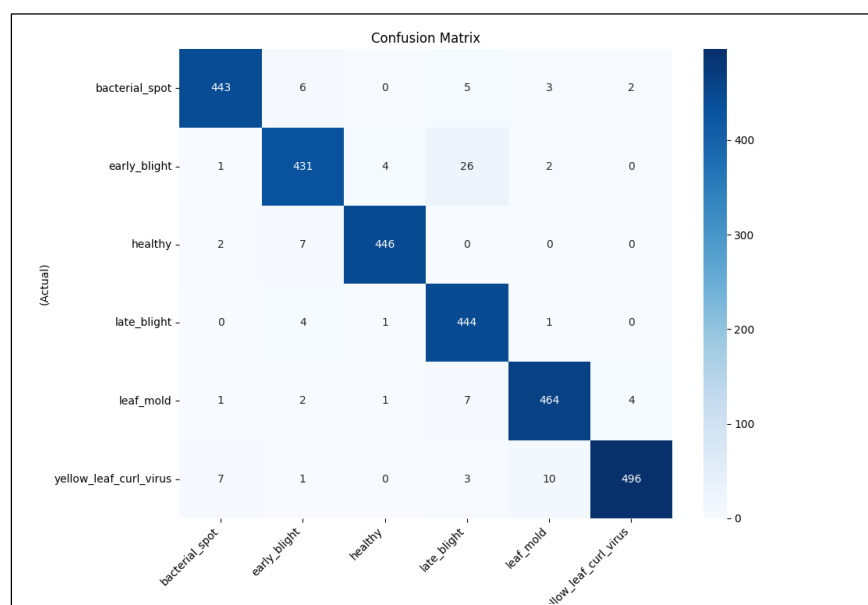
2. Classification Performance

```
=== The report ===
```

	precision	recall	f1-score	support
bacterial_spot	0.98	0.97	0.97	459
early_blight	0.96	0.93	0.94	464
healthy	0.99	0.98	0.98	455
late_blight	0.92	0.99	0.95	450
leaf_mold	0.97	0.97	0.97	479
yellow_leaf_curl_virus	0.99	0.96	0.97	517
accuracy			0.96	2824
macro avg	0.96	0.96	0.96	2824
weighted avg	0.97	0.96	0.96	2824

The classification performance of the trained model was further evaluated using precision, recall, F1-score, and support for each class. **Recall** was considered to assess the model's ability to correctly identify images belonging to each actual class. The model achieved recall values ranging from **93% to 99%** across the six classes. The highest recall was achieved for **Late Blight (99%)**, while the lowest was observed for **Early Blight (93%)**. The model achieved a **macro-average recall of 96%**, indicating consistent classification performance across the different classes.

4. Confusion Matrix



The confusion matrix was used to provide a detailed evaluation of the model's classification performance across the six classes. The diagonal values represent correctly classified images, while the off-diagonal values represent misclassified images. As shown in the figure, most of the predictions are concentrated along the diagonal, indicating that the model correctly classified the majority of the test images.

The highest number of correct predictions was obtained for **Yellow Leaf Curl Virus**, with **496 out of 517 images** correctly classified, followed by **Healthy**, with **446 out of 455 images** correctly classified. The most noticeable misclassification occurred between **Early Blight** and **Late Blight**, where **26 Early Blight images were classified as Late Blight**. This corresponds with the lower recall observed for **Early Blight (93%)** compared with the other classes.

9.3.5 Model Conversion to TensorFlow.js

```
!tensorflowjs_converter \
  --input_format=tf_saved_model \
  /content/agrosnap_saved_model_clean \
  /content/web_model
```

```
models/
├── web_model/
│   ├── group1-shard1of8.bin
│   ├── group1-shard2of8.bin
│   ├── group1-shard3of8.bin
│   ├── group1-shard4of8.bin
│   ├── group1-shard5of8.bin
│   ├── group1-shard6of8.bin
│   ├── group1-shard7of8.bin
│   ├── group1-shard8of8.bin
│   └── model.json
```

After training, the **EfficientNetB2** model was converted from **Keras** format to TensorFlow.js format using **tensorflowjs_converter**. The converted model consists of a **model.json** file and **binary weight shards**, allowing it to be loaded and executed directly in the browser using **TensorFlow.js**.

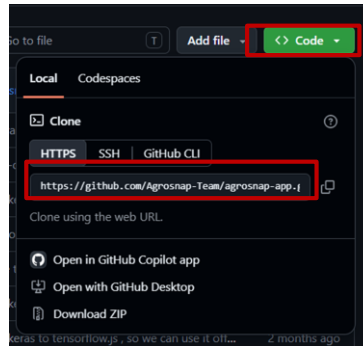
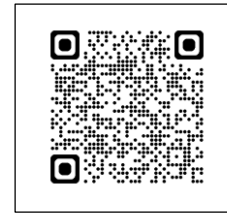
The converted model was then loaded in the browser through TensorFlow.js and used by the Web Worker to perform inference without blocking the main UI thread.

10. Installation & Start

10.1 GitHub

10.1.1 Scan QR code to go to project

10.1.2 Clone the repository



10.1.3 Open the terminal in your IDE

The project can be obtained from the GitHub repository by cloning the repository and navigating to the project directory.

```
git clone https://github.com/Agrosnap-Team/agrosnap-app.git
```

```
>> cd agrosnap-app
```

10.1.4 Install Requirements

```
> pip install -r requirements.txt
```

10.2 Open Website on the device

10.2.1 Local Server

2. Runs the FastAPI application locally on port 8080 and automatically reloads the server whenever code changes are detected.

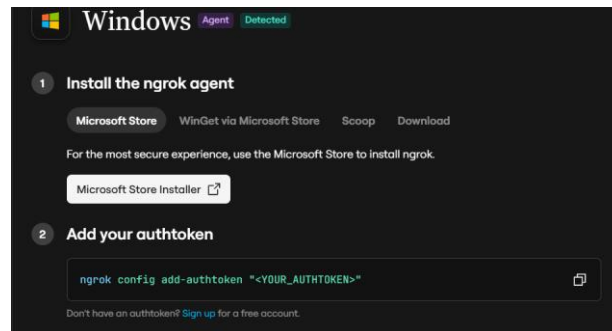
```
(.venv) PS C:\Users\aseel\PycharmProjects\agrosnap-app> uvicorn app.main:app --port 8080
```

1. Open the displayed URL in a web browser to access the application.

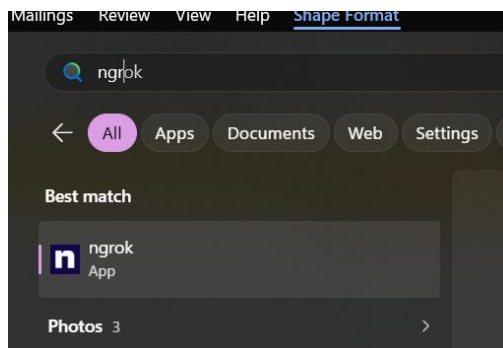
```
(.venv) PS C:\Users\aseel\PycharmProjects\agrosnap-app> uvicorn app.main:app --port 8080
Agrosnap Database initialized successfully!
Agrosnap Database initialized successfully!
INFO: Started server process [8872]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:8080 (Press CTRL+C to quit)
```

10.2.2 Ngrok Startup

Scan the QR code on the ngrok website to download ngrok, then follow the provided steps to configure and activate the authtoken.



Search about ngrok CLI on your device and open it



Then run the following command

```
ngrok is a command line application, try typing 'ngrok.exe http 80'
at this terminal prompt to expose port 80.
C:\Windows\System32>ngrok http 8080
```

NOTE : The port should be the same port which you used to run FastAPI server

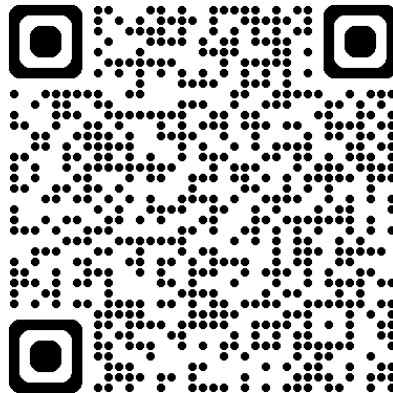
Then wait until this URL appear on the CLI , copy it or CTRL + click to open it

```
* One gateway for every AI model. https://app.ngrok.ai

Session Status      online
Account             aseel.a.kh01@gmail.com (Free)
Version             3.39.9-msix-stable
Region              Europe (eu)
Latency              135ms
Web Interface        http://127.0.0.1:4040
Forwarding            https://excursion-lego-reproach.ngrok-free.dev -> http://localhost:8080

Connections          ttl    opn    rt1    rt5    p50    p90
                     0      0      0.00   0.00   0.00   0.00
```

10.3 Open Website on Render



11. References & Sources

Dataset	1. <u>Find Open Datasets and Machine Learning Projects Kaggle</u> 2. <u>Roboflow: Computer vision tools for developers and enterprises</u>
TensorFlow	1. <u>TensorFlow.js guide</u> 2. <u>API Documentation TensorFlow v2.16.1</u>
FastAPI	1. <u>Tutorial - User Guide - FastAPI</u> 2. <u>https://www.uvicorn.org/</u>
PWA	1. <u>MDN Web Docs</u>
Deployment	1. <u>Render Dashboard</u> 2. <u>ngrok: deliver your apps, APIs, and AI on local and prod</u> 3. <u>Cloudflare Quick Tunnels</u>
Control Version & share codes	1. <u>Git</u> 2. <u>GitHub</u>

12. Conclusion

AgroSnap is a **Progressive Web Application** designed to provide users with an accessible and reliable solution for plant disease diagnosis using Artificial Intelligence. Throughout this project, we integrated several web technologies to create a system that is not only functional, but also responsive, offline-capable, and easy to use.

The application combines a **Single Page Application (SPA)** architecture with a responsive user interface, IndexedDB and Service Workers for offline functionality, and **PWA** features to provide a more app-like experience. The AI model was integrated into the browser using TensorFlow.js and Web Workers, allowing image analysis to be performed efficiently without unnecessarily blocking the user interface.

The system also supports **saving, deleting, and synchronizing** reports between the **local database and the main server database**. This allows users to continue working even when there is no Internet connection, while ensuring that their data is synchronized once connectivity is restored.

Overall, the project demonstrates how modern web technologies and Artificial Intelligence can be combined to build a practical, reliable, and user-friendly application. The final system provides a complete workflow from scanning a plant image and obtaining a diagnosis to managing saved reports, while maintaining functionality in both online and offline environments.